

natural language processing

pablo cumpián díaz

onboarding

objetivos del curso

- preprocesar y limpiar texto eficientemente
- extraer y vectorizar características
- clasificación de texto mediante modelos clásicos
- aprender modelado de tópicos y NET
- arquitectura transformer
- uso de LLMs pre-entrenados para resolver problemas de negocio
- prompt engineering

estructura del curso

1ª semana: fundamentos y preprocesado. Vectorización y modelos clásicos

2ª semana: word embeddings y análisis de sentimientos

3ª semana: modelado de tópicos y tareas de NLU

4ª semana: modelos de lenguaje y transformers

5ª semana: aplicaciones prácticas de los LLMs

bloques del curso

1º bloque: algoritmos

- mayor control sobre toda la pipeline
- funcionalidad limitada / menor presión de recursos

2º bloque: modelos de lenguaje (cajas negras) -> arquitectura

- salto de abstracción: mayor utilidad / mayor presión de recursos
- herramientas y casos de uso

un pequeño reto

ecosistema en python

- Python es el lenguaje de facto de NLP
 - Capa 1 – procesamiento clásico y lingüístico: [NLTK](#) y [spaCy](#)
 - Capa 2 – Machine Learning aplicado a texto: [scikit-learn](#), [gensim](#)
 - Capa 3 – Deep Learning y transformers: [Hugging Face Transformers](#), [PyTorch](#)
 - Capa 4 – LLMs y herramientas de alto nivel: [LangChain](#), [LlamaIndex](#); [OpenAI API](#), etc
- en este curso empezaremos desde la capa base y escalaremos progresivamente hacia los modelos modernos. Entender las capas inferiores es esencial para usar bien las superiores

materiales del curso

- presentación
- repositorio
- notebooks
- examen en la última sesión

feedback

NLP

- el Procesamiento de Lenguaje Natural es la rama de la Inteligencia Artificial que estudia como las máquinas pueden leer, comprender, interpretar y generar lenguaje humano de forma útil y significativa
 - el lenguaje humano es ambiguo, contextual y evolutivo - muy diferente al lenguaje formal de las máquinas
 - NLP se sitúa en la intersección de la lingüística, la estadística y la informática
 - no se trata sólo de de procesar texto: incluye lenguaje hablado, traducción, razonamiento sobre significado y más
 - el objetivo último es que una máquina pueda comunicarse con personas de forma natural, sin fricciones
- NLP es al lenguaje lo que la visión por computadora es a las imágenes, enseñar a las máquinas a ver lo que hay detrás de las palabras

Nivel	Pregunta clave	Ejemplo	Herramienta NLP
Morfológico estructura interna	¿Cómo está formada la palabra?	"corriendo" → corr + iendo	Stemming, lematización
Léxico categoría gramatical	¿Qué categoría gramatical tiene?	"rápido" → adjetivo	POS tagging
Sintáctico estructura de la frase	¿Cómo se relacionan las palabras?	"el gato duerme" → sujeto, verbo → objeto	Dependency parsing, constituency parsing
Semántico significado	¿Qué significa? ¿A qué se refiere?	"Madrid" → entidad: ciudad → lugar en España	NER, word embeddings, resolución de correfer.
Pragmático intención y contexto	¿Qué intención hay detrás del enunciado?	"¿Tienes hora?" → no pregunta la hora → petición implícita	Detección de intención, análisis de sentimiento

ejercicio: ¿qué hacer?

procesar, entender o generar

- Traducir un texto del inglés al español
- Detectar si un correo es spam
- Generar el resumen de un contrato
- Entender qué quiere decir un usuario cuando escribe "ponme lo de siempre"
- Responder automáticamente a una reseña negativa
- Extraer las fechas y nombres de un documento legal
- Escribir una descripción de producto a partir de sus características técnicas

NLP, NLU, NLG

término	Nombre completo	¿Qué hace?	Direccion
NLP	Natural Language Processing	Paraguas general: engloba todo el procesamiento del lenguaje	Bidireccionalidad
NLU	Natural Language Understanding	Extrae significado: intencion, entidades y contexto	Texto -> Máquina
NLG	Natural Language Generation	Produce texto comprensible para humanos a partir de datos o lógica	Máquina -> texto

Use cases

- chatbots y asistentes
- motores de búsqueda
- clasificación de documentos
- análisis de sentimiento
- generación de texto
- traducción automática
- narrativa de datos
- reconocimiento de entidades (NER)
- resolución de correferencias

breve historia del NLP

- el NLP no nació con ChatGPT: tiene más de 70 años de historia
 - años 50-60: sistemas basados en reglas
 - Alan Turing propone en 1950 el famoso test de Turing como criterio de inteligencia lingüística de una máquina
 - los primeros sistemas funcionaban con diccionarios con patrones de sustitución de texto
 - ELIZA (1966, MIT) simulaba una conversación con patrones de sustitución de texto – impresionaba, pero no entendía nada
 - años 80-90: estadística y corpus
 - el cambio de paradigma: en lugar de escribir reglas, se aprende de grandes cantidad de texto
 - aparecen los modelos de lenguaje estadísticos (n-gramas), los HMMs para etiquetado
 - la disponibilidad de corpus digitales (WSJ, Penn Treebank) acelera el progreso

breve historia del NLP

- años 2000-2010: Machine Learning clásico
 - SVM, Naive Bayes y regresión logística se aplican a clasificación de texto con resultados sólidos
 - Word2Vec (2013, Google) revoluciona la representación del lenguaje: las palabras pasan a ser vectores con significado geométrico
- años 2017-hoy: transformers y LLMs
 - el paper "Attention is All You Need" (2017, Google) introduce la arquitectura Transformer
 - BERT, GPT, TS LLaMA, Claude, los modelos crecen en tamaño y capacidad
 - el NLP pasa de ser una herramienta especializada a estar en el centro de la IA moderna

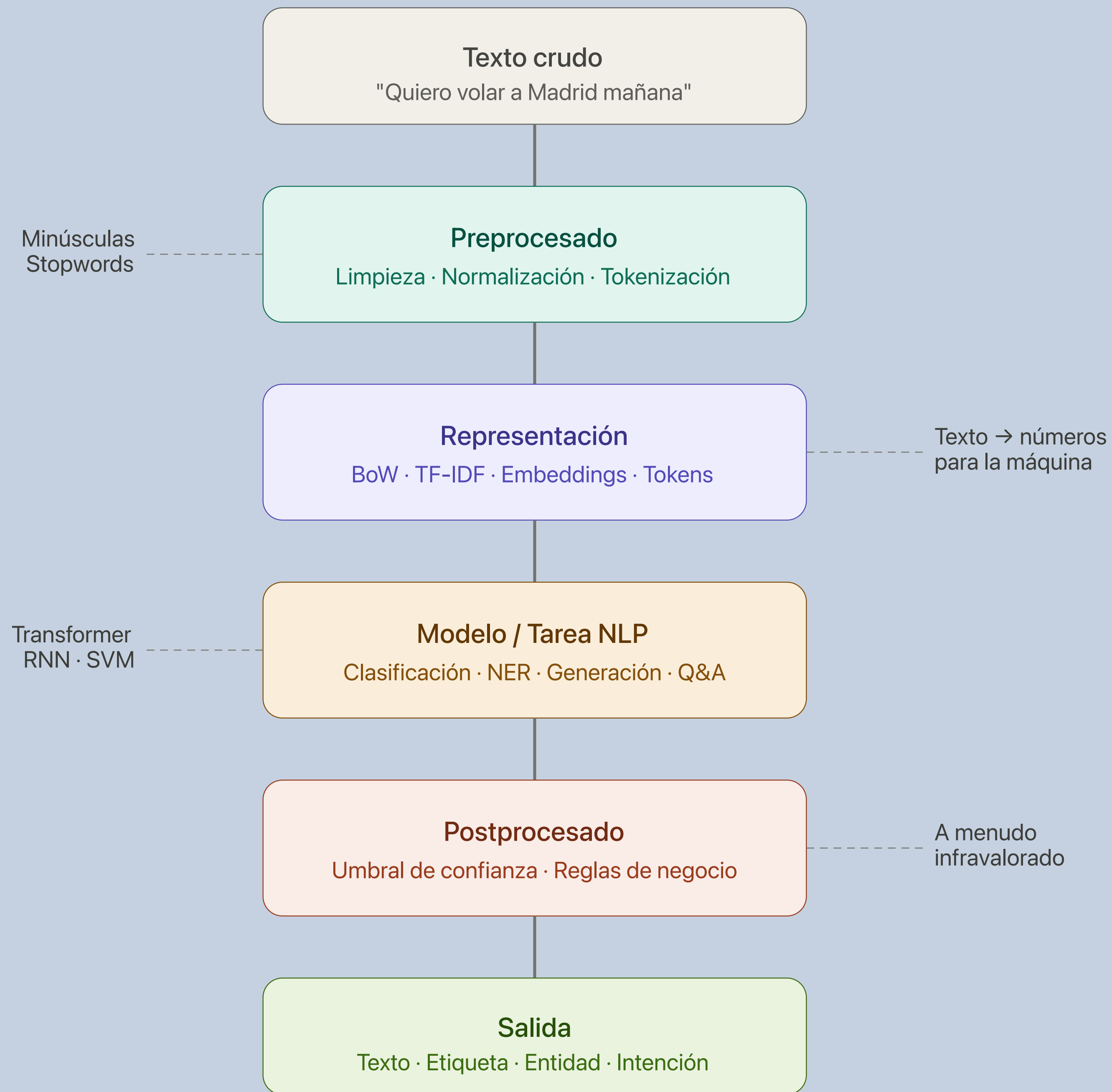
¿por qué el lenguaje natural es difícil para las máquinas?

el lenguaje humano está diseñado para humanos, no para máquinas

- ambigüedad léxica – una palabra, varios significados
- ambigüedad sintáctica
- ironía y sarcasmo
- contexto implícito y conocimiento del mundo
- idioms y expresiones hechas
- variación lingüística – dialectos, jerga, errores ortograficos, emojis, abreviaturas...

representación del texto

- primer problema del NLP: las máquinas no entienden palabras, entienden números
- antes de aplicar cualquier modelo, hay que responder: **¿cómo convertimos texto en algo computable?**
 - bag of words (contar cuántas veces aparece cada palabra)
 - TF-IDF (ponderar palabras por su importancia relativa)
 - word embeddings (cada palabra es un vector en su espacio de significado)
 - embeddings contextuales (el vector de cada palabra depende de su contexto en la frase)



el gran olvidado

el postprocesado

- es la etapa que convierte la salida “cruda” del modelo en algo que un sistema real puede usar de forma **segura y coherente**. Es probablemente la etapa más subestimada del pipeline
- los modelos NLP no devuelven decisiones – devuelven distribuciones de probabilidad, texto generado, o listas con puntuaciones. El postprocesado traduce eso en acciones concretas del sistema, con todas las garantías que un entorno de producción exige
- tareas habituales: tratamiento de **umbrales de confianza** – decisiones, normalización de entidades (“mañana” -> 15/04), resolución de conflictos, **filtros y reglas de negocio**, **formato de salida**, **detección de alucinaciones** o respuestas fuera de dominio...
- se subestima porque en los tutoriales y notebooks académicos no existe – el modelo predice y se imprime el resultado directamente. Pero en producción, ignorar el postprocesado es la causa de una gran cantidad de problemas que no caen del lado técnico sino del lado comercial o de producto

limitaciones y sesgos

- sesgos en los datos de entrenamiento
- sesgo lingüístico
- falta de sentido común y razonamiento
- alucinaciones en modelos generativos

NLP multilingüe

la mayoría del NLP asume el inglés como idioma por defecto

- desequilibrio de recursos
- estrategias para trabajar en español u otros idiomas
 - modelos multilingües – entrenados en decenas de idiomas simultáneamente: mBERT, XLM-RoBERTa
 - modelos específicos por idioma – mejores resultados al especializarse: BETO (español), CamemBERT (francés)

tareas a tratar

- clasificación: sentimiento, spam, intención
- extracción: NER, fechas, relaciones
- búsqueda y similitud: embeddings, RAG
- resumen: transformers
- Q&A: RAG aplicado
- generación: texto libre, LLMs

herramientas

- python 3.14 - conda
- preprocessing, NER: spaCy
- modelos clásicos: scikit-learn
- gensim: NLU (topic modeling)
- neural networks: pytorch
- transformers: hugging-face (transformers, datasets, evaluate), sentence-transformers

workshop: tour de sentimientos

APIs y código

- este no es un curso orientado a la programación o la enseñanza de APIs concretas
 - contexto convulso debido a la IA
 - librerías y APIs cambiantes (caso TensorFlow - PyTorch)
 - la construcción de sistemas de NLP se acercan más al despliegue de infraestructura que a la tarea artesanal de programar
- sin embargo la **ubicuidad** de muchas de las herramientas se presta a tenerlas en muy en cuenta

corpus del curso

- Twitter/X
 - vectorización
 - embeddings
 - transformers: clasificación
- Amazon customer reports
 - tópicos
 - transformers: Q&A

preprocessing | vectorización | clasificación

preprocessing | vectorización | clasificación

review del dataset

- antes de entrenar cualquier modelo, dedicamos una pequeña sesión a entender los datos. Lo que encontremos aquí condicionará todas las decisiones posteriores
- un modelo aprende exactamente lo que hay en los datos – errores incluidos
- los problemas detectados ahora cuestan minutos. Detectarlos después, semanas
- La review responde 3 preguntas:
 - ¿qué hay?
 - ¿qué falla?
 - ¿qué debería estar?

exploración inicial

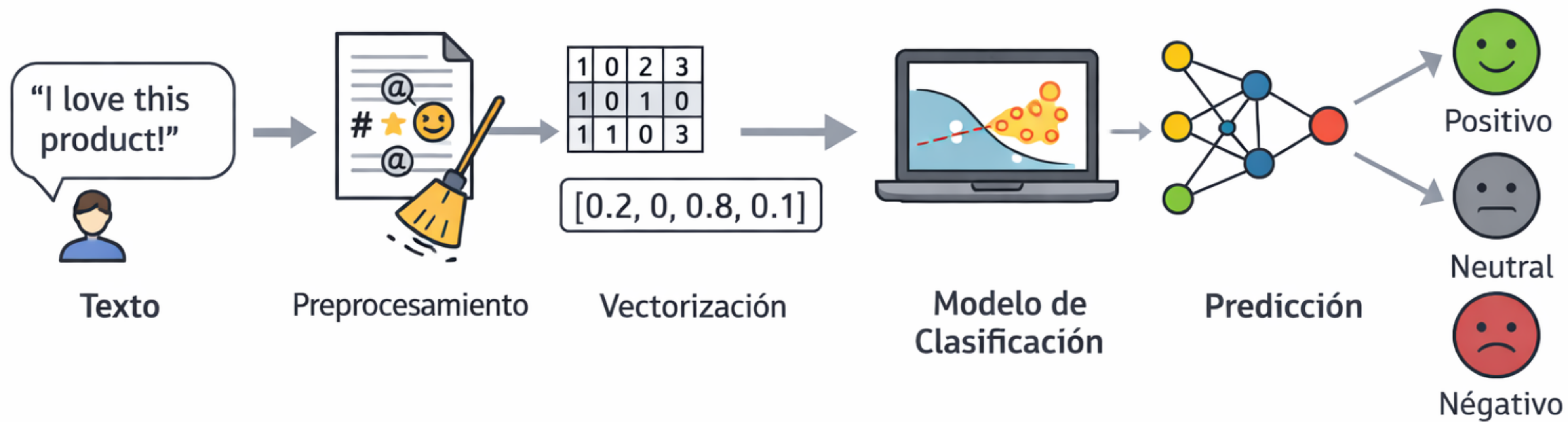
- tenemos que atender a lo siguiente:
 - el shape: nº de tweets y columnas disponibles
 - ¿hay valores vacios?
 - primera impresión del formato real de los tweets
 - ¿están equilibradas las clases?
 - ¿cuánto texto hay realmente? ¿los negativos tienden a ser más largos?
 - ¿hay tweets duplicados? mismo texto con/sin etiqueta distintas
 - ¿qué elementos propios de los tweets hay? URLs, menciones, hasttags, RTs...
 - hastags: de qué hablan los tweets
 - edge cases: tweets muy cortos, negaciones, errores de anotación o ambigüedad

identificación de issues

- desbalance de las clases
- textos vacíos
- ruido
- contradicciones
- duplicados

retos

- falta de datos
- poor-quality data
 - ejemplos no representativos
 - features irrelevantes
 - clases desbalanceadas (datasets con bias)
- ...



datasets

- Un dataset es un conjunto estructurado de datos para entrenar modelos y evaluar su desempeño
- En el caso de tareas de clasificación cada ejemplo del dataset viene con su **etiqueta** correcta
- Dividimos los datasets en:
 - **train**: para entrenamiento
 - **validation**: ajuste de hiperparámetros
 - **test**: evaluación final
- El **tamaño** de un dataset depende del fin de nuestro modelo: pequeño para prototipado y grande para modelos en producción

datasets

- Lo mejor para **escalar** tamaños es:
 - aplicar muestreo (sampling)
 - aplicar batching (lotes)
 - pipelines eficientes
 - uso de GPUs

datasets

batching

- Podemos también dividir un dataset para que cada batch ocupe lo mismo
- Con ello conseguimos:
 - no saturar recursos (RAM y GPU)
 - acelerar y estabilizar el entrenamiento
- De esta forma el entrenamiento produce una serie de incrementos o actualizaciones del modelo
 - Con un batch pequeño se produce más ruido, pero obtenemos mejor generalización
 - Con un batch grande el aprendizaje es más estable y rápido (por el startup y cooldown final entre batches)

datasets

sampling

- consiste en seleccionar subconjuntos de datos del dataset
- esto nos sirve para:
 - entrenar más rápido
 - balancear clases (evitar sobrerrepresentación y sesgar mucho el modelo)
 - crear datasets manejables
- tipos: aleatorios, estratificado, subsampling (reducción)

shuffle

```
from torch.utils.data import DataLoader  
  
loader = DataLoader(dataset, batch_size=32, shuffle=True)
```

sampler

```
from torch.utils.data import DataLoader, RandomSampler  
  
sampler = RandomSampler(dataset)  
  
loader = DataLoader(dataset, batch_size=32, sampler=sampler)
```

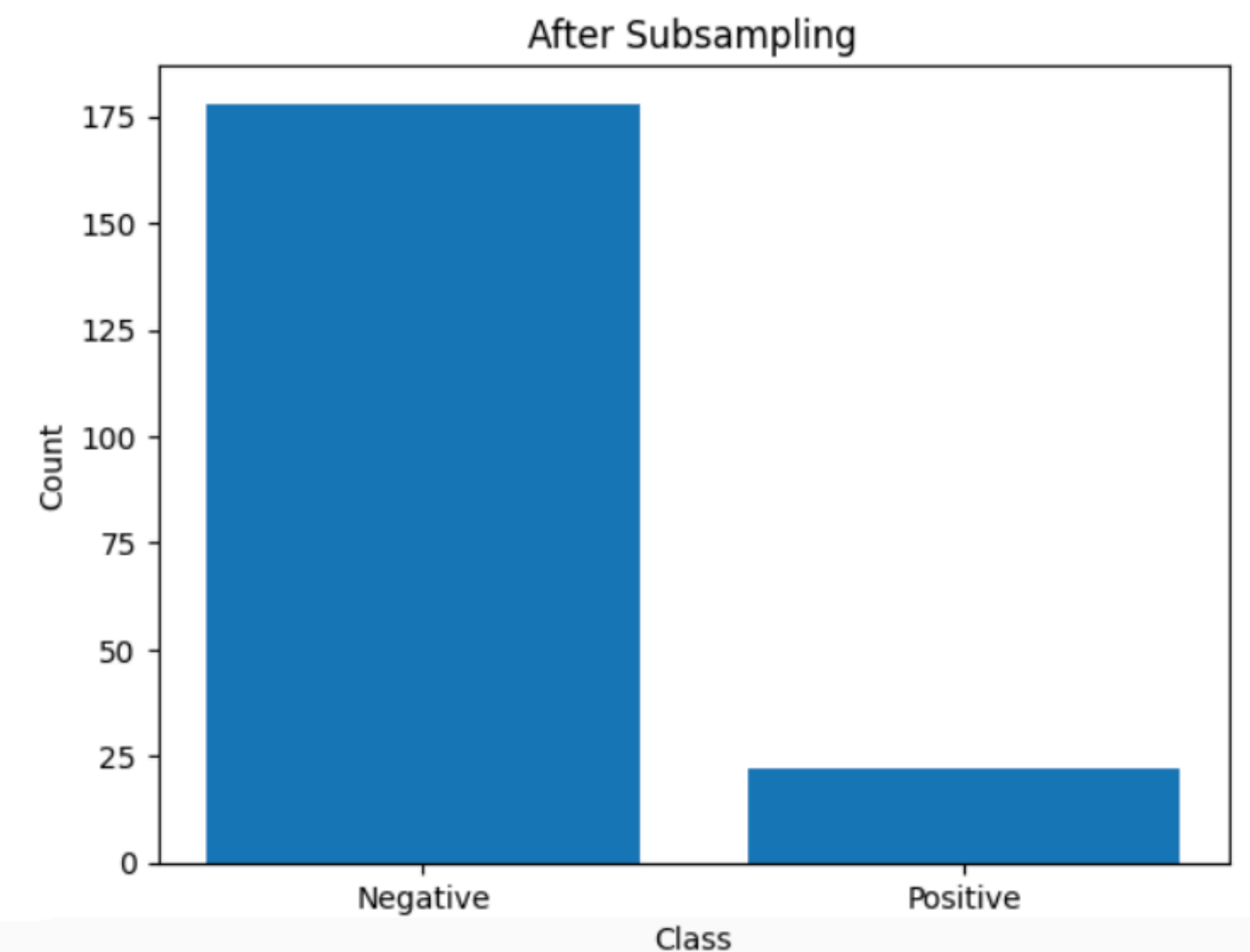
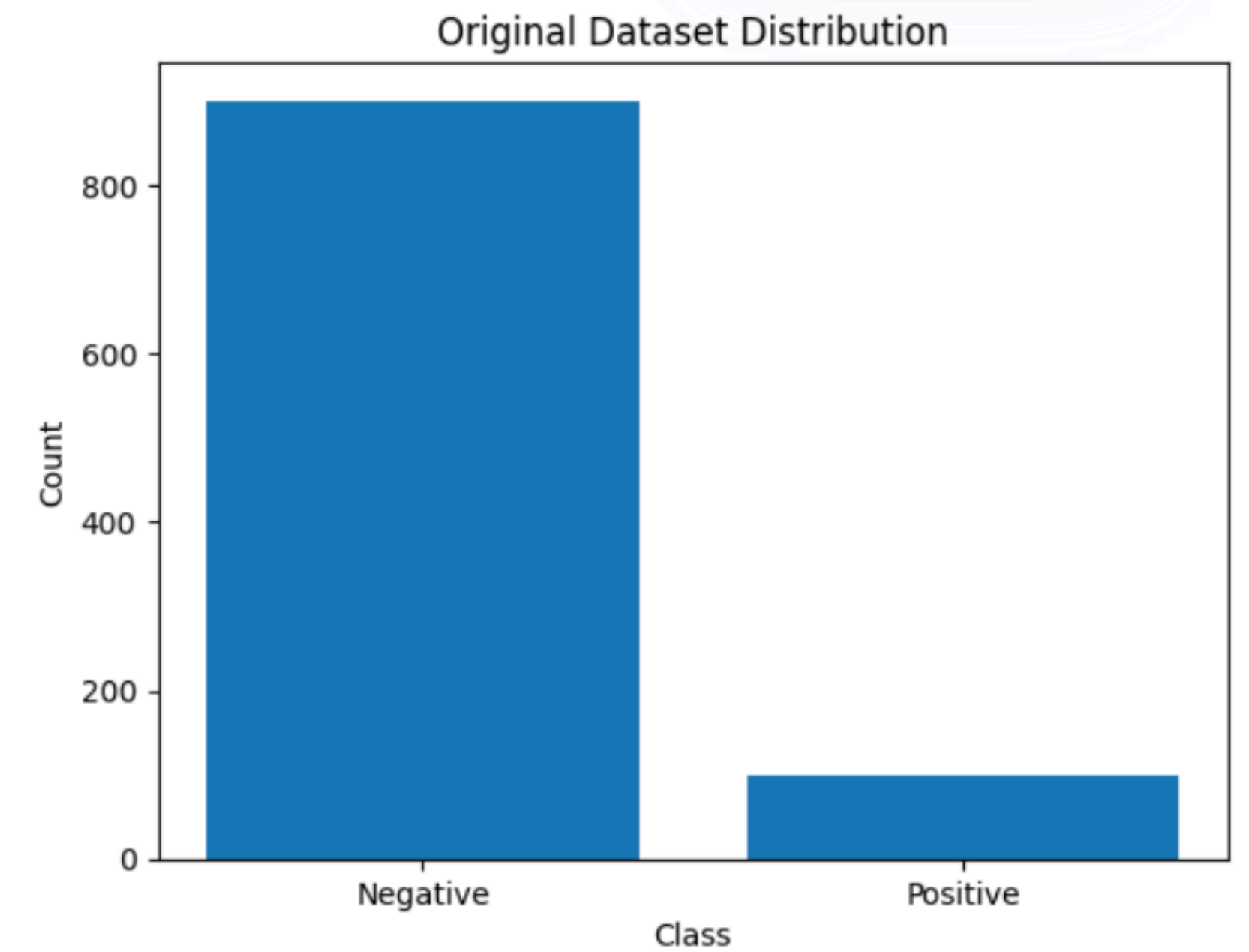
subsampling

```
from torch.utils.data import Subset
import random

indices = random.sample(range(len(dataset)), 5000)

subset = Subset(dataset, indices)

loader = DataLoader(subset, batch_size=32, shuffle=True)
```



sampling con reemplazo

```
from torch.utils.data import RandomSampler

sampler = RandomSampler(dataset, replacement=True,
num_samples=10000)

loader = DataLoader(dataset, batch_size=32, sampler=sampler)
```

weighted sampling

balanceo de clases

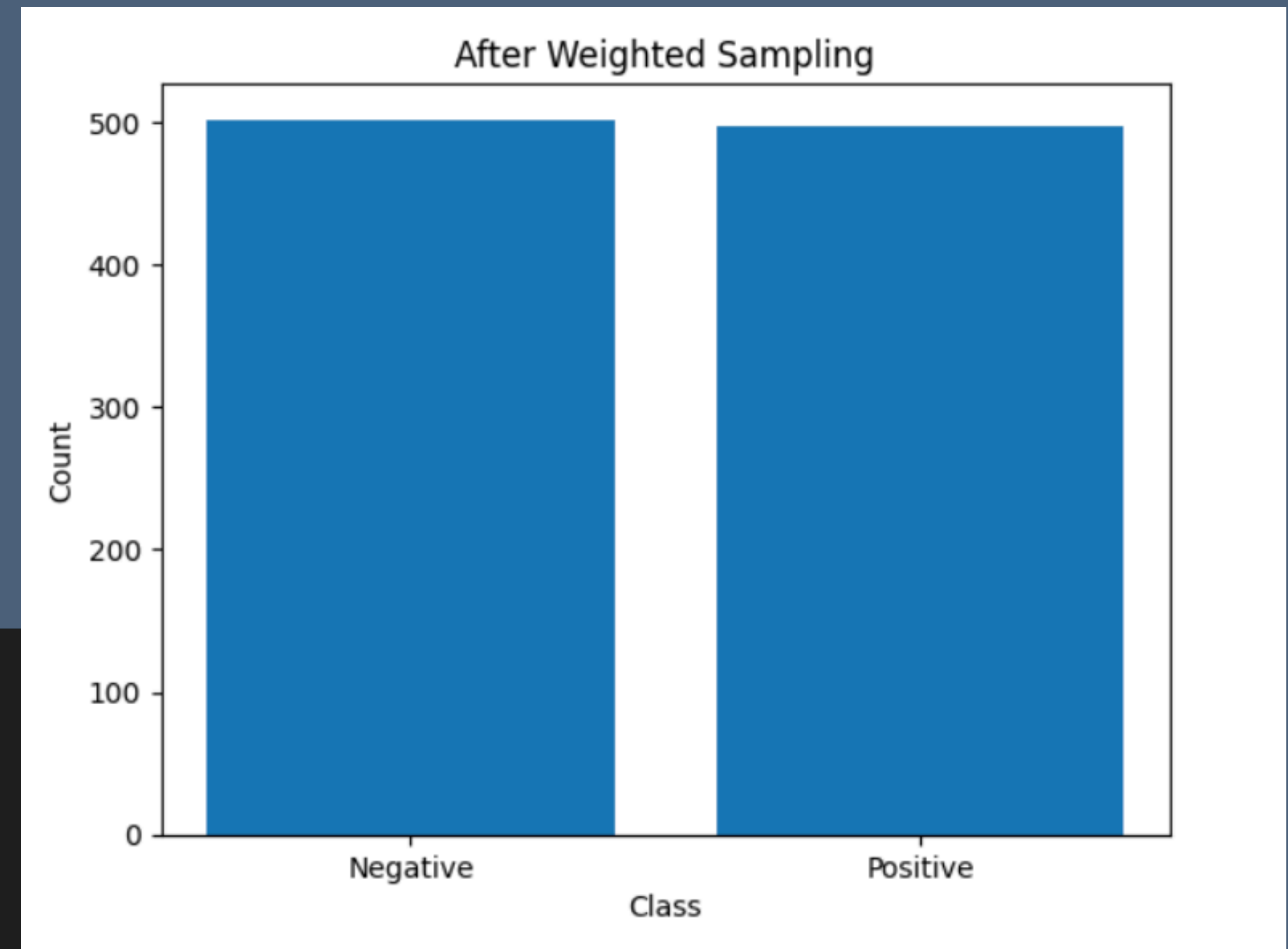
```
from torch.utils.data import WeightedRandomSampler
import torch
```

```
# ejemplo: pesos por clase
```

```
weights = torch.tensor([0.1, 0.9, 0.1, 0.9, ...])
```

```
sampler = WeightedRandomSampler(weights,
num_samples=len(weights))
```

```
loader = DataLoader(dataset, batch_size=32, sampler=sampler)
```



data-centric
VS
model-centric

datos vs modelo

- podemos dedicar más patrones y esfuerzo a mejorar el modelos (tuneando hiperparámetros, por ejemplo)
- o buscar mejores datos o hacer un tratamiento de mayor calidad de los mismos
- normalmente, lo segundo da mejores resultados, porque aunque los modelos sean limitados, aprenden mejor

preprocessing

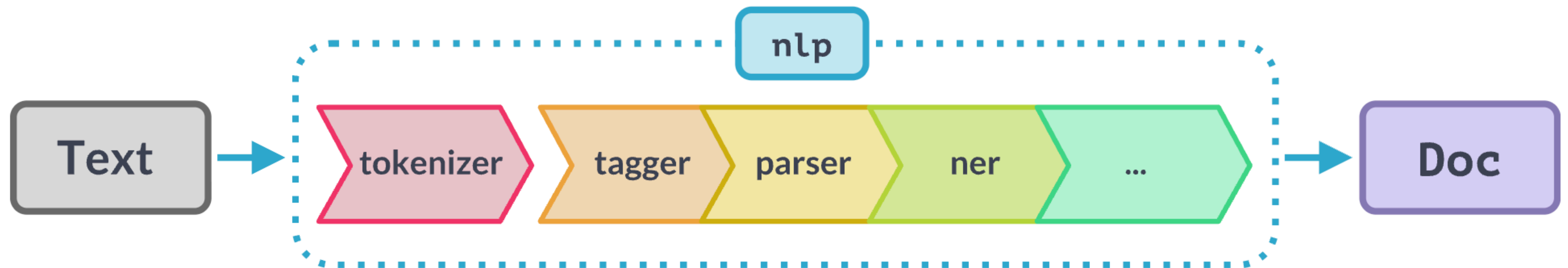
spaCy

- La opción más madura para realizar preprocessing
 - Soporte para casi todos los lenguajes
 - Muchísimo soporte y modelos preentrenados
 - Facilmente integrable con PyTorch
- Alternativa a NLTK
- Más adelante nos ayudará a realizar POS Tagging y NER

spaCy

pipelines

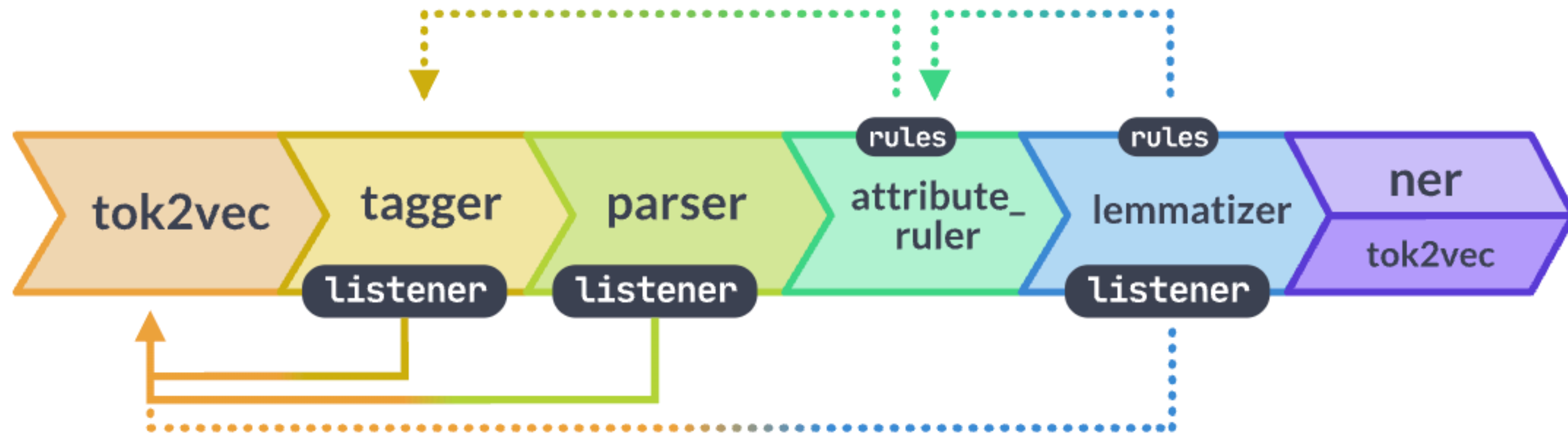
- Cuando se aplica la función **nlp** a un texto, spaCy primero lo tokeniza para generar un objeto Doc. Este objeto Doc se procesa en varias etapas, lo que se conoce como flujo de procesamiento. El flujo de procesamiento utilizado por los modelos entrenados suele incluir un etiquetador, un lematizador, un analizador sintáctico y un reconocedor de entidades.



spaCy

pipelines

- <https://spacy.io/models>



tokenización

- Extraer de un string cada token:
 - "Love is all you need" -> ["Love", "is", "all", "you", "need"]
 - Es un proceso bien conocido en compiladores
- Esta es la word tokenization, pero además contamos con:
 - char tokenization: "Love" -> ["L", "o", "v", "e"]
 - sentence tokenization

stopwords

- Términos o símbolos sin apenas contenido semántico (preposiciones, artículos... pero también contextuales)
- Susceptibles de ser filtradas para garantizar un procesado óptimo de nuestro texto cuando nos encontremos:
 - clasificación de texto
 - extracción de información
 - análisis de sentimientos
- Imprescindibles de mantener cuando:
 - traducimos texto
 - resúmenes
 - tareas que requieren mantener la estructura

filtrado de stopwords

```
nlp = spacy.load("en_core_web_sm")  
  
text = "This is a very good phone"  
  
doc = nlp(text)  
  
tokens = [  
    token.text  
    for token in doc  
    if not token.is_stop  
]  
  
# ['good', 'phone']
```

normalización

- case folding
- **stemming/lemmatization**: reducir los términos a sus formas base:
 - stemming: studies -> **studi** (corta el sufijo)
 - lemmatization: studies -> **study** (encuentra la raíz). Pero también: better -> **good**
- **Stemming es más rápido**, pero lemmatization es mejor para situaciones en las que hay que preservar semántica:
 - stemming: tratamiento de datasets grandes, search engines, donde nos importa procesar grandes cantidades de información
 - lemmatization: chatbots, sentimental analysis, resúmenes

normalización

```
import spacy

nlp = spacy.load("en_core_web_sm")

text = "The easiest way to learn NLP
is by doing projects and practicing
regularly."

doc = nlp(text)

for token in doc:
    print(token.text, token.lemma_)

# The the
# easiest easy
# ...
# is be
# ...
# doing do
# projects project
# ...
# practicing practice
# regularly regularly
```


spaCy

performance

- De cara a performance lo mejor es procesar los textos como un **stream** usando **pipe**
- Sólo debemos aplicar los componentes de la pipeline que necesitemo

```
texts = [  
    "Net income was $9.4 million compared to the prior year of $2.7 million.",  
    "Revenue exceeded twelve billion dollars, with a loss of $1b.",  
]  
  
nlp = spacy.load("en_core_web_sm")  
for doc in nlp.pipe(texts, disable=["tok2vec", "tagger", "parser", "lemmatizer"]):  
    print([(ent.text, ent.label_) for ent in doc.ents])
```

regex

Expresiones regulares

- Las expresiones regulares nos ayudan a las siguientes tareas preprocesando texto:
 - Pattern matching: direcciones de email, números de teléfono...
 - Eliminar tags, stopwords...
 - Sustituir términos
 - Convertir todo a lowercase

preprocessing | vectorización | clasificación

bag of words

- BoW representa un texto como un vector de conteos de palabras, ignorando el orden
- Por ejemplo:
 - Si tenemos un vocabulario: ["gato", "perro", "casa"]
 - Y el texto es: ["gato gato casa"]
 - El vector resultado es: [2, 0, 1]

bag of words

- Representación vectorial fija: determinista
- Ignora orden y contexto
- Alta dimensionalidad
- Muy disperso
- Apto para clasificación básica
- Pero no captura semántica, no filtra palabras por importancia ni retiene información de orden

frecuencia de término

TF

- TF mide qué proporción del documento corresponde a cada término, en vez de usar conteos absolutos

$$TF(t) = \frac{\text{número de veces que aparece } t}{\text{total de términos del documento}}$$

- Con el ejemplo anterior ("gato gato casa"), su TF sería:
 - gato: 2/3 casa: 1/3

frecuencia de término

TF

- Qué mejora respecto a BoW:
 - Normaliza por longitud del documento
 - Evita que textos largos dominen por tener más palabras
- Pero sigue sin diferenciar palabras comunes de importantes

TF-IDF

- Algunas palabras aparecen en casi todos los documentos (determinantes, por ejemplo)
- Estas palabras tienen poco valor informativo y tienen sobrerepresentación en algoritmos como TF
- La idea clave en TF-IDF es penalizar que aparezcan de forma repetida:

$$IDF(t) = \log \left(\frac{N}{df(t)} \right)$$

TF-IDF

- Consigue destacar términos específicos
- Reduce el peso de palabras comunes
- Mejora mucho en clasificación
- Pero sigue siendo un vector fijo disperso y sin orden

TF-IDF

```
from sklearn.feature_extraction.text import TfidfVectorizer
import torch

from vectorizer import TextVectorizer

class TfidfTextVectorizer(TextVectorizer):
    def __init__(self):
        self.vectorizer = TfidfVectorizer()

    def fit(self, texts):
        self.vectorizer.fit(texts)

    def transform(self, texts):
        X = self.vectorizer.transform(texts)
        return torch.tensor(X.toarray(), dtype=torch.float32)

    @property
    def output_dim(self):
        return len(self.vectorizer.get_feature_names_out())
```

n-gramas

- Todos los algoritmos anteriores tienen algo en común: ignoran el orden
- Por ejemplo, los textos:
 - “no es bueno”
 - “es bueno”
 - tienen casi las mismas palabras, pero significados distintos
- Un n-grama es una secuencia de n palabras consecutivas

n-gramas

- “me gusta aprender”
 - unigrama: [“me”, “gusta”, “aprender”]
 - bigrama: [“me gusta”, “gusta aprender”]
 - trigrama: [“me gusta aprender”]

n-gramas

- Introducen información de orden local
- Capturan expresiones fijas
- Detectan negaciones ("no bueno")
- Pero:
 - aumenta el tamaño del vocabulario
 - aumenta la dimensionalidad
 - puede mejorar precisión si el dataset no es pequeño

quiz time

```
corpus = [  
    "el gato come pescado",  
    "el perro come carne",  
    "el gato y el perro juegan"  
]
```

```
vectorizer = TfidfVectorizer()  
X = vectorizer.fit_transform(corpus)
```

```
pd.DataFrame(X.toarray(), columns=vectorizer.get_feature_names_out())
```

#		carne	come	gato	juegan	perro	pescado	el	y
# doc	0	0.000	0.478	0.478	0.000	0.000	0.628	0.281	0.000
# doc	1	0.628	0.478	0.000	0.000	0.478	0.000	0.281	0.000
# doc	2	0.000	0.000	0.371	0.489	0.371	0.000	0.436	0.489

verdadero o falso

- “La lemmatización y el stemming producen siempre el mismo resultado”
- “Eliminar stopwords siempre mejora el rendimiento del modelo”
- “Bag of Words tiene en cuenta el orden de las palabras”
- “Una palabra que aparece en todos los documentos tiene un IDF alto”

verdadero o falso

- “La lemmatización y el stemming producen siempre el mismo resultado” Falso
- “Eliminar stopwords siempre mejora el rendimiento del modelo” Falso
- “Bag of Words tiene en cuenta el orden de las palabras” Falso
- “Una palabra que aparece en todos los documentos tiene un IDF alto” Falso

```
# ¿Qué está mal?  
import spacy  
  
for texto in corpus:  
    nlp = spacy.load("es_core_news_sm")  
    doc = nlp(texto)  
  
    # ...
```

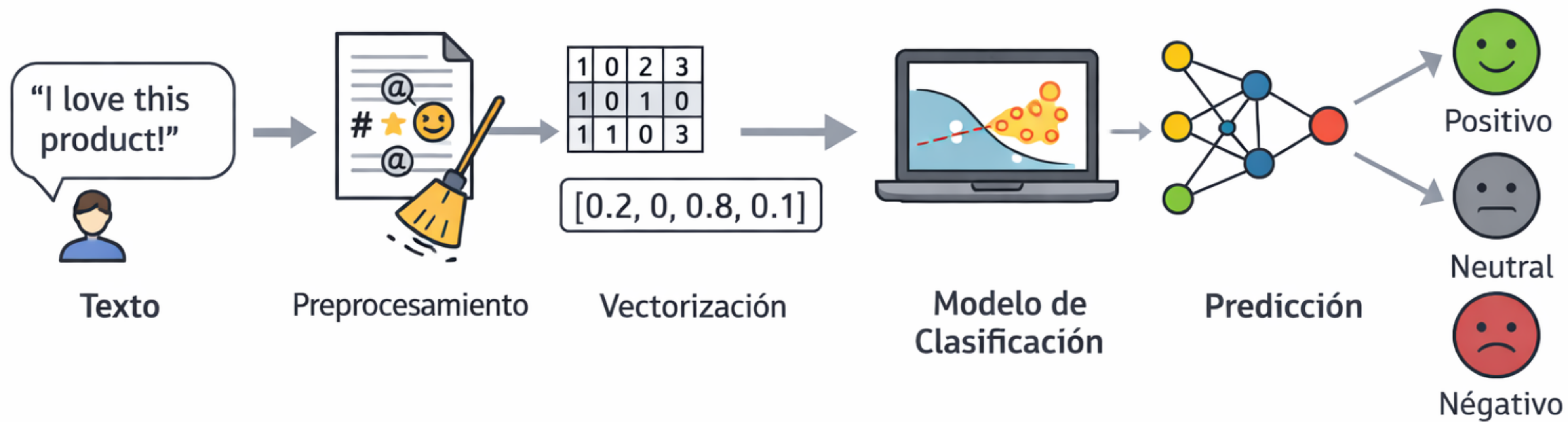


```
# ¿Qué tokens conservarías para un modelo de clasificación de texto? ¿Por qué?  
# ¿Cambiaría tu respuesta si la tarea fuera análisis de sentimiento?
```

```
doc = nlp("El producto no es tan malo como esperaba, pero tampoco es bueno")  
for token in doc:  
    print(token.text, token.lemma_, token.pos_, token.is_stop)
```

```
# El          el          DET      True  
# producto    producto    NOUN     False  
# no          no          ADV       True    ← ojo aquí  
# es          ser          AUX       True  
# tan         tan          ADV       True    ← ojo aquí  
# malo        malo        ADJ       False  
# como       como        SCONJ     True  
# esperaba    esperar    VERB     False  
# ,           ,           PUNCT     False  
# pero       pero          CCONJ     True  
# tampoco    tampoco    ADV       True    ← ojo aquí  
# es         ser          AUX       True  
# bueno      bueno        ADJ       False
```

preprocessing | vectorización | clasificación



¿Qué es un modelo de clasificación en NLP?

- un modelo de clasificación asigna una etiqueta discreta a un texto de entrada. Es la tarea supervisada más frecuente en NLP aplicado
- el flujo siempre es el mismo
 - texto
 - preprocesado
 - vectorización
 - modelo
 - etiqueta
- lo que cambia de un modelo a otro es únicamente el último paso –cómo se toma la decisión de clasificación.

clasificación con ML

- consiste en asignar una o varias etiquetas a un documento
- casos de uso:
 - detectar spam
 - clasificar sentimientos
 - categorizar noticias
 - clasificar tickets de soporte
- un clasificador aprende una función que recibe texto y devuelve una etiqueta

ejemplos de modelos clásicos

antes de transformers, estos modelos dominaban NLP

- **Naive-Bayes**: probabilidad condicional de palabras. Perfecto para baseline rápido con corpus pequeño
- **Regresión logística**: combinación lineal de features. Interpretable, buen balance velocidad/precisión
- SVM: hiperplano de máximo margen. Textos cortos, pocas clases
- Árbol de decisión: reglas jerárquicas sobre features. Útil cuando se necesita explicabilidad total
- Random forest: ensemble de árboles. Más robusto que un solo árbol
- Gradient boosting: árbol en secuencia corrigiendo errores. Cuando el rendimiento es prioritario

entrenamiento

- ¿qué es entrenar un modelo?
- 3 ideas:
 - predecir (forward)
 - comparar con la realidad (backwards)
 - corregir

entrenamiento

- en el entrenamiento de un modelo hay que tener en cuenta 3 cosas principales:
 - features (X): el input, los datos vectorizados que hemos estudiado antes
 - labels (y): el output (vectorizado también)
 - loss: la medida de cuánto se equivoca el modelo en cada predicción (para poder corregir)

naive-bayes

- es un modelo de clasificación que clasifica según que palabras son más probables en cada clase
- ¿Qué etiqueta hace más probable este texto? “amazing product” -> positive
- $P(\text{clase} \mid \text{texto}) \propto P(\text{texto} \mid \text{clase}) * P(\text{clase})$

$$P(A|B) = \frac{P(B|A) * P(A)}{P(B)}$$

Paciente	Fiebre	Tos	Fatiga	Diagnóstico
P1	Sí	Sí	Sí	Gripe
P2	Sí	No	Sí	Gripe
P3	No	Sí	Sí	Gripe
P4	Sí	Sí	No	Gripe
P5	No	Sí	No	Resfriado
P6	No	Sí	No	Resfriado
P7	No	No	Sí	Resfriado
P8	Sí	No	No	Resfriado

P(clase)

Gripe = $4/8 = 0.5$
Resfriado = $4/8 = 0.5$

P(Fiebre | clase)

Gripe: $3/4 = 0.75$
Resfriado: $1/4 = 0.25$

P(Tos | clase)

Gripe: $3/4 = 0.75$
Resfriado: $2/4 = 0.50$

Nuevo paciente: Fiebre=Sí, Tos=Sí, Fatiga=No → ¿Gripe o Resfriado?

Gripe

$0.5 \times 0.75 \times 0.75 = 0.28$

Resfriado

$0.5 \times 0.25 \times 0.50 = 0.06$

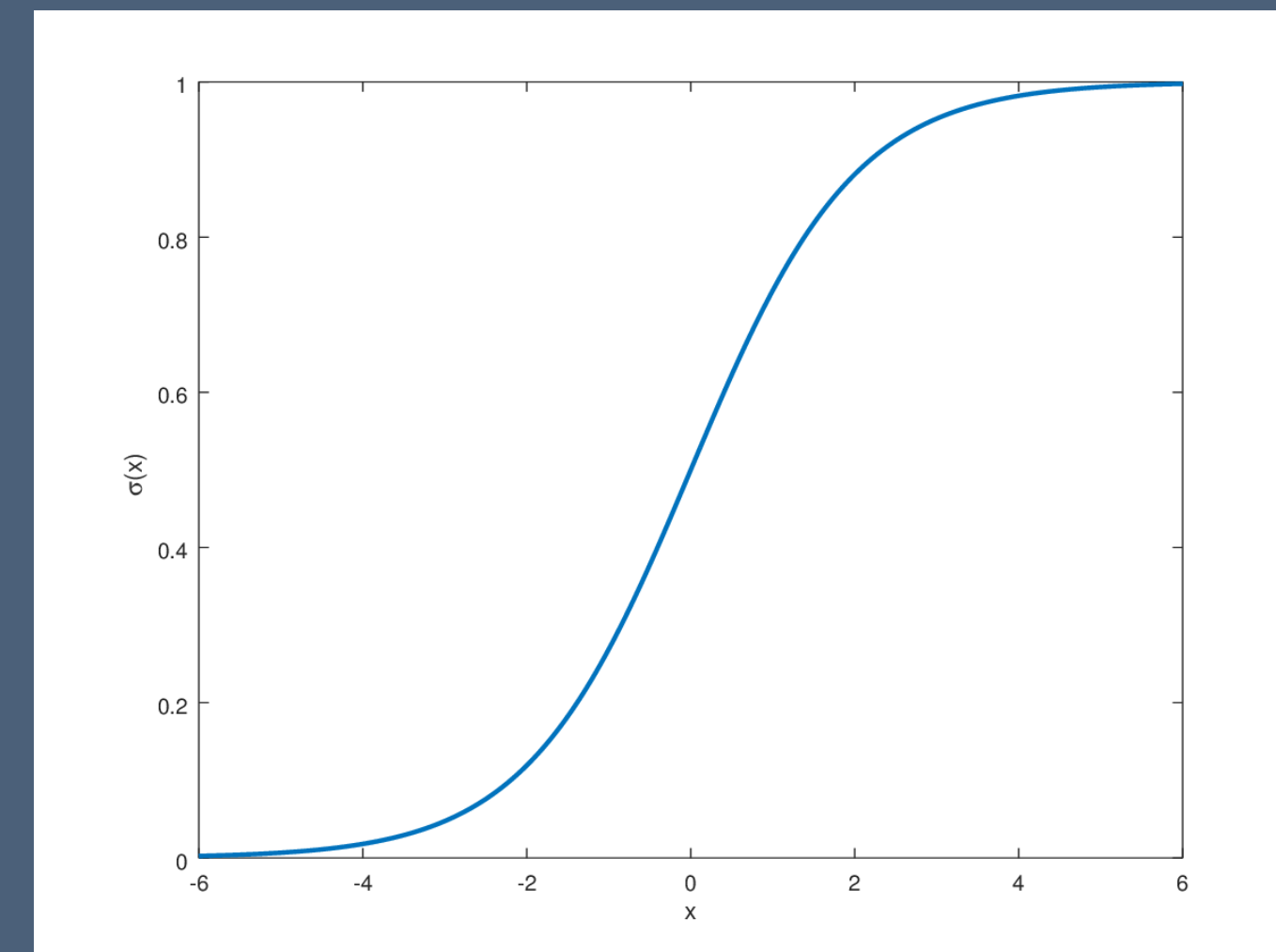
$0.28 > 0.06 \rightarrow$ diagnóstico: Gripe

naive-bayes

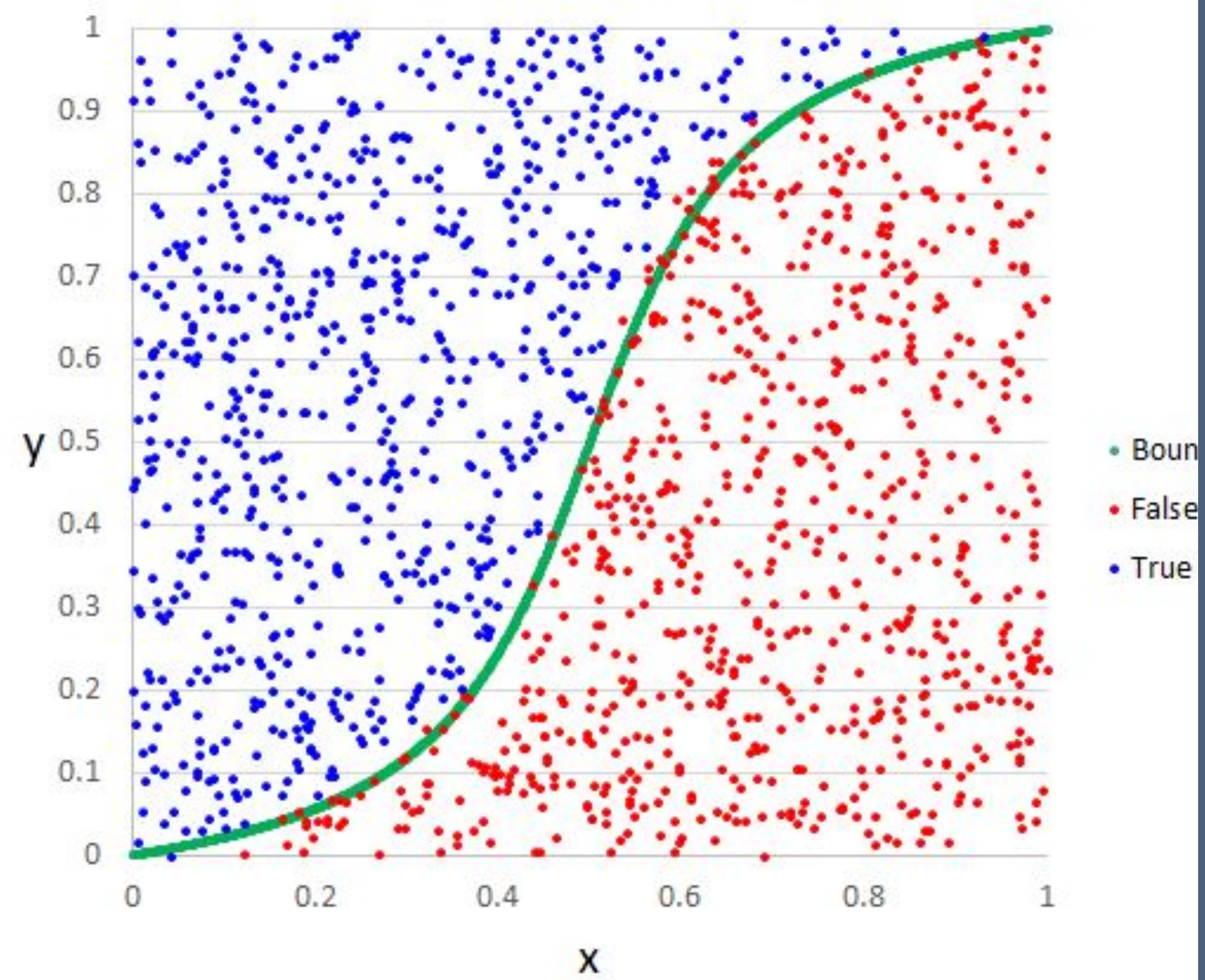
- naive-bayes asume que las palabras son independientes
- “very good product” - $P \approx P(\text{very}|+) * P(\text{good}|+) * P(\text{product}|+)$
- es básicamente: contar palabras y su probabilidad de aparecer bajo una etiqueta u otra
 - es naive porque simplifica
 - es bayesiano porque actualiza las probabilidades según la experiencia (la secuencia de inputs)

regresión logística

- en vez de usar probabilidades usamos pesos para clasificar palabras
- en vez de buscar qué palabras son típicas dentro de una clase se mide qué palabras miden la decisión
 - peso **positivo** -> clase positiva
 - peso **negativo** -> clase negativa
 - para interpretar la salida se utiliza la función sigmoide
- ambos modelos no entienden el texto, sólo ponderan por feature (palabra)
- la ventaja sobre naive-bayes es que este modelo es interpretable



Logistic Regression Example



diferencias

- Naive-Bayes piensa de la siguiente manera:
 - *si aparece 'gratis', 'oferta' y 'clic' -> probablemente es **spam***
 - va acumulando probabilidades palabra por palabra
 - muy bueno cuando:
 - hay poco dataset
 - necesitamos rapidez
 - necesitas un fundamento sólido (la formula de Bayes)

diferencias

- La regresión logística piensa:
 - *voy a aprender qué palabras empujan la decisión hacia spam o no spam. Aprende **pesos***
 - "gratis" -> +2.1 (se acerca a spam)
 - "trabajo" -> -1.4 (se aleja)
- buena elección cuando:
 - tienes más datos
 - quieres más precisión
 - las relaciones son más complejas

evaluación

- La validación de un modelo ya entrenado se realiza mediante un set dedicado exclusivamente a contrastar los resultados obtenidos con los esperados
- Pero además tenemos una serie de herramientas para medir cómo de efectivo es un modelo clasificando:
 - matriz de confusión
 - accuracy/recall
 - f1-score
 - support

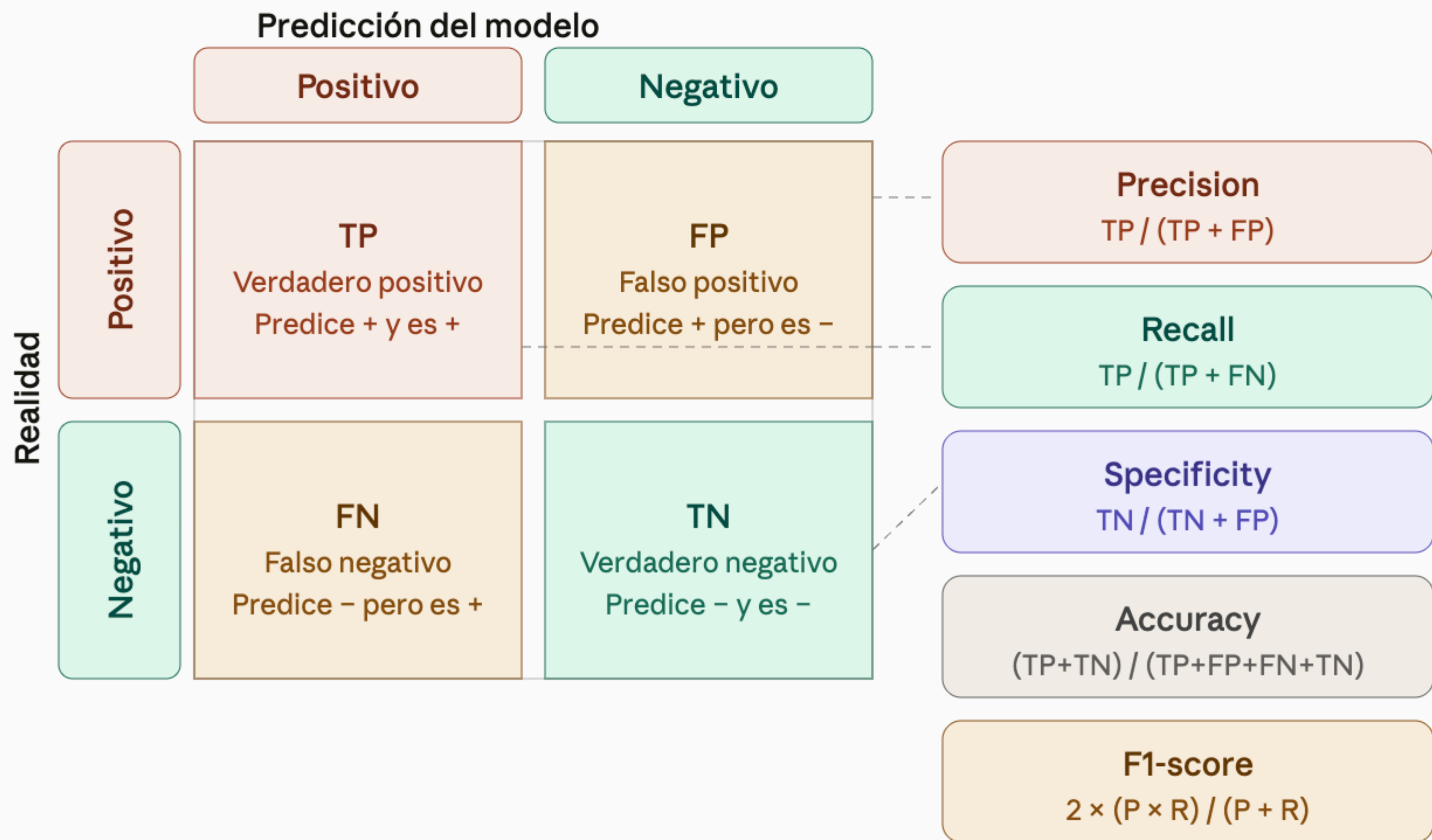
matriz de confusion

- Esta matriz nos cuenta cómo de fácil es engañar al modelo midiendo los aciertos pero tambien:
 - los falsos positivos (FP)
 - los falsos negativos (FN)
- Cuanto más valores se acumulen en la diagonal principal, mejor
- Las filas cuentan el balance en cada clase
- Sumando los valores de positivos y negativos podemos ver el sesgo del modelo

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

métricas

- **accuracy**: es la proporción de todas las clasificaciones que fueron correctas, ya sean positivas o negativas. Se usa como un indicador aproximado del progreso o la convergencia del entrenamiento del modelo para conjuntos de datos equilibrados.
- **precision**: es la proporción de todas las clasificaciones positivas del modelo que son realmente positivas. Se usa cuando es muy importante que las predicciones positivas sean precisas.
- **recall**: proporción de todos los positivos reales que se clasificaron correctamente como positivos. Se usa cuando los falsos negativos son más costosos que los falsos positivos.



f1-score

- Suele ser conveniente combinar *precision* y *recall* en una única métrica que refleje la media armónica. Como resultado, el clasificador sólo tendrá un valor alto en esta métrica si ambos son valores altos.
- La métrica f1 favorece clasificadores que tienen una *precision* y *recall* bajos. Pero esto no es lo que necesitamos siempre. Hay casos donde nos importan menos los falsos negativos (*recall* bajo, filtros infantiles, por ejemplo) y otras donde vamos a preferir tener un *recall* alto, como en análisis médicos (donde queremos que no se escape ningún caso, aunque haya que revisarlos manualmente).

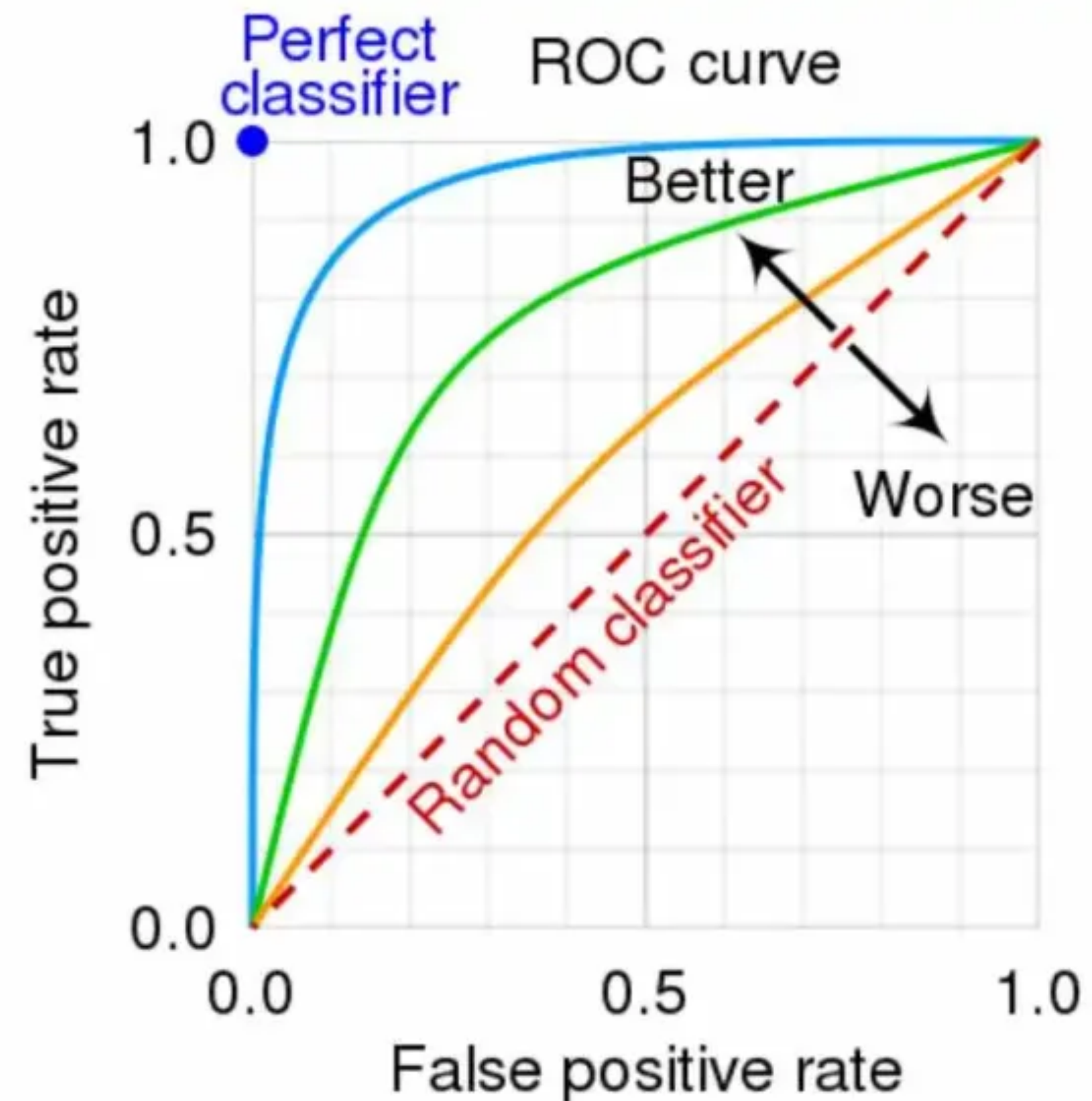
precision/recall tradeoff

- No podemos tener ambos mundos: un recall alto supone baja precisión y viceversa
- No existe un modelo perfecto, tenemos que elegir entre:
 - Ser precisos (no equivocarnos): evitar falsas alarmas -> molestar poco al usuario
 - Ser seguros (acertar casi siempre): clasificar como positivo de más -> acertar siempre
- Lo importante es encontrar el equilibrio perfecto para nuestra aplicación

Curva ROC

Receiver Operating Characteristic

- Representa la relación entre:
 - True Positive Rate (TPR): un buen modelo lo tiene alto
 - False Positive Rate (FPR): un buen modelo lo tiene bajo



UAC

Area Under the Curve

- Indica qué bien el modelo separa las clases
- La curva ROC permite:
 - Analizar el comportamiento del modelo para todos los umbrales
 - comparar clasificadores
 - medir la capacidad de separación entre clases
- La AUC resume esa información en un único número
- Márgenes: 0,5 (modelo aleatorio) - 0,7+ aceptable - 0,9+ excelente

precision/recall vs ROC

- Aunque ambos son dos formas comunes de evaluar clasificadores binarios, ambas analizan el comportamiento del modelo para distintos umbrales de decisión
- Curva ROC es útil cuando las clases están relativamente balanceadas (considera todos los negativos correctamente clasificados)
- PR es mejor en datasets desbalanceados, es más común en NLP y muy utilizada en casos como:
 - detección de fraude
 - spam/toxicidad
 - clasificación de eventos raros

Ejemplo

- En un dataset con un 95% de mails normales y un 5% de spam un modelo puede obtener:
 - ROC AUC = 0.90
 - y un **recall muy bajo**
- La curva precision-recall revela el problema, pero ROC no

Métrica	Qué mide realmente	Cuándo importa más	Ejemplo típico NLP	Qué error quiere minimizar	Caso donde puede engañar
Accuracy	% total de aciertos	Cuando las clases están balanceadas	Clasificación sencilla de noticias por categoría	Errores generales	Datasets desbalanceados
Precision	De lo que el modelo predijo como positivo, cuánto era correcto	Cuando los falsos positivos son costosos	Detector de spam que no debe ocultar emails legítimos	Falsos positivos (FP)	Puede ignorar muchos positivos reales
Recall	De todos los positivos reales, cuántos encontró	Cuando perder positivos es peligroso	Detección de fraude, cáncer o contenido tóxico	Falsos negativos (FN)	Puede generar demasiadas falsas alarmas
F1-score	Equilibrio entre precision y recall	Cuando ambas son importantes	Moderación automática de contenido	Balance FP/FN	Puede ocultar diferencias concretas entre precision y recall
Specificity	Qué bien detecta los negativos reales	Cuando evitar falsas alarmas es importante	Filtrado de contenido seguro/no tóxico	Falsos positivos	Poco útil si el foco son los positivos

quiz time

matriz de confusión

	Pred Spam	Pred No Spam
Spam real	40	10
No spam real	5	45

- ¿Cuántos verdaderos positivos?
- ¿Cuántos falsos positivos?
- ¿Cuántos falsos negativos?
- ¿Cuántos verdaderos negativos?

matriz de confusión

	Pred Spam	Pred No Spam
Spam real	40	10
No spam real	5	45

- ¿Cuántos verdaderos positivos? 40
- ¿Cuántos falsos positivos? 5
- ¿Cuántos falsos negativos? 10
- ¿Cuántos verdaderos negativos? 45

- En un dataset médico hay 95 pacientes sanos y 5 enfermos. El modelo predice que todos están sanos ¿cuál es el recall?
 - 5%
 - 50%
 - 95%
 - 100%

¿es este modelo bueno?

- un detector de spam marca 20 emails como spam, de los cuales 15 eran spam realmente y 5 no ¿cuál es la **precisión**?
- 15/20
- 15/total de spam reales
- 20/total de emails
- no se puede calcular

- un detector de spam marca 20 emails como spam, de los cuales 15 eran spam realmente y 5 no ¿cuál es la **precisión**?
- 15/20
- 15/total de spam reales
- 20/total de emails
- no se puede calcular

- Había 30 emails reales de spam. El modelo detectó 24 ¿cuál es el recall?
 - 24/30
 - 30/24

- Estás usando Naive-Bayes para detectar si enfermos tienen un virus o no ¿qué métrica priorizas?
 - precision
 - recall

- Estás usando Naive-Bayes para detectar si enfermos tienen un virus o no ¿qué métrica priorizas?
- precision
- **recall**: no quieres dejar escapar enfermos reales, prefiero sobre-alarmar

embeddings | análisis de sentimientos

embeddings | análisis de sentimientos

concepto

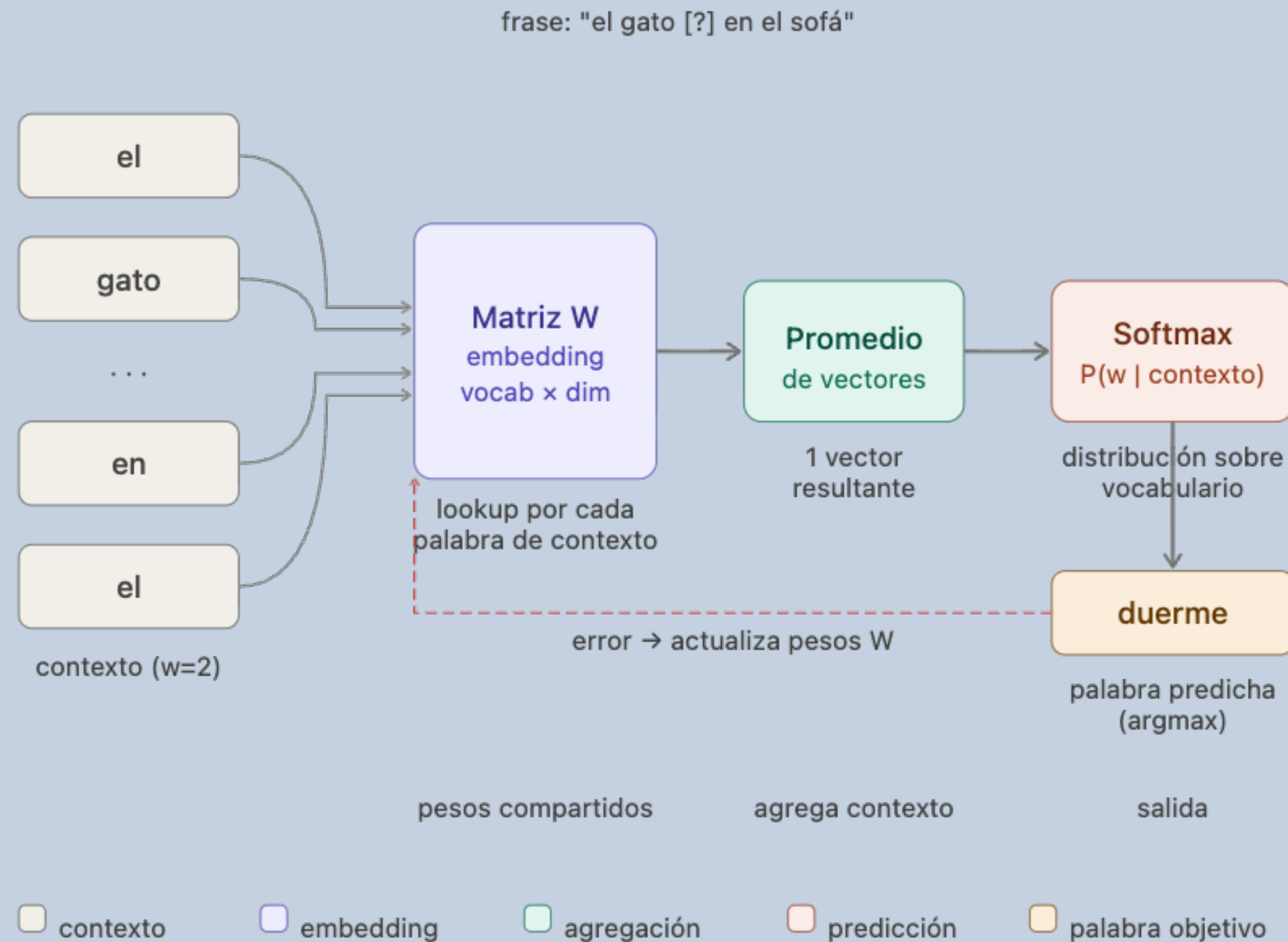
- los **word embeddings** son representaciones vectoriales de palabras que modelan el significado de esa palabra
- con esta forma vectorial, palabras que tengan un significado similar estarán más cerca en el espacio vectorial

Word2Vec

- en 2013 Google publicó un paper que explicaban cómo vectorizar un corpus de palabras mediante redes neuronales
- es una técnica para aprender asociaciones de palabras sin necesidad de que estén etiquetadas
- una vez entrenado puede detectar sinónimos o sugerir palabras adicionales para para completar una frase
- Word2Vec es una familia de algoritmos que producen word embeddings
- en cierta forma, conseguimos **capturar la personalidad de las palabras**

CBOW architecture

predice una palabra a partir de su contexto (qué palabras suelen estar juntas)

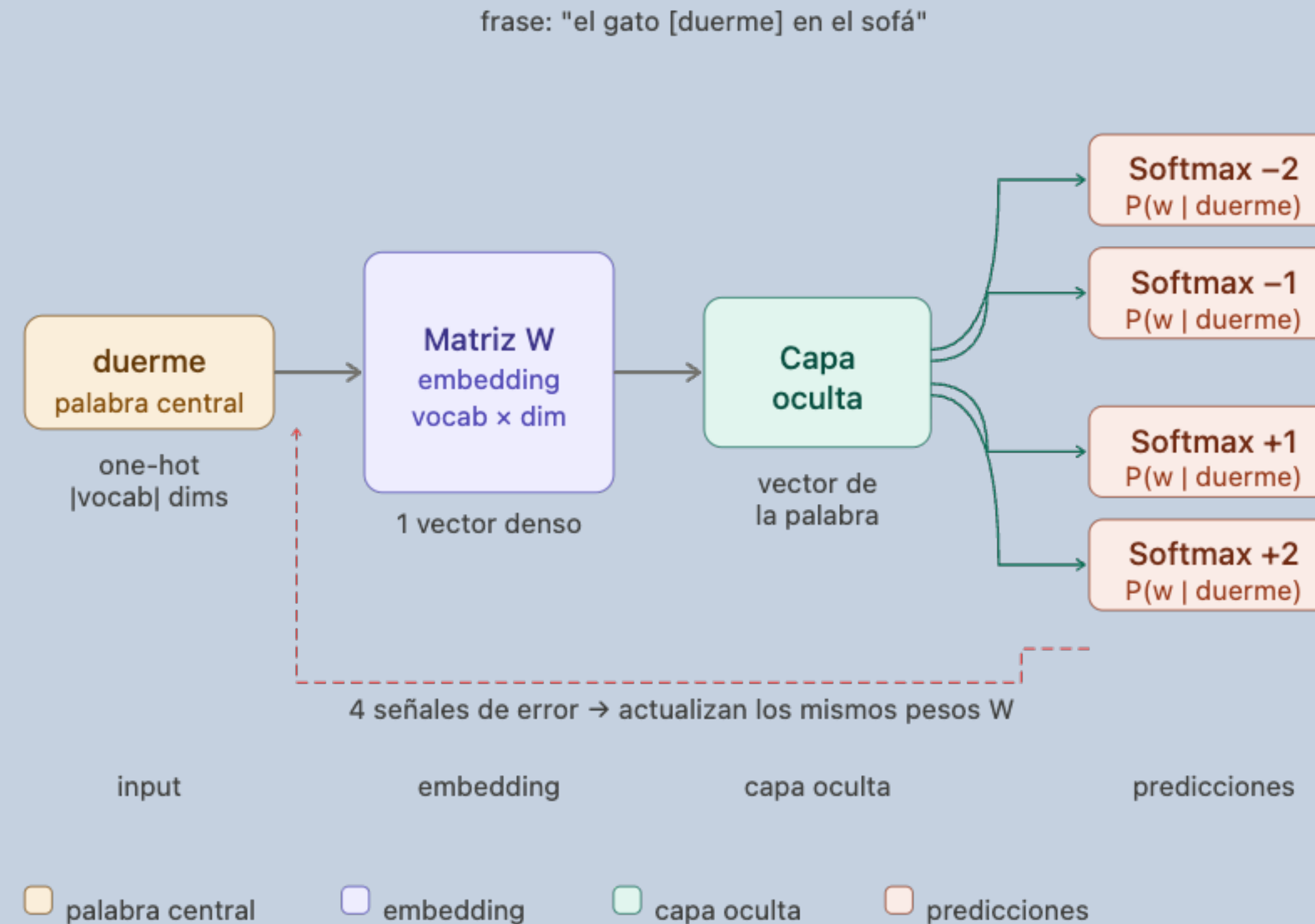


CBOW — muchos contextos → una palabra central

aprende promediando señales de múltiples palabras de contexto

skip-gram architecture

predice el contexto a partir de la palabra



Skip-gram — una palabra central → múltiples contextos

ventana $w=2 \rightarrow 4$ pares de entrenamiento por palabra
aprende mejor relaciones de palabras poco frecuentes

Word2Vec

propiedades

- captura **propiedades** como:
 - género: hombre -> mujer
 - capital-país: francia -> paris
- es capaz de generalizar incluso con palabras no vistas con frecuencia

Word2Vec

limitaciones

- no tiene contexto dinámico
- no entiende polisemia (una palabra significa sólo una cosa)
- ha sido superado por modelos como BERT o GPT
- en resumen, sólo aprende la semántica mediante las palabras que lo rodean
- a pesar de ello, fue un antes y un después en NLU

GloVe

- desarrollado por la universidad de Stanford
- frente a word2vec, aprovecha estadísticas globales de todo el corpus en lugar de solo contexto local, por lo que aprende de conteos globales de co-ocurrencia
- la forma de lograrlo es construyendo una **matriz de co-ocurrencias**
- no usa directamente los conteos, sino ratios de probabilidades
- la relación de palabras se captura mejor mediante factorización que prediciendo (como hacía el algoritmo anterior)

corpus de entrenamiento (ventana de contexto $w=2$)

"el gato duerme en el sofá" · "el gato come pescado" · "el perro duerme en el sofá"

contar co-ocurrencias

simétrica:
 $X(i,j) = X(j,i)$

	gato	duerme	come	perro	sofá
gato	0	1	1	0	0
duerme	1	0	0	1	2
come	1	0	0	0	0
perro	0	1	0	0	1
sofá	0	2	0	1	0

máx: 2
co-ocurren en
2 frases

factorización + función objetivo

Función objetivo
minimiza $(w_i \cdot w_j - \log X(i,j))^2$

Vectores w_i, w_j
un vector por palabra

Embeddings
 $w_i + \tilde{w}_i$ (suma final)

diferencia clave respecto a Word2Vec:

Word2Vec
aprende de ventanas locales

GloVe
aprende de estadísticas globales

 co-ocurrencia = 0  co-ocurrencia = 1  co-ocurrencia = 2 (máx)

GloVe — la co-ocurrencia global como señal de aprendizaje

GloVe

Ventajas

- usa toda la información del corpus
- el entrenamiento es más estable
- buen rendimiento semántico

GloVe

Limitaciones

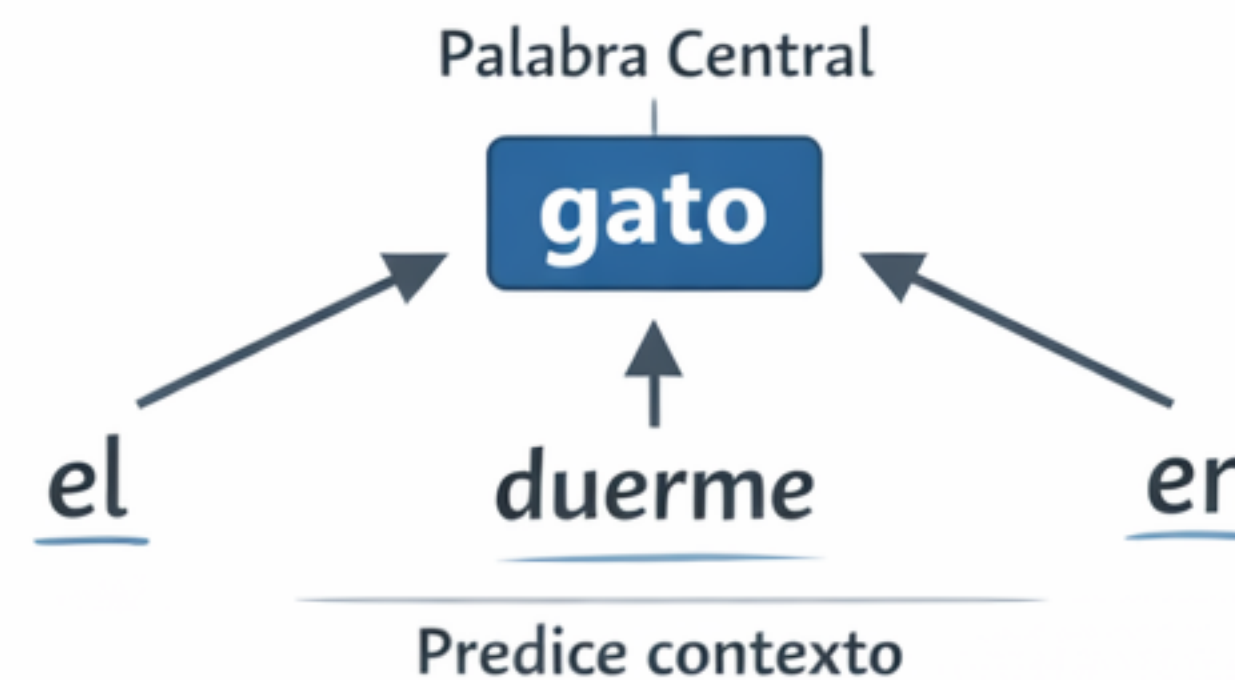
- coste de memoria: necesita construir la matriz global
- menos online que Word2Vec
- tampoco resuelve polisemia
- Word2Vec aprende leyendo frases una a una. GloVe, en cambio, tiene en cuenta todo el corpus y aprende a partir de las estadísticas globales

Word2Vec: Context Window



Aprende de palabras cercanas.

Word2Vec: Skip-gram Model



Intenta predecir palabras alrededor.

GloVe: Matriz de Co-ocurrencia

	hielo	vapor	sólido
hielo	—	2	15
vapor	2	—	1
sólido	15	1	—

Cuenta global de co-ocurrencias.

Word2Vec vs. GloVe

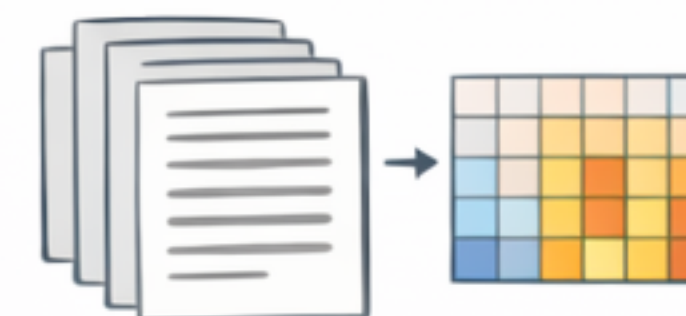
Word2Vec

Contexto Local: Ventana



GloVe

Estadísticas Globales: Matriz



embeddings | análisis de sentimientos

solución 1

embeddings preentrenados

- podemos obtener embeddings ya extraídos de un corpus
- el modelo nunca conoció nuestro dataset
- listo para usar
- es importante inspeccionar cuánto de nuestro vocabulario está cubierto por estos embeddings (puede haber divergencias grandes):
 - los tokens no encontrados se inicializan sus vectores de forma aleatoria
 - un 80-85% de cobertura es habitual para casos de uso como el de twitter. Por ejemplo textos técnicos o por el contrario muy coloquiales tienen menor cobertura

solución 2

entrenar nuestros propios embeddings

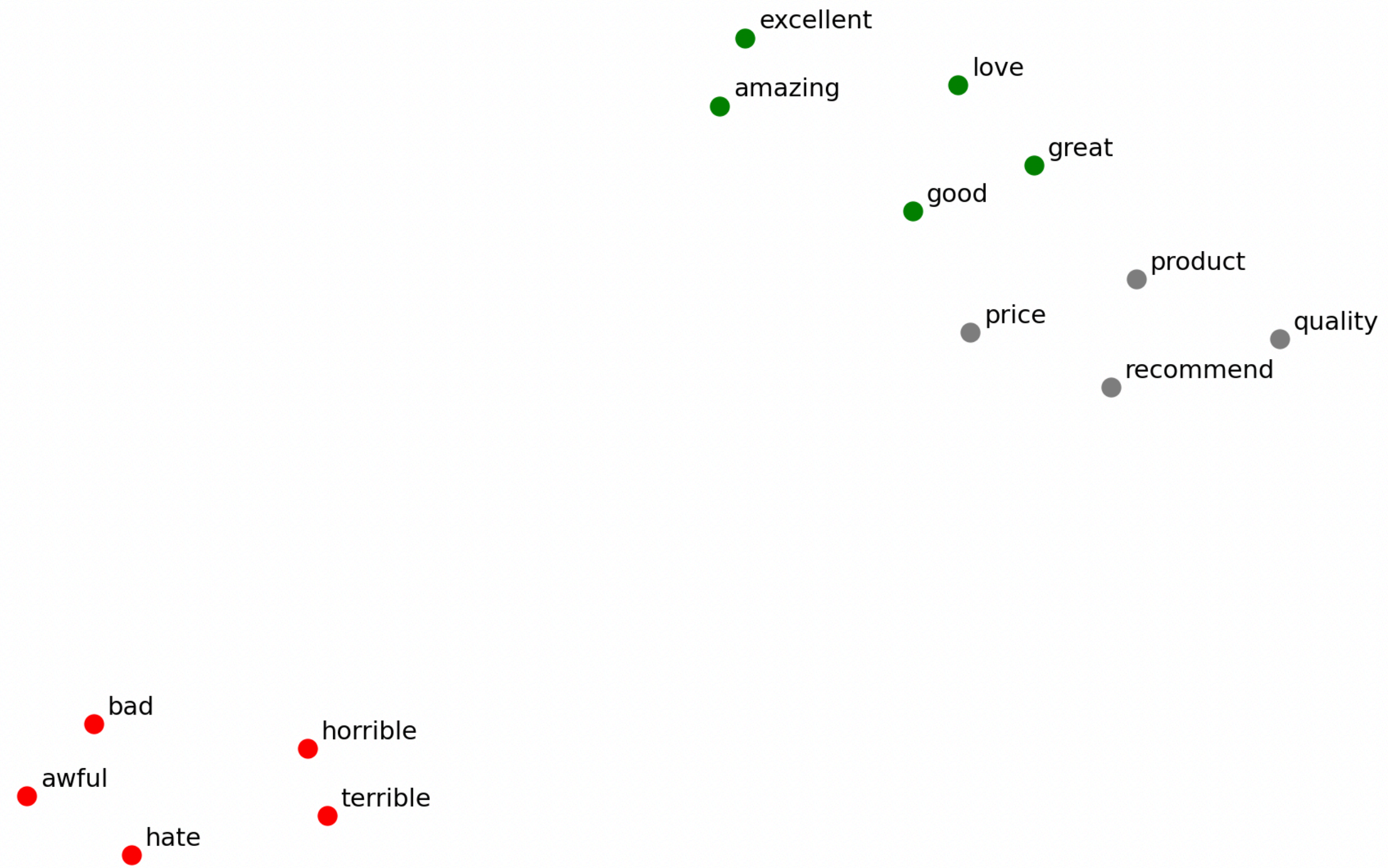
- la opción contraria: aprender desde 0
- el modelo descubre por si solo qué vectores son útiles para predecir sentimiento
- ventaja: los vectores se especializan exactamente para la tarea
- limitación: necesita muchos datos y de calidad

t-SNE

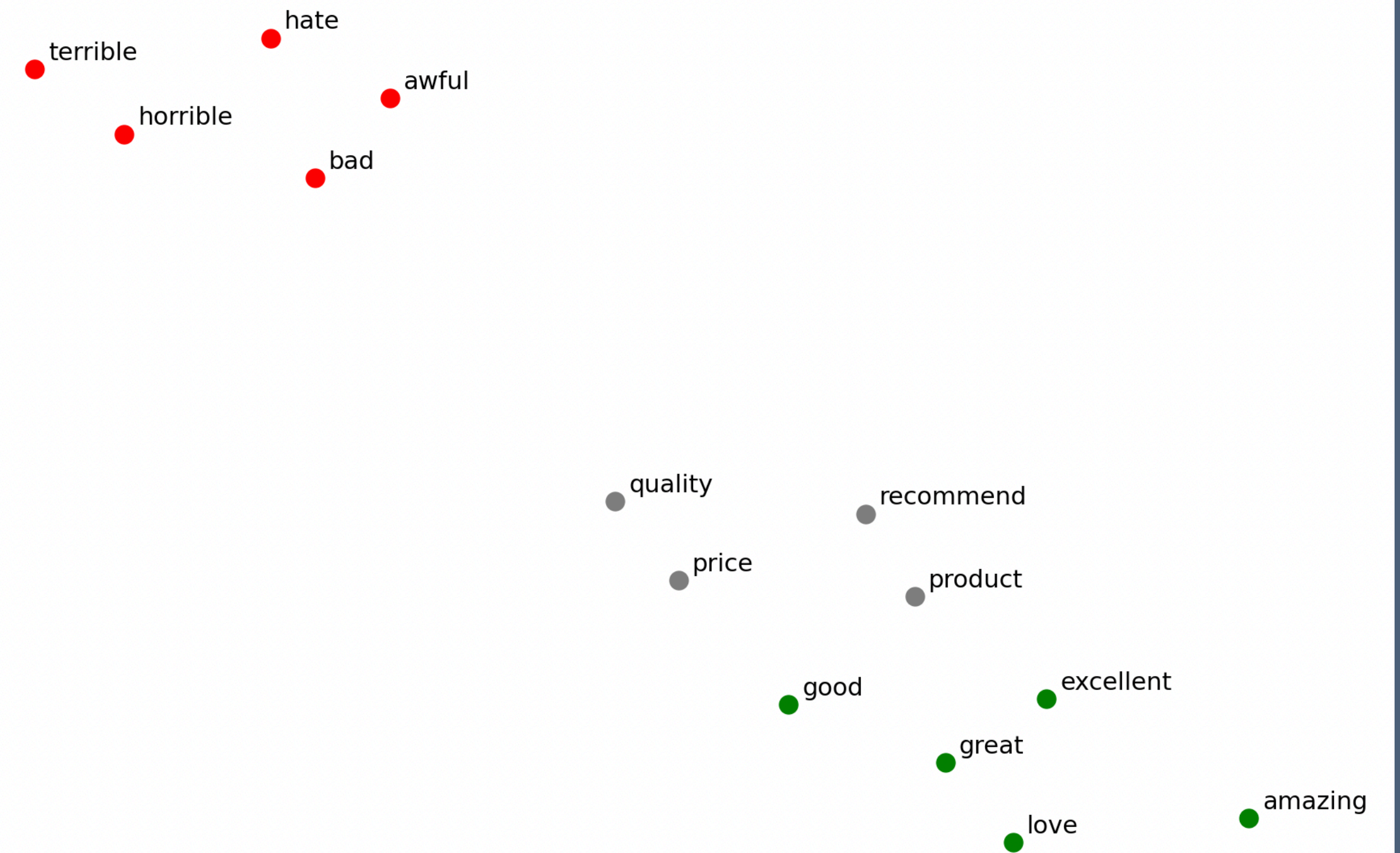
ver embeddings en 2D

- los vectores de palabras viven en un espacio de aproximadamente 100 dimensiones, por lo que no podemos visualizarlo directamente, pero podemos proyectarlos a 2D preservando las distancias relativas
- t-distributed Stochastic Neighbor Embedding es un algoritmo de reducción de dimensiones diseñado específicamente para visualización: palabras que eran vecinas en 100D son vecinas en 2D
- la distribución t es clave: evita que puntos moderadamente cercanos colapsen en el centro, separando mejor los grupos
- algunas limitaciones:
 - no es determinista, cada ejecución crea un mapa distinto
 - la escala global no es interpretable
 - el parámetro perplexity controla cuántos vecinos considera cada punto. Con pocos puntos se usa un valor bajo
- t-SNE es una **herramienta de exploración no de medición**

t-SNE — embeddings FastText aprendidos



t-SNE — embeddings FastText aprendidos



topics | NLU

topics | NLU

¿qué es un tópico?

de palabras a temas

- un **tópico** es una distribución de probabilidad sobre el vocabulario
 - asigna una probabilidad alta a un conjunto coherente de palabras (ej: book, read, story) y casi cero al resto
- no es una etiqueta manual: el modelo descubre estas agrupaciones de forma no supervisada, sin ver ninguna categoría predefinida
- intuitivamente, un tópico es el patrón de co-ocurrencia estadística más frecuente entre palabras: si battery, camera y device aparecen juntas, el modelo las agrupa
- cada documento es una mezcla de tópicos, no pertenece a uno solo. Una reseña puede ser 60% electrónica, 30% atención al cliente y 10% precio

topic modeling como aprendizaje no supervisado

- **no requiere etiquetas**: aprende estructura latente directamente del texto. Es especialmente valioso cuando etiquetar es costoso o el corpus enorme
- a diferencia del clustering (ej: k-means) un documento puede pertenecer a múltiples tópicos con distinto peso
- el resultado principal son dos matrices relacionando: documentos con tópicos y otra con tópicos con palabras
- la validación es indirecta: coherencia interna (**c_v**), interpretabilidad humana, o comparación etiquetas conocidas si las hay (como en nuestro dataset de Amazon)

aplicaciones reales de Topic Modeling

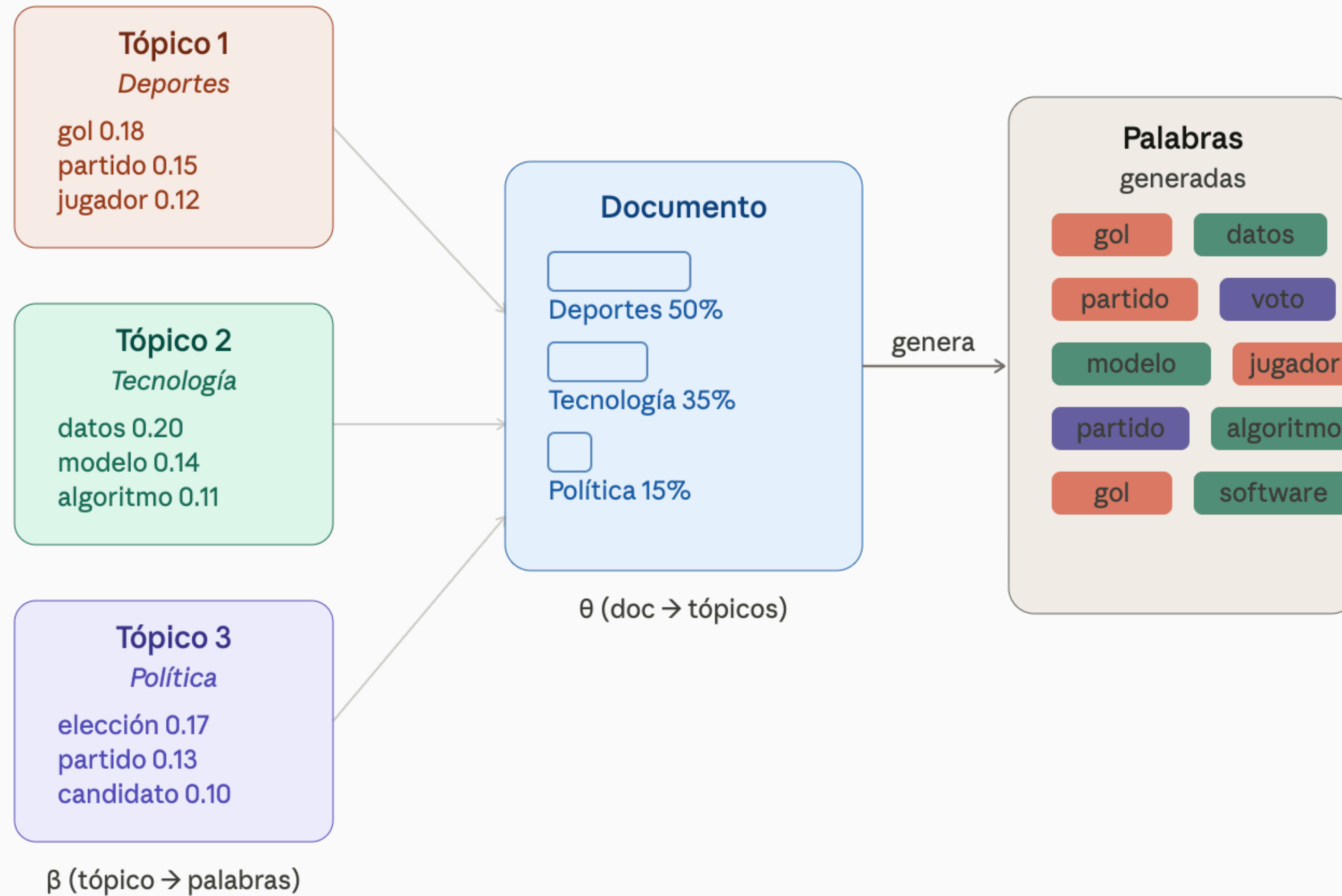
- análisis de prensa
- soporte al cliente
- investigación académica
- recomendación de contenido
- análisis de redes sociales

LDA

Latent Dirichlet Allocation

- Cada documento tiene una distribución sobre K tópicos: $\text{Dir}(\alpha)$
- Cada tópico tiene una distribución sobre el vocabulario: $\text{Dir}(\beta)$
- **modelo generativo bayesiano**: produce **distribuciones** interpretables
 - funciona bien con textos de longitud media
 - limitación: asume **BoW** (sin orden ni contexto)

Proceso generativo de LDA



Leyenda

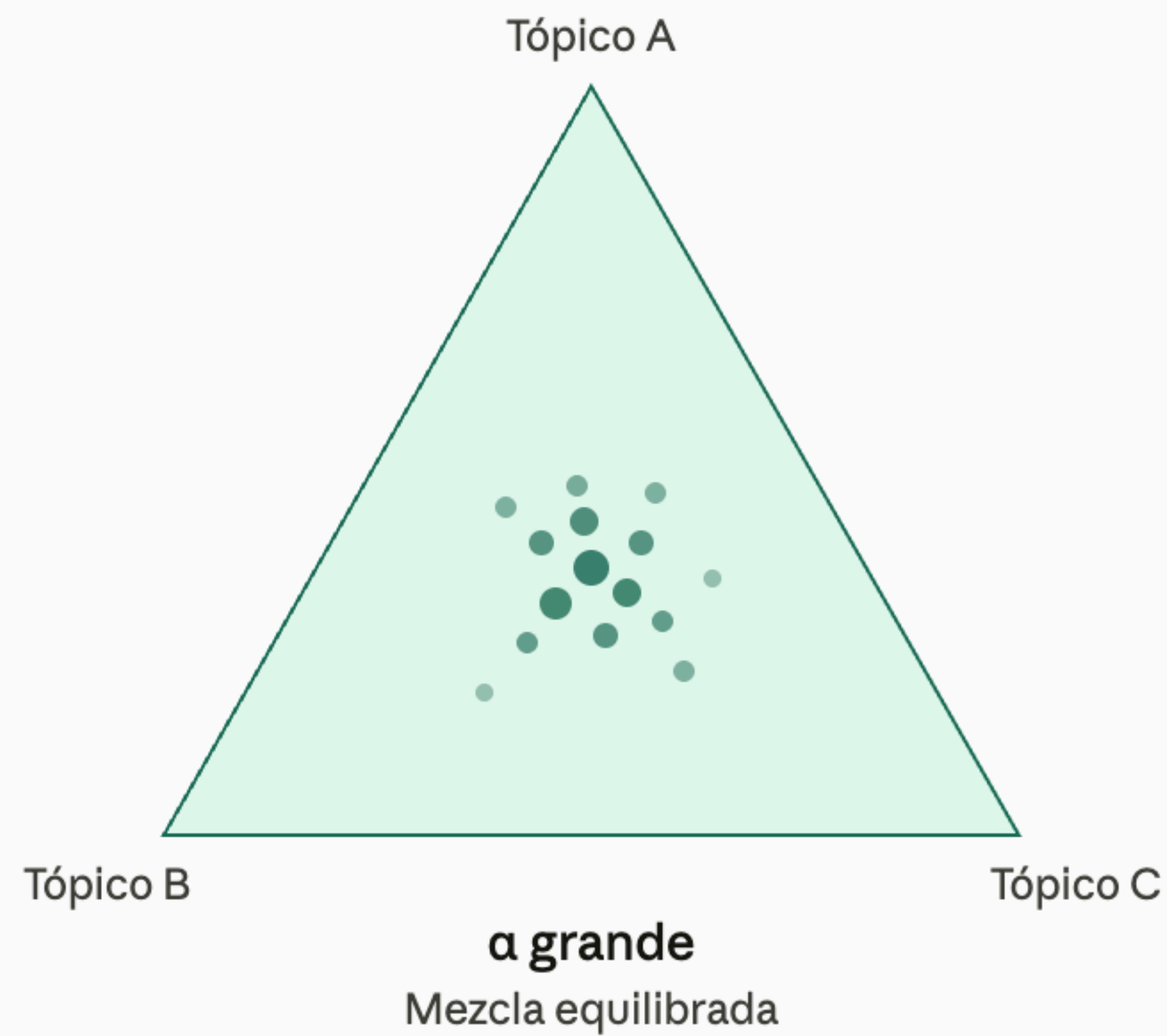
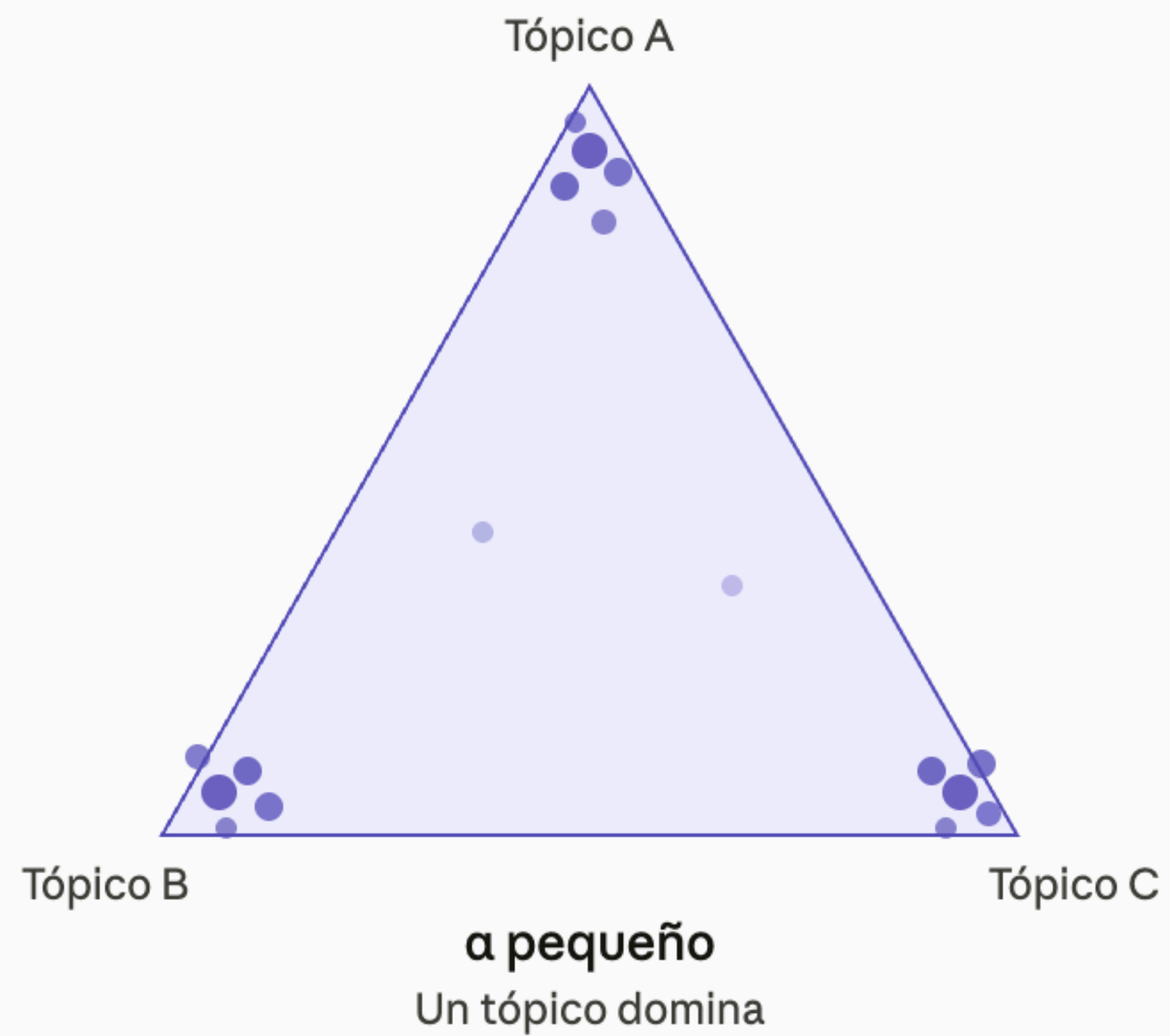
■ Tópico Deportes

■ Tópico Tecnología

■ Tópico Política

hiperparámetros

- **K**: número de tópicos a descubrir. Relacionado con la **coherencia**
 - si es pequeño: tópicos amplios y generales
 - si es grande: tópicos específicos, pero mayor exposición al ruido
- **α** : distribución documento-tópico. Controla cuántos tópicos mezcla cada documento
 - pequeño: cada doc se concentra en pocos tópicos
 - grande: cada doc mezcla muchos tópicos por igual
- **β** : distribución tópico-palabra. Controla cuántas palabras caracterizan cada tópico. Valor típico: 0.01
 - pequeño: tópicos con vocabulario específico y concentrado
 - grande: tópicos con vocabulario amplio y difuso



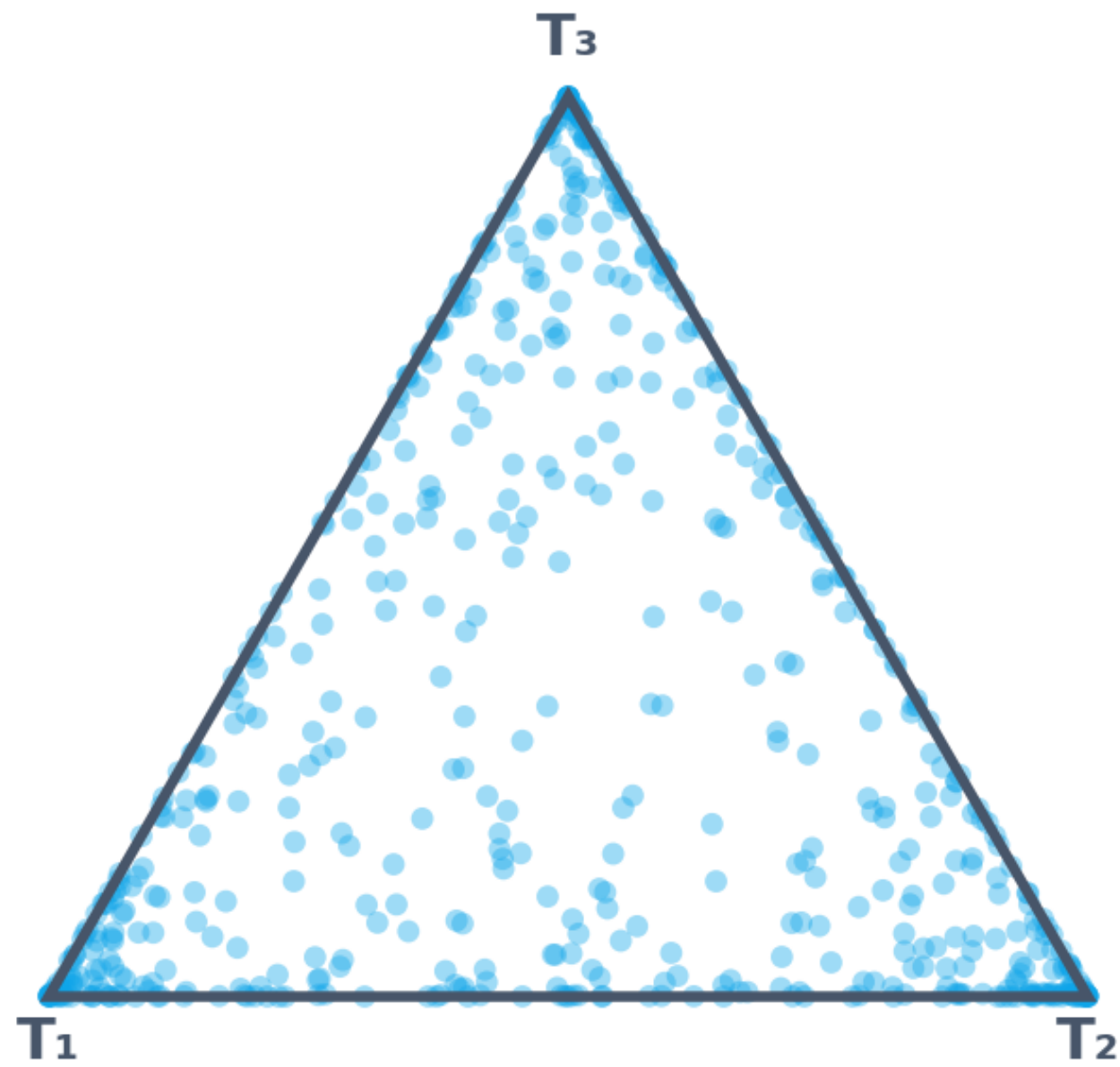
la distribución de Dirichlet

- distribución de probabilidad sobre vectores de probabilidades. Su output es siempre un vector que suma 1. Ideal para representar mezclas
- en LDA se usa dos veces: para θ (mezcla de tópicos) y para ϕ (mezcla de palabras)
- no genera palabras ni tópicos directamente: Genera la *proporción* con la que se mezclan
- Es la distribución que decide "cuánto" de cada tópico tiene un documento

Distribución de Dirichlet — efecto del parámetro α

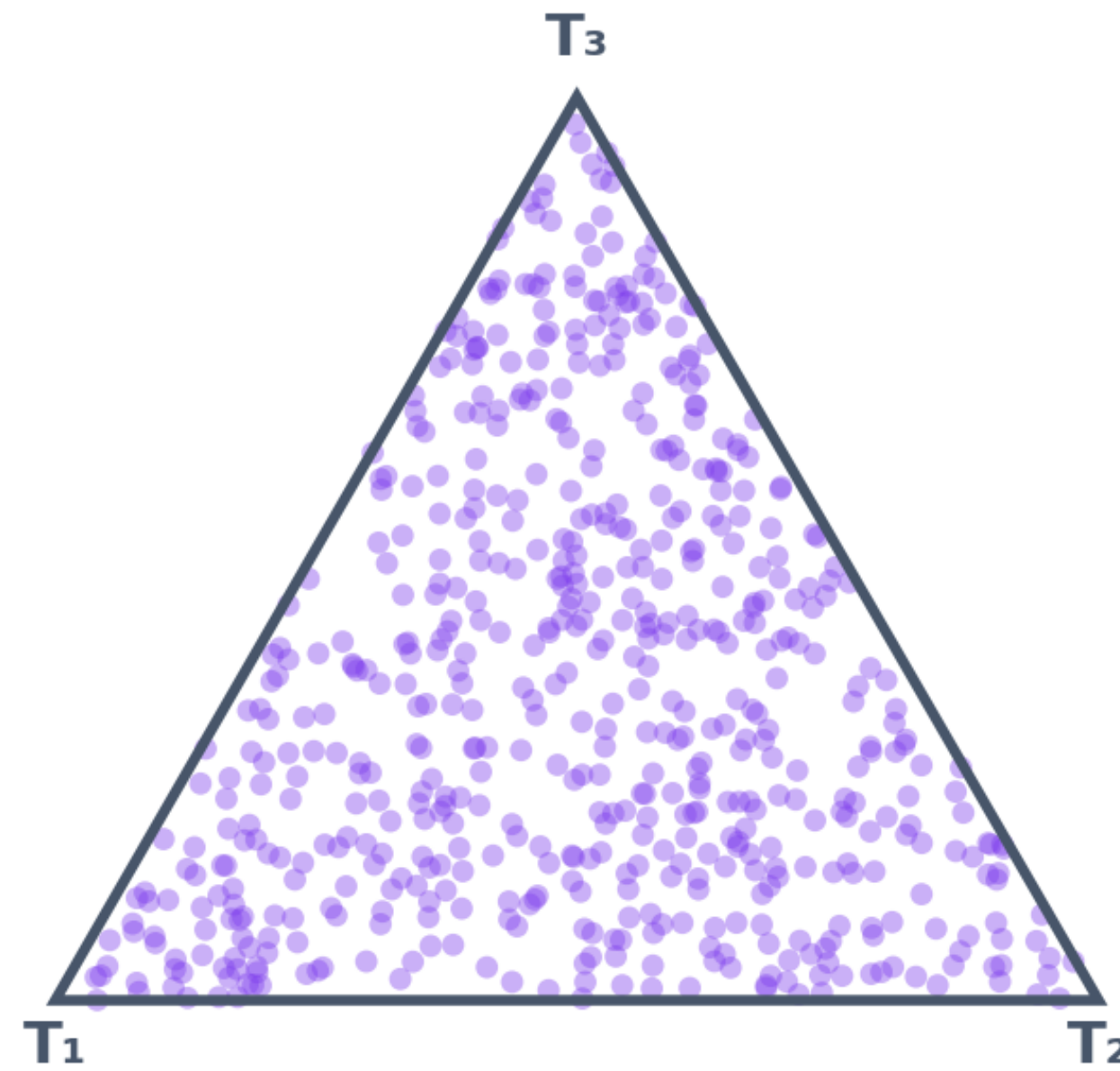
$$\alpha = 0.3 (< 1)$$

Documentos especializados
— concentrados en vértices



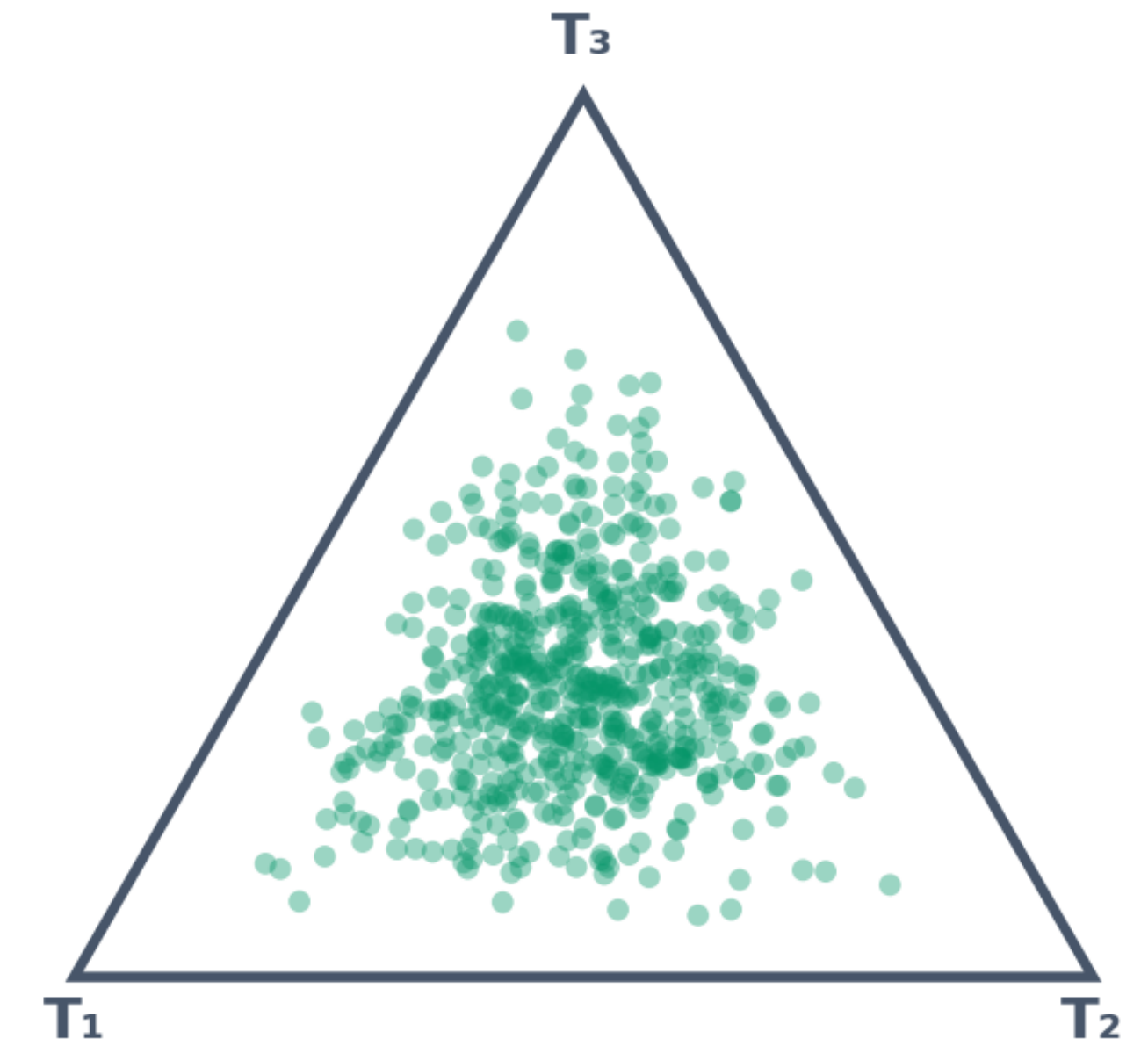
$$\alpha = 1.0$$

Distribución uniforme
sobre el simplex



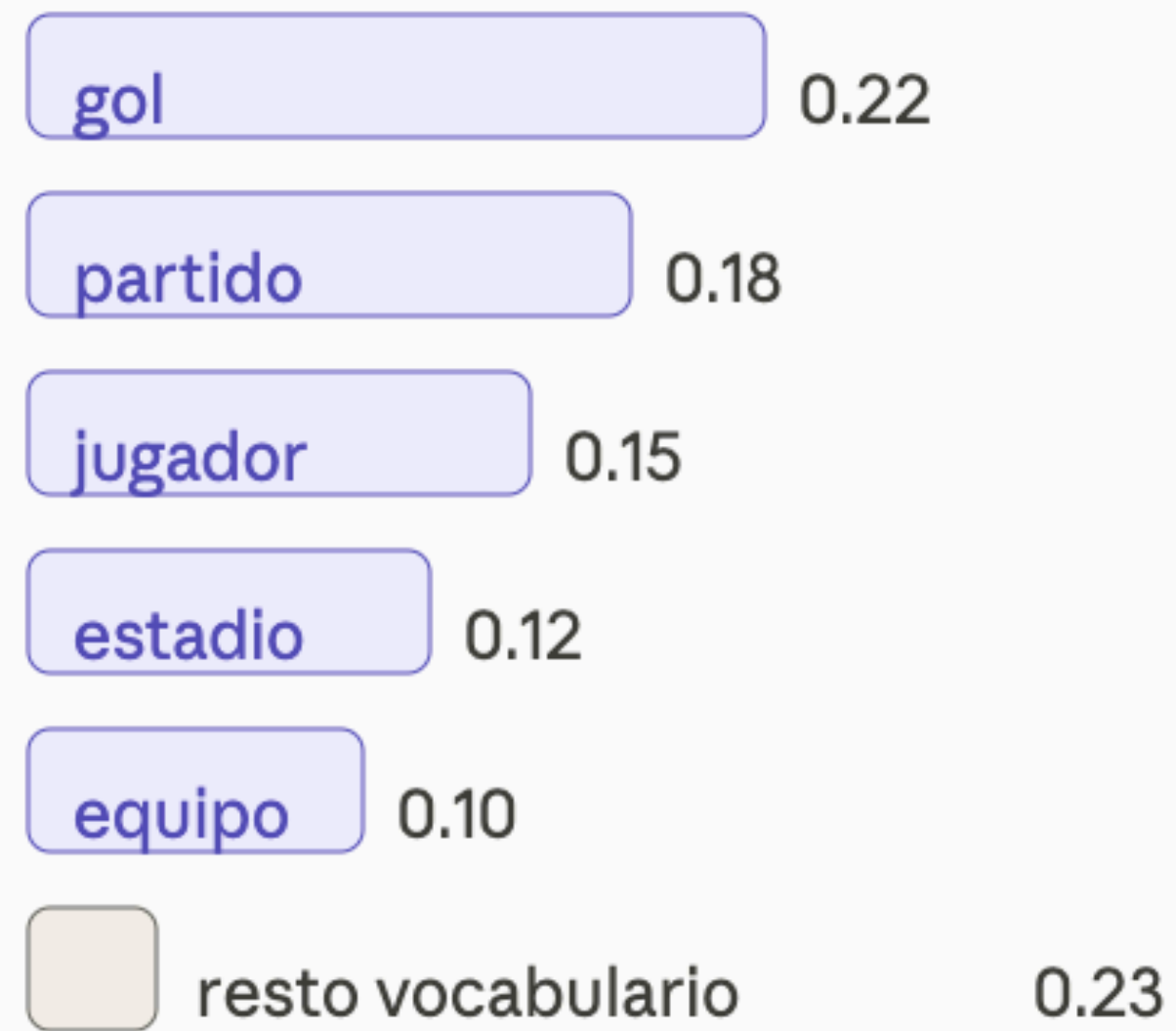
$$\alpha = 5.0 (> 1)$$

Documentos generalistas
— concentrados en el centro



β pequeño

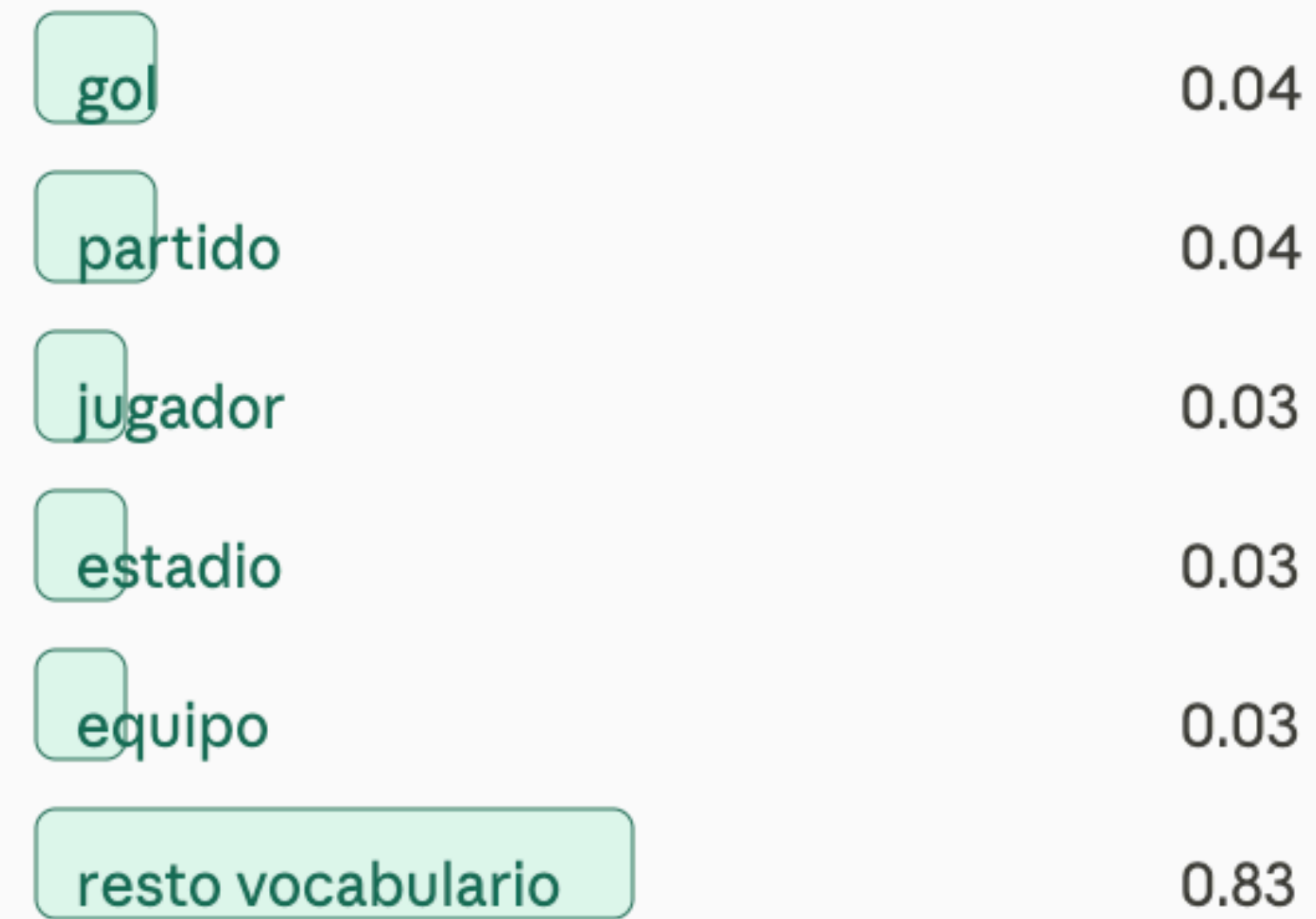
Tópico específico



Pocas palabras, pesos altos
Tópico fácil de interpretar

β grande

Tópico difuso



Muchas palabras, pesos bajos
Tópico difícil de interpretar

la inferencia

encontrar estructura oculta en el texto no etiquetado

- las palabras son las únicas variables observables. El marco probabilístico permite calcular la verosimilitud del corpus y aplicar métodos bayesianos para la inferencia
- hay que inferir:
 - θ (mezclas por doc)
 - φ (mezclas por tópico)
 - z (qué tópico generó cada palabra). Es la variable "puente" que permite estimar las dos anteriores
- la inferencia devuelve:
 - θ por documento: "este doc es 60% tópico 1, 40% tópico 2"
 - φ por tópico: "el tópico 3 tiene estas palabras con estas probabilidades"

las dos distribuciones clave: θ y ϕ

el output de la inferencia

- **theta θ** : distribución de tópicos para el documento d. Vector de dimensión K que suma 1.
Response a **¿de qué temas habla este documento?**
- **phi ϕ** : distribución de palabras para el tópico k. Vector de |V| (vocabulario) que suma 1.
Responde a **¿qué palabras usa este tópico?**
- leemos un tópico en theta mirando las top-N palabras con mayor probabilidad:
 - ej: tópico 1: book (0.08), read (0.06), story (0.05)

preprocesado

¿por qué el preprocesado en Topics es distinto al de Sentimientos?

- en análisis de sentimientos queremos capturar la señal emocional:
 - negaciones: (not good)
 - intensificadores (very bad)
 - modalidad (could have been better)
- en topic modeling queremos capturar la señal temática: sustantivos y verbos de dominio que distinguen un tema de otro. **Las negaciones y los adverbios son ruido**
- eliminar más agresivamente en topics: además de stopwords estándar, **añadimos stopwords de dominio** (producto, item, amazon, but) que aparecen en todas las categorías
- **la lematización es más crítica en topics**: batteries / battery / batería deben confluir en un solo token para que la señal no se disperse

Comparativa de estrategias de preprocesado: Sentimientos vs. Topics

Módulo 3 — Sentimientos

Módulo 4 — Topics

Paso	Sentimientos (Módulo 3)	Topics (Módulo 4)	¿Por qué?
Tokenización	Preservar negaciones (no, never, don't)	Solo palabras de contenido	Sentimientos depende del contexto sintáctico; topics, del léxico temático
Stopwords	Conservar intensificadores (very, extremely)	Eliminar + stopwords de dominio	Los intensificadores cambian la polaridad pero no el tema
Filtro POS	Todos los POS (adverbios, verbos)	NOUN, PROPN, VERB, ADJ	Adverbios y funcionales no aportan señal temática
Lematización	Opcional / stemming	Obligatoria (batteries → battery)	Agrupar variantes morfológicas en un único token

lematizacion y filtrado POS con spaCy

- ¿qué términos mantenemos en el preprocesado?
- KEEP_POS = {'NOUN', 'PROPN', 'VERB', 'ADJ'} conservamos las 4 clases con mayor carga semántica temática. Eliminamos determinantes, preposiciones, conjunciones y adverbios
- **en_core_web_sm** vs **en_core_web_md**: el modelo md incluye vectores GloVe y realiza lematización contextual más precisa
 - Ej: worse -> bad en md, pero worse -> worse en sm

stopwords de dominio

más allá de las genéricas

- las stopwords estándar de spaCy (the, is, at, which, on...) eliminan palabras gramaticales. En reviews de Amazon, hay palabras de dominio igualmente vacías de información temática
- Candidatos a DOMAIN_STOPS: product, item, buy, purchase, order, amazon, review, star, price, use, get, make, come, would, could, one, also, like, great o good
- estrategia para identificarlas: mirar los tokens más frecuentes del vocabulario. Si un token aparece en las 3 categorías con peso similar, probablemente es genérico
- **cuidado**: no hay que eliminar demasiado. Palabras como sound, light, feel pueden ser relevantes en Electronics pero genéricas en Sports. El equilibrio depende del corpus

BoW como representación de entrada

- BoW descarta el orden de las palabras: “the cat sat on the mat” y “the mat sat on the cat” tiene la misma representación. LDA sólo ve co-ocurrencias, no secuencias
- esta limitación es una fortaleza en Topic Modeling: los tópicos son bolsas de palabras, no frases. **El orden no aporta señal temática relevante**

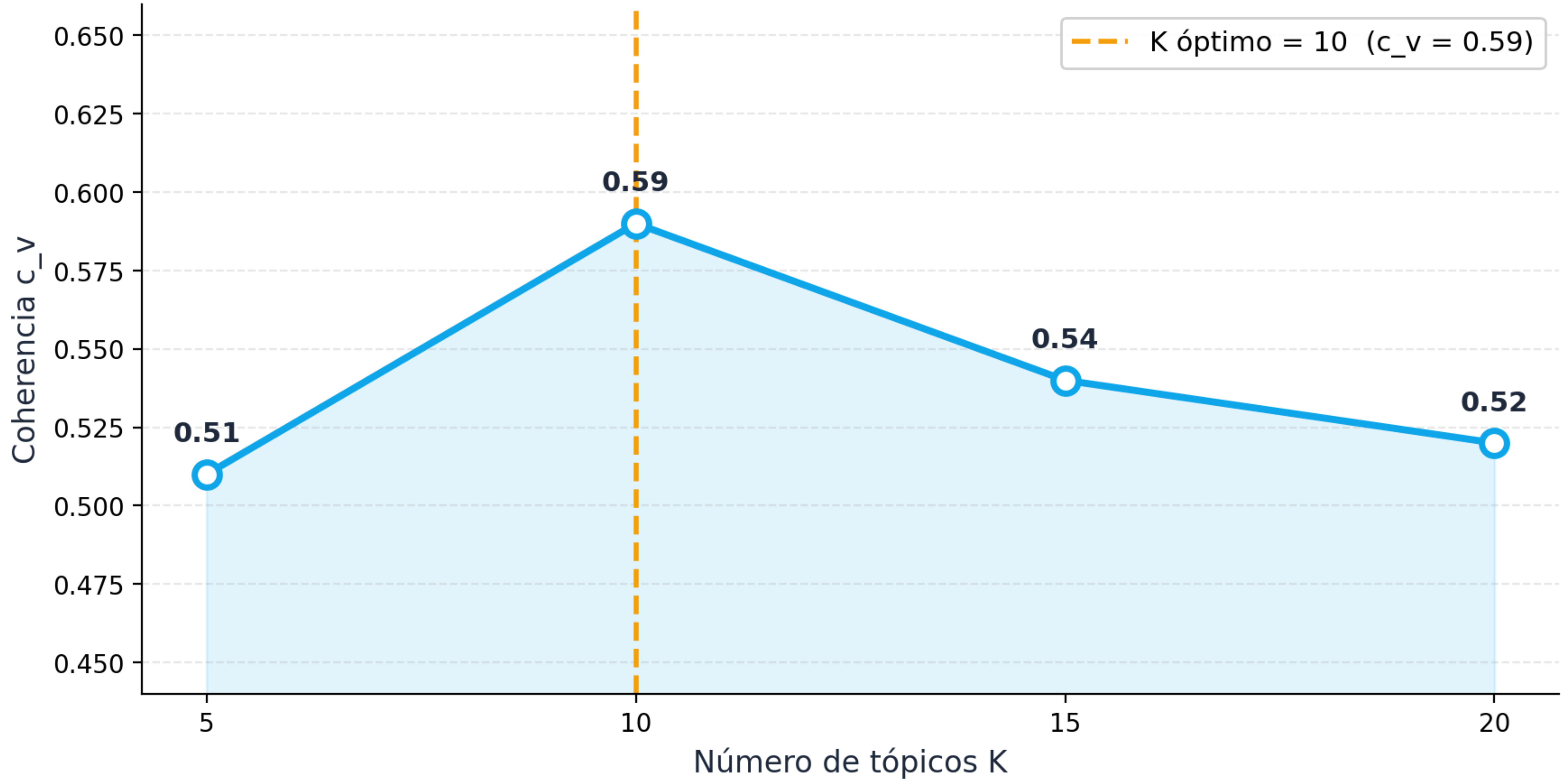
leer e interpretar un tópico

- la probabilidad de la palabra más probable suele ser baja (0.05-0.15): **un tópico bien formado tiene muchas palabras relevantes, no una sola dominante**
- **estilo vs. contenido**: si un tópico tiene palabras muy dispares (easy, love, work, little, look), probablemente es un tópico de estilo (patrón de escritura genérico), no de contenido
- además de las palabras, es útil mirar los documentos con alta concentración en un tópico para validar la interpretación

elegir K: la curva de coherencia c_v

- **la coherencia c_v** mira si las top-N palabras de cada tópico co-ocurren frecuentemente en el corpus. Es el proxy estándar de “interpretabilidad humana”
- el rango típico de c_v es 0.4 - 0.7.
 - Valores > 0.6 son buenos;
 - 0.4 implica tópicos incoherentes
- el procedimiento:
 - entrenar un modelo por cada K en [5, 10, 15, 20]
 - calcular c_v , visualizar y buscar el codo o el pico
 - no usar mayor K donde la curva ya ha caído significativamente
- **problema:** coherencia alta no garantiza utilidad aplicada. Un $k = 5$ puede tener coherencia similar a $K = 10$ pero dar menos información al analista. Hay que combinar con criterio de negocio

Curva de coherencia c_v — elección de K



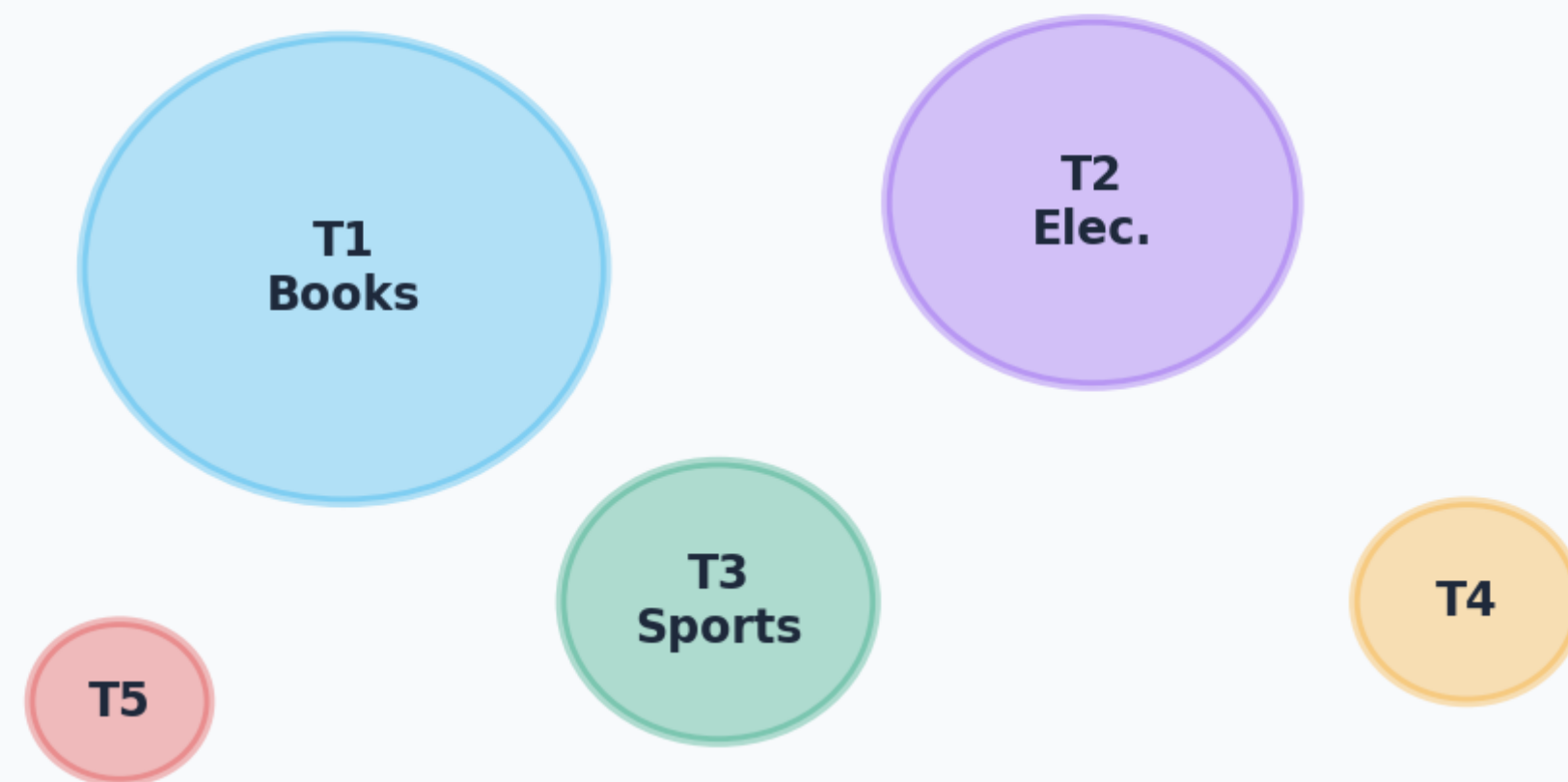
pyLDAvis

el mapa interactivo de tópicos usando PCA

- tópicos cercanos son más similares
- el tamaño del círculo es proporcional a la prevalencia del tópico en el corpus (frecuencia con que es dominante). Los tópicos grandes son los más generales.
- señal de alarma: tópicos que se solapan mucho en el mapa -> el modelo está repitiendo tópicos; considerar reducir K. Tópicos muy pequeños -> fragmentación; considerar reducir K

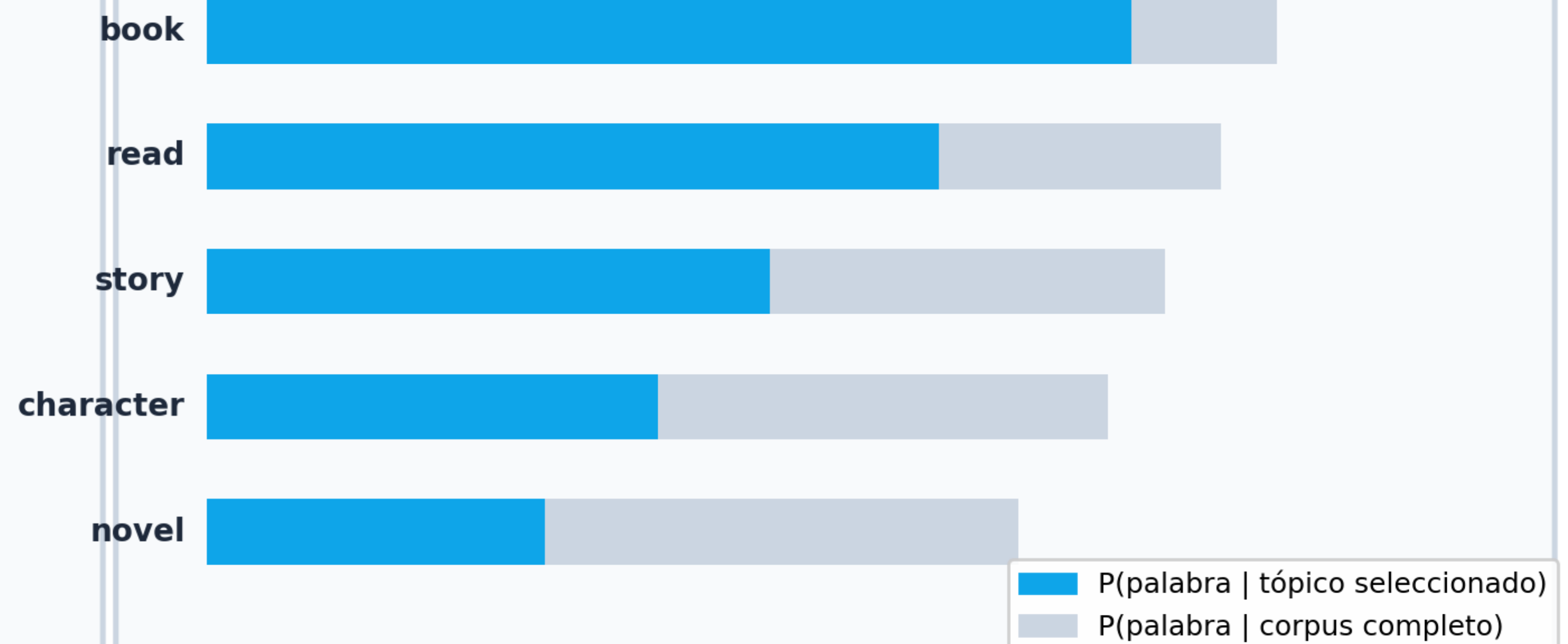
Anatomía del visualizador pyLDAvis

Panel izquierdo — Mapa de tópicos (PC1 / PC2)



Distancia Jensen-Shannon proyectada con PCA

Panel derecho — Top palabras (λ slider)



$\lambda = 1 \rightarrow$ frecuencia global | $\lambda = 0 \rightarrow$ exclusividad del tópico | $\lambda \approx 0.6$ óptimo

validación con ground truth

- con nuestro dataset de Amazon tenemos etiquetas reales con las que podemos comprar el tópico dominante de cada documento con su etiqueta para evaluar la coherencia externa del modelo
- método: para cada documento identificar el tópico dominante y relacionar tópico dominante con categoría. Si LDA funciona, cada categoría debería concentrarse en 1 o 2 tópicos distintos
- resultado observado: books tiene la separación más limpia
 - books -> tópico 8 (68.5%)
 - electronics 2+4 (64%)
 - sports tópico 0 (70%)
- esta validación sólo es posible cuando hay etiquetas. En producción, la interpretación cualitativa y la coherencia c_v son las principales herramientas de evaluación

limitaciones de LDA

tópicos estilísticos y textos cortos

- tópicos estilísticos: LDA no distingue entre contenido y estilo de escritura. Si muchas reseñas comparten easy, love, work, little el modelo crea un tópico aunque no tenga sustancia semántica
- textos cortos: las reseñas de sports son a menudo muy breves. Con pocos tokens por documento, la señal de co-ocurrencia es débil
- orden ignorado: BoW pierde información sobre negaciones, colocaciones y estructura argumental. Esto puede inflar la similitud entre documentos de polaridad opuestas
- K fijo: el número de tópicos debe especificarse a priori

más allá de LDA

BERTopic y modelos neuronales

- BERTopic:
 - embeddings contextuales con sentence-transformers
 - reducción dimensional con UMAP
 - clustering con HDBSCAN
 - representación de tópicos con c-TF-IDF
 - Resuelve el problema de BoW y textos cortos
- la conexión con el bloque anterior (embeddings) es directa: BERTopic usa los mismos embeddings densos
- cuándo seguir usando LDA: corpus pequeños (<100k docs), máquinas sin GPU, necesidad de interpretabilidad probabilística total, o cuando la estabilidad y reproductibilidad son prioritarias sobre la calidad semántica máxima. LDA no está obsoleto, es la base conceptual que hace comprensible BERTopic.

¿verdadero o falso?

- "En LDA, cada documento pertenece a un único tópico"
- "LDA necesita que los documentos estén etiquetados con temas para funcionar"
- "LDA descubre los tópicos automáticamente a partir de las palabras"
- "Si $K = 1$, todos los documentos compartirían el mismo único tópico"
- "LDA trabaja con el orden de las palabras dentro de la frase"

¿verdadero o falso?

- "En LDA, cada documento pertenece a un único tópico"
- "LDA necesita que los documentos estén etiquetados con temas para funcionar"
- "LDA descubre los tópicos automáticamente a partir de las palabras"
- "Si $K = 1$, todos los documentos compartirían el mismo único tópico"
- "LDA trabaja con el orden de las palabras dentro de la frase"

¿Qué representa K en LDA?

- a) El número de palabras del vocabulario
- b) El número de tópicos que el modelo va a descubrir
- c) El número de documentos del corpus
- d) El número de iteraciones del entrenamiento

¿Qué representa K en LDA?

- a) El número de palabras del vocabulario
- b) El número de tópicos que el modelo va a descubrir
- c) El número de documentos del corpus
- d) El número de iteraciones del entrenamiento

Tienes un corpus de noticias y entrenas LDA con $K = 3$

¿Qué obtienes al finalizar?

- a) 3 documentos agrupados por fecha
- b) 3 listas de autores frecuentes
- c) 3 tópicos etiquetados
- d) 3 tópicos, cada uno con sus palabras más probables

Tienes un corpus de noticias y entrenas LDA con $K = 3$

¿Qué obtienes al finalizar?

- a) 3 documentos agrupados por fecha
- b) 3 listas de autores frecuentes
- c) 3 tópicos etiquetados
- d) 3 tópicos, cada uno con sus palabras más probables

¿Qué problema puede ocurrir si eliges un K muy grande?

- a) El modelo tarda menos en entrenar
- b) Los tópicos se vuelven demasiado generales
- c) Aparecen tópicos muy similares entre sí, difíciles de distinguir
- d) El corpus se reduce automáticamente

¿Qué problema puede ocurrir si eliges un K muy grande?

- a) El modelo tarda menos en entrenar
- b) Los tópicos se vuelven demasiado generales
- c) Aparecen tópicos muy similares entre sí, difíciles de distinguir
- d) El corpus se reduce automáticamente

topics | NLU

de NLP a NLU: niveles de análisis lingüístico

diferencias clave

- NLP engloba cualquier procesamiento de texto: desde contar palabras hasta generación
- NLU se enfoca en extraer significado **estructurado** del texto
- aquí vamos a pasar de
 - “¿cuántas veces aparece battery?” a
 - “¿qué dice el cliente sobre la batería?”
- pasamos de etiquetar como reseña positiva a **qué aspecto** del producto es positivo/negativo

la pipeline de spaCy

de texto a anotaciones

- spaCy procesa el texto en una pipeline secuencial: cada componente recibe un Doc anotado por el anterior y añade nuevas anotaciones
- ej: **componentes activos** en en_core_web_sm:

- tok2vec (representación vectorial compartida)

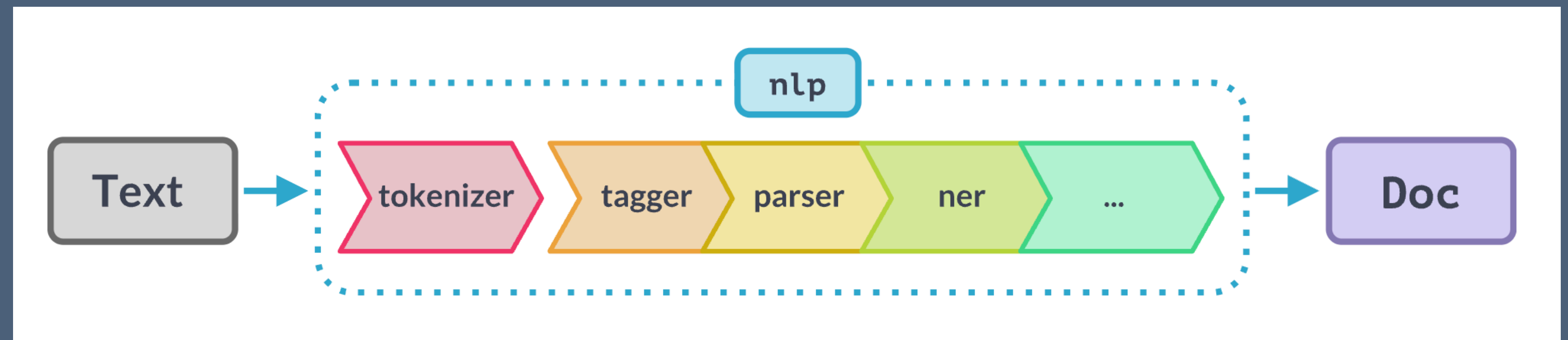
- tagger (POS)

- parser (dependencias)

- NER (entidades)

- attribute_ruler

- lemmatizer



- desactivar componentes innecesarios multiplica la velocidad: para Topics (solo lemas y POS) desactivamos parser y ner -> ~5x más rápido. **Para NER completo necesitamos toda la pipeline**

modelos de spaCy: sm, md, lg, trf

- **sm (small)**: sin vectores de palabras, CNN liviana. Rápido, ~12MB, suficiente para POS, parsing básico y NER estándar
- **md / lg (medium/large)**; incluyen vectores GloVe (685k palabras). Mejoran lematización contextual y similitud semántica. Útiles si queremos `token.vector` o `doc.similarity()`
- **trf (transformer)**: utiliza RoBERTa. ~440 MB, 10-15x más lento que sm pero ~8% mejor en NER y ~6% en parsing. Para producción con GPU
- regla práctica: **sm** para **prototipado**; **md/lg** cuando necesitas **vectores**; **trf** cuando la calidad de **NER** es crítica (como extracción de información en un dominio específico)

POS Tagging

- **POS Tagging (Part of Speech)** asigna a cada token su categoría gramatical: **sustantivo**, **verbo**, **adjetivo**... Es el primer nivel de análisis morfosintáctico
- usos directos en NLP:
 - filtrado para topic modeling (sólo NOUN/VERB(ADJ))
 - lematización precisa (el lema de better es good si es ADJ, pero well si es ADV)
 - extracción de aspectos (buscar ADJ que modifican NOUN)
- los modelos modernos de POS (como tagger de spaCy) usan redes neuronales entrenadas sobre **treebanks** anotados manualmente. Hay una precisión de >97%

Frase de entrada

La joven científica publicó tres artículos importantes

POS tagger

Tokens etiquetados

La

DET

joven

ADJ

científica

NOUN

publicó

VERB

tres

NUM

artículos

NOUN

importantes

ADJ

el estándar universal POS (UPOS)

- **UPOS** define 17 categorías consistentes entre idiomas, parte del proyecto **Universal Dependencies**. Permite transferir pipelines entre idiomas con mínimas modificaciones
- las 5 categorías más importantes para NLP de reseñas:
 - **NOUN** (contenido temático)
 - **ADJ** (valoración)
 - **VERB** (accidental/estado)
 - **ADV** (intensidad/negación)
 - **PROPN** (marcas/productos)
 - **AUX** distingue verbos auxiliares de verbos principales. Importante para análisis temporal y de modalidad
- limitación: en dominio de e-commerce, palabras como wireless pueden etiquetarse como **ADJ** o **NOUN** según el contexto. El tagger comete más errores en neologismos y términos técnicos no vistos en el treebank de entrenamiento

Etiquetas UPOS (Universal Part-of-Speech) — uso con spaCy en reseñas de Amazon

Etiqueta	Categoría	Ejemplos en reseñas de Amazon
 NOUN	Sustantivo	battery, sound, price, quality, screen
 PROPN	Nombre propio	Sony, Amazon, iPhone, Kindle, Bose
 VERB	Verbo	work, buy, last, break, recommend, love
 ADJ	Adjetivo	great, slow, cheap, comfortable, sharp
 ADV	Adverbio	very, really, quickly, not, never, highly
 ADP	Preposición	for, in, on, with, after, during
 DET	Determinante	the, a, my, this, every, another
 CCONJ	Conj. coordinante	and, but, or, yet, so
 PUNCT	Puntuación	. , ! ? — ...
 NUM	Número	5, two, 99.99, 2nd, three months

Un mismo token puede tener distinta
etiqueta POS según el contexto

Los verbos auxiliares y los verbos principales
comparten siempre la misma etiqueta POS

El POS tagging requiere que el texto esté etiquetado manualmente para funcionar

ciudad	
ha	
rápidamente	
el	
Madrid	

ciudad	NOUN
ha	AUX
rápidamente	ADV
el	DET
Madrid	PROPN

El POS tagging es útil en LDA porque
permite filtrar el texto y quedarse solo con
_____ y _____

El POS tagging es útil en LDA porque
permite filtrar el texto y quedarse solo con
NOUN y VERB

Named Entity Recognition (NER)

- **NER** identifica y clasifica menciones de **entidades del mundo real en el texto**: personas, organizaciones, lugares, productos, fechas, cantidades
- los modelos modernos usan transformers para esta forma de etiquetado
- casos de uso en e-commerce: extraer marcas mencionadas (ORG), modelos de producto (PRODUCT), orígenes de envío (GPE), precios (MONEY) y duraciones (DATE/QUANTITY) de forma estructurada
- a diferencia del POS tagger (contexto local), **NER necesita contexto de oración completa**: Apple puede ser ORG o entidad (fruta). El modelo en_core_web_sm resuelve la mayoría de casos con ~85% f1 en OntoNotes

Tipos de entidades NER (en_core_web_sm) con ejemplos de reseñas de Amazon

ORG Sony, Bose, Amazon, Apple, Samsung, Nike	PRODUCT WH-1000XM5, Kindle Paperwhite, AirPods	GPE New York, China, United States, Germany	DATE last December, Black Friday, 2 days
MONEY \$299, 89 dollars, under 100 bucks	QUANTITY 500g, 10 inches, 48 hours, 2000 mAh	CARDINAL 5 stars, three months, one year warranty	WORK_OF_ART "The Great Gatsby", Harry Potter series

NER puede identificar fechas y cantidades
además de personas y lugares

El contexto de la frase no influye
en la detección de entidades

DATE PER ORG LOC MISC

Lionel Messi	
Buenos Aires	
Naciones Unidas	
Champions League	
el próximo martes	

Lionel Messi	PER
Buenos Aires	LOC
Naciones Unidas	ORG
Champions League	MISC
el próximo martes	DATE

NER es útil combinado con LDA porque
permite saber _____ protagoniza cada tópico

Un modelo de NER entrenado en noticias
puede fallar con textos médicos porque ...

Dependency parsing

el árbol de dependencias sintácticas

- el dependency parsing **construye un árbol** donde cada token apunta a su head (**gobernador**) con una etiqueta de relación. La raíz (**ROOT**) no tiene head y suele ser el verbo principal
- cada token expone: `token.head`, `token.dep_`, `token.children`, `token.subtree`
- el árbol codifica la estructura argumental de la oración, quién hace qué a quién. Esto es lo que permite extraer información estructurada más allá de las bolsas de palabras

Arbol de dependencias: "The battery lasts a really long time"



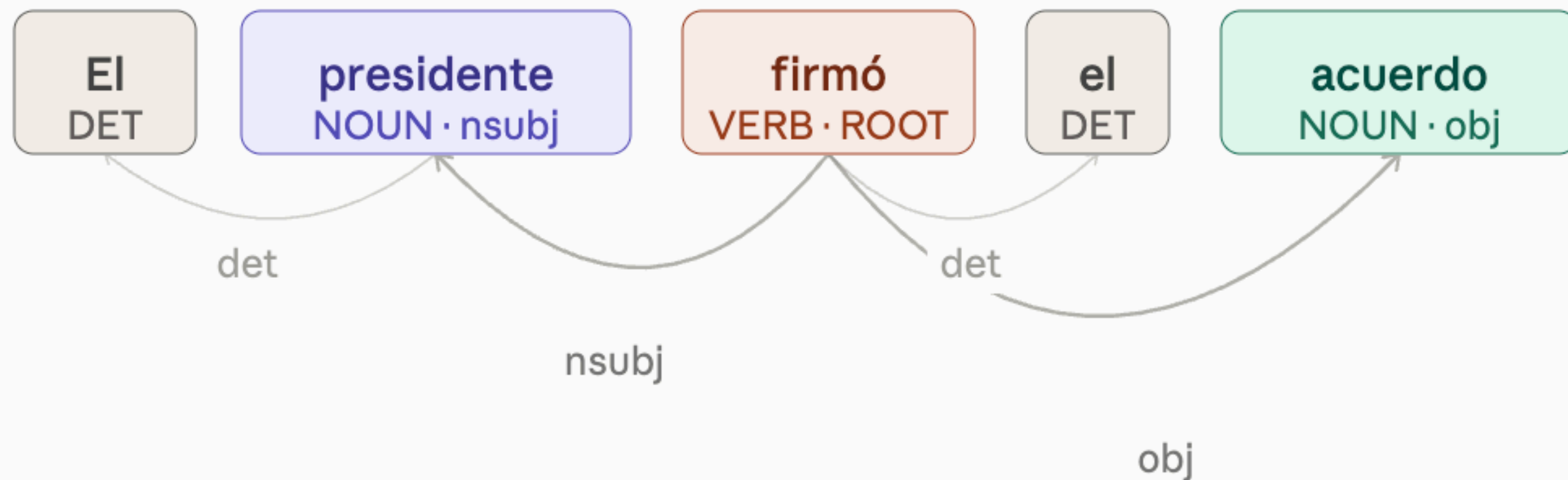
Extraccion SVO: (battery) —[lasts]—> (time) | amod: long -> time | advmod: really -> long

relaciones clave: nsubj, dobj, amod, neg

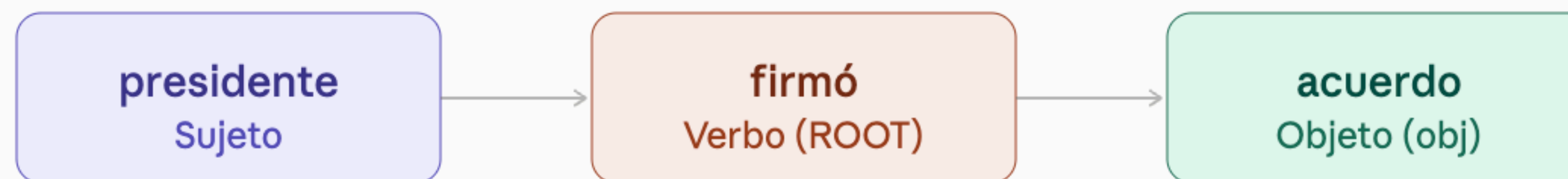
- **nsubj** (normal subject): **sujeto** del verbo. *The camera takes sharp photos*
- **dobj** (direct object): **objeto directo** del verbo. *takes photos*
- **amod** (adjectival modifier): **adjetivo** que modifica a un sustantivo. *sharp photos*
- **neg** (negation modifier): negación del verbo o adjetivo. *does not work*
 - detectar negación antes de asignar polaridad es esencial

extracción de tripletas sujeto-verbo-objeto

- algoritmo:
 - iterar sobre todos los tokens VERB del doc
 - para cada verbo buscar entre sus hijos aquellos etiquetados como sujetos o como objetos
- los modificadores de sustantivo enriquecen la tripleta



Tripleta extraída



Tokens descartados: "El" y "el" (determinantes, sin rol en la tripleta)

La tripleta se forma buscando el verbo raíz (ROOT),
su dependiente nsubj (sujeto) y su dependiente obj (objeto).
Los determinantes y modificadores se descartan.

negación y modalidad en el árbol de dependencias

- **la negación** en inglés se expresa principalmente con not/n't como hijo directo del verbo con dep_='neg'. Detectarla es sencillo: `any(c.dep_=='neg' for c in verb.children)`
- **formas negativas compuestas**: "not very good". Hay que recorrer dos niveles del árbol para capturar la negación del adjetivo
- **modalidad** (could, would, should, might): estos auxiliares (dep_='aux') expresan incertidumbre: *The battery should last longer*. La tripleta SVO existe pero no expresa un hecho, sino una expectativa no cumplida. Señal negativa implícita

En una triplete sujeto-verbo-objeto, el verbo es siempre el núcleo de la relación

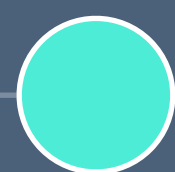
El análisis de dependencias y el POS tagging
son independientes y no se relacionan

El análisis de dependencias es útil para LDA
porque extrae _____ más informativas que las
palabras sueltas

models | transformers

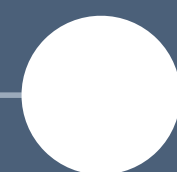
models | transformers

regresión
lineal



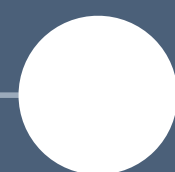
$$y = Wx + b$$

perceptrón
/ MLP



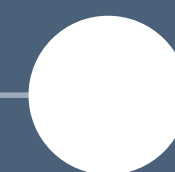
$$y = f(Wx + b)$$

RNN
/ LSTM



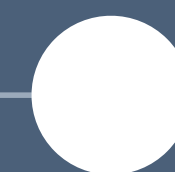
$$h_t = f(Wx_t + Wh_{t-1})$$

atención



$$\text{softmax}(QK^T/\sqrt{d}) \cdot V$$

transformer
BERT·GPT



escala masiva

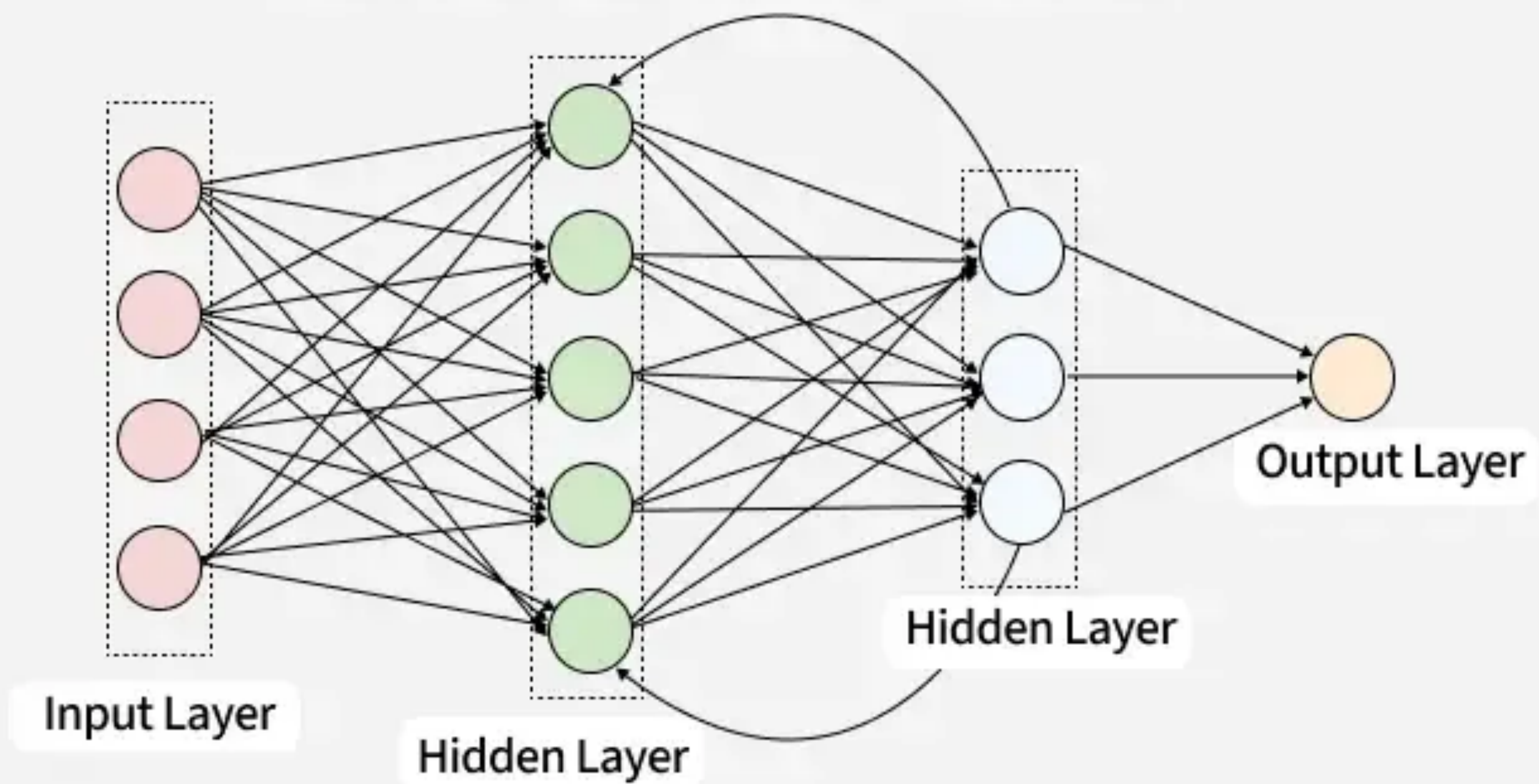
cada paso añade una capacidad nueva manteniendo la misma lógica de optimización

Las RNNs pueden procesar todos los tokens de una frase en paralelo, lo que las hace muy eficientes en entrenamiento.

RNN en NLP

- Procesan el texto palabra a palabra, de izquierda a derecha, secuencialmente
- En cada paso, actualizan un "estado oculto" que resume lo visto hasta ahora. Ese estado viaja de palabra en palabra como una mochila que vas llenando. Cuando se queda sin espacio sobrescribe lo anterior
- Para dotar de un mecanismo de memoria más potente se utilizan LSTMs (Long Short-Term Memory) que añaden un mecanismo adicional de memoria con el que pueden incluso filtrar qué guardar y qué no.
 - Limitación: siguen sin ser capaces de relacionar la palabra x con la $x+1$ (secuencialidad)
 - el modelo «olvida» lo lejano

Recurrent Neural Network



Una RNN procesa la frase "El banco del río estaba desbordado". Tiene capacidad para guardar 4 palabras

¿Cuál es su principal problema al llegar a "desbordado"?

- A) No conoce la palabra "desbordado"
- B) La información de "banco" se ha diluido en el estado oculto
- C) No puede manejar más de 5 tokens
- D) Confunde mayúsculas y minúsculas

Una RNN procesa la frase "El banco del río estaba desbordado". Tiene capacidad para guardar 4 palabras

¿Cuál es su principal problema al llegar a "desbordado"?

- A) No conoce la palabra "desbordado"
- B) La información de "banco" se ha diluido en el estado oculto
- C) No puede manejar más de 5 tokens
- D) Confunde mayúsculas y minúsculas

En una RNN, la misma palabra "banco" produciría exactamente el mismo estado oculto sin importar el contexto de la frase

¿por qué esto no basta para texto?

- el problema del orden: una **bolsa de palabras** trata igual «el perro muerde al hombre» que «el hombre muerde al perro»
- longitud variable: las frases tienen **longitudes distintas**. un MLP necesita entrada de tamaño fijo – truncar o hacer padding pierde información
- dependencias largas: en «el banco que está al lado del río estaba cerrado», banco y cerrado están separados pero relacionados
- la solución recurrente: mantener un **estado oculto h_t** que resume todo lo visto hasta el token t . la memoria viaja de paso en paso. Pero esto es insuficiente

atención

- En lugar de leer en orden, el mecanismo de atención mira todas las palabras a la vez.
- Para cada palabra, calculamos cuánto atiende una palabra al resto: **una matriz de pesos**
- A diferencia de lo anterior este mecanismo permite paralelizar la procesamiento de las palabras

$$\textit{Attention}(\underset{\substack{\text{from}}}{\textcolor{blue}{q}}, \underset{\substack{\text{to}}}{\textcolor{brown}{k}}, \textcolor{red}{v}) = \overbrace{\textit{softmax}\left(\frac{\textcolor{blue}{q}\textcolor{brown}{k}^T}{\sqrt{d_k}}\right)}^{\text{Attention weights}} \textcolor{red}{v}$$

vector dimensionality of K, V

En el mecanismo de self-attention, ¿qué representa el peso de atención entre dos tokens?

- A) La distancia posicional entre ellos en la frase
- B) La frecuencia con la que aparecen juntos en el corpus
- C) Cuánto debe influir un token en la representación del otro
- D) El coste computacional de procesarlos juntos

En el mecanismo de self-attention, ¿qué representa el peso de atención entre dos tokens?

- A) La distancia posicional entre ellos en la frase
- B) La frecuencia con la que aparecen juntos en el corpus
- C) Cuánto debe influir un token en la representación del otro
- D) El coste computacional de procesarlos juntos

En self-attention, cada token solo puede atender a los tokens que aparecen antes de él en la frase.

¿Cuál de estas frases ilustra mejor la intuición del mecanismo de atención?

- A) Cada token se actualiza sumando el embedding de todos los tokens
- B) Para entender una palabra, el modelo decide cuánto "mirar" a cada otra palabra del contexto
- C) La atención sustituye los embeddings por vectores de posición
- D) Cada token se procesa de izquierda a derecha como en una RNN

¿Cuál de estas frases ilustra mejor la intuición del mecanismo de atención?

- A) Cada token se actualiza sumando el embedding de todos los tokens
- B) Para entender una palabra, el modelo decide cuánto "mirar" a cada otra palabra del contexto
- C) La atención sustituye los embeddings por vectores de posición
- D) Cada token se procesa de izquierda a derecha como en una RNN

Transformers

- Basado en la idea anterior se construye una arquitectura con los siguientes componentes:
 - **embeddings**: representación vectorial
 - **self-attention**: cada token mira a todos los demás
 - **feed-forward**: la red neuronal que procesa toda la secuencia para aprender patrones
 - **apilado de capas**: los 3 pasos anteriores se repiten N veces (BERT-base tiene 12, pero GPT-4 tiene 96)

¿Cuál es la limitación principal de las LSTMs que motivó el desarrollo del Transformer?

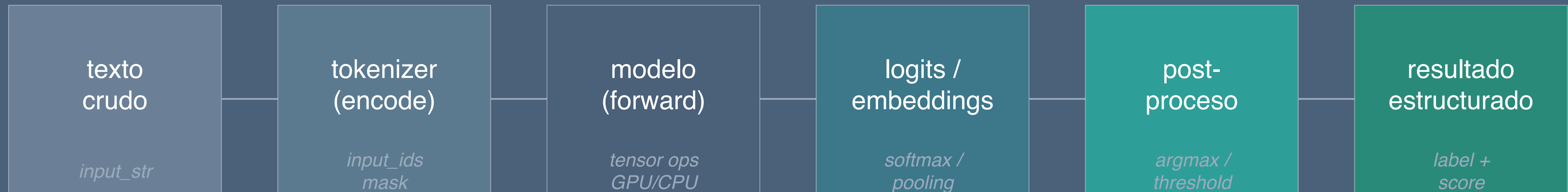
- A) Usan demasiada memoria RAM
- B) Son incapaces de procesar texto en español
- C) Su procesamiento secuencial impide la paralelización en entrenamiento
- D) No pueden manejar vocabularios grandes

¿Cuál es la limitación principal de las LSTMs que motivó el desarrollo del Transformer?

- A) Usan demasiada memoria RAM
- B) Son incapaces de procesar texto en español
- C) Su procesamiento secuencial impide la paralelización en entrenamiento
- D) No pueden manejar vocabularios grandes

flujo interno de pipeline() en Hugging Face

de texto crudo a resultado estructurado



pipeline() gestiona tokenización · dispositivo (CPU/GPU) · padding · postproceso automáticamente



```
from transformers import pipeline

# Sentiment analysis
clf = pipeline("sentiment-analysis")
clf("Este curso me está encantando")
# → [{'label': 'POSITIVE', 'score': 0.9998}]

# Zero-shot: tú defines las categorías
clf2 = pipeline("zero-shot-classification")
clf2("El Madrid ganó 3-0", candidate_labels=["deporte", "política", "economía"])
```

modelos pre-entrenados

- El modelo ya ha procesado enormes cantidades de texto
- Ha aprendido representaciones generales del lenguaje: gramática, semántica, hechos del mundo
- Limitaciones:
 - Falta de jerga específica
 - Sesgos
 - En inglés el desempeño es mayor que en otros idiomas
 - A partir de una fecha no reconoce nuevos eventos (no ha visto las noticias del último día)

	BERT	GPT
Dirección de atención	Bidireccional (ve todo el contexto)	Unidireccional (solo ve lo anterior)
Tarea de preentrenamiento	Rellenar palabras enmascaradas	Predecir la siguiente palabra
Uso típico	Clasificación, Q&A, NER	Generación de texto, chat
Ejemplo de modelo actual	RoBERTa, DeBERTa	GPT-4, Llama, Mistral

Taxonomía de modelos

- Todo modelo del Hub pertenece a una de estas tres familias según su arquitectura:
 - **Encoder** (ej. BERT, RoBERTa): entiende texto, no genera. Ideal para clasificar, extraer, comparar.
 - **Decoder** (ej. GPT-2, Llama, Mistral): genera texto token a token. Ideal para completar, resumir, chatear.
 - **Encoder-Decoder** (ej. T5, BART, mT5): transforma texto en texto. Ideal para traducir, resumir con control, Q&A generativo.

Regla práctica:

si la salida es una etiqueta o un número → encoder.

si la salida es texto libre → decoder o encoder-decoder.

Modelos Encoder: para entender

casos de uso

- Clasificación de texto (sentimiento, tema, intención)
- Reconocimiento de entidades (NER): personas, lugares, organizaciones
- Question Answering extractivo: localizar la respuesta en un texto
- Similitud semántica: ¿estas dos frases dicen lo mismo?
- Embeddings: representar texto como vector para búsqueda o clustering

Modelos representativos:

- bert-base-uncased: inglés, punto de partida
- roberta-base: BERT mejorado, más robusto
- dccuchile/bert-base-spanish-wwm-cased (BETO): BERT en español
- PlanTL-GOB-ES/roberta-base-bne: RoBERTa entrenado con corpus español masivo
- **Cuando NO usarlos**: cuando necesitas que el modelo genere texto nuevo, no solo analice el que das

Modelos Decoder: para generar

casos de uso

- Completar texto o código a partir de un prompt
- Chatbots y asistentes conversacionales
- Resumen abstractivo (el modelo reescribe con sus propias palabras)
- Generación de datos sintéticos
- Razonamiento paso a paso

Modelos representativos

- gpt2: el clásico open source, pequeño, bueno para aprender
- mistral/Mistral-7B-Instruct-v0.2: muy capaz, open weights, corre en local con GPU modesta
- meta-llama/Meta-Llama-3-8B-Instruct: estado del arte open weights a fecha de este curso
- google/gemma-2-2b-it: pequeño y eficiente, bueno para entornos con recursos limitados
- **Cuándo NO usarlos**: cuando necesitas una etiqueta concreta o una extracción precisa – los generativos pueden alucinar o dar formato inconsistente.

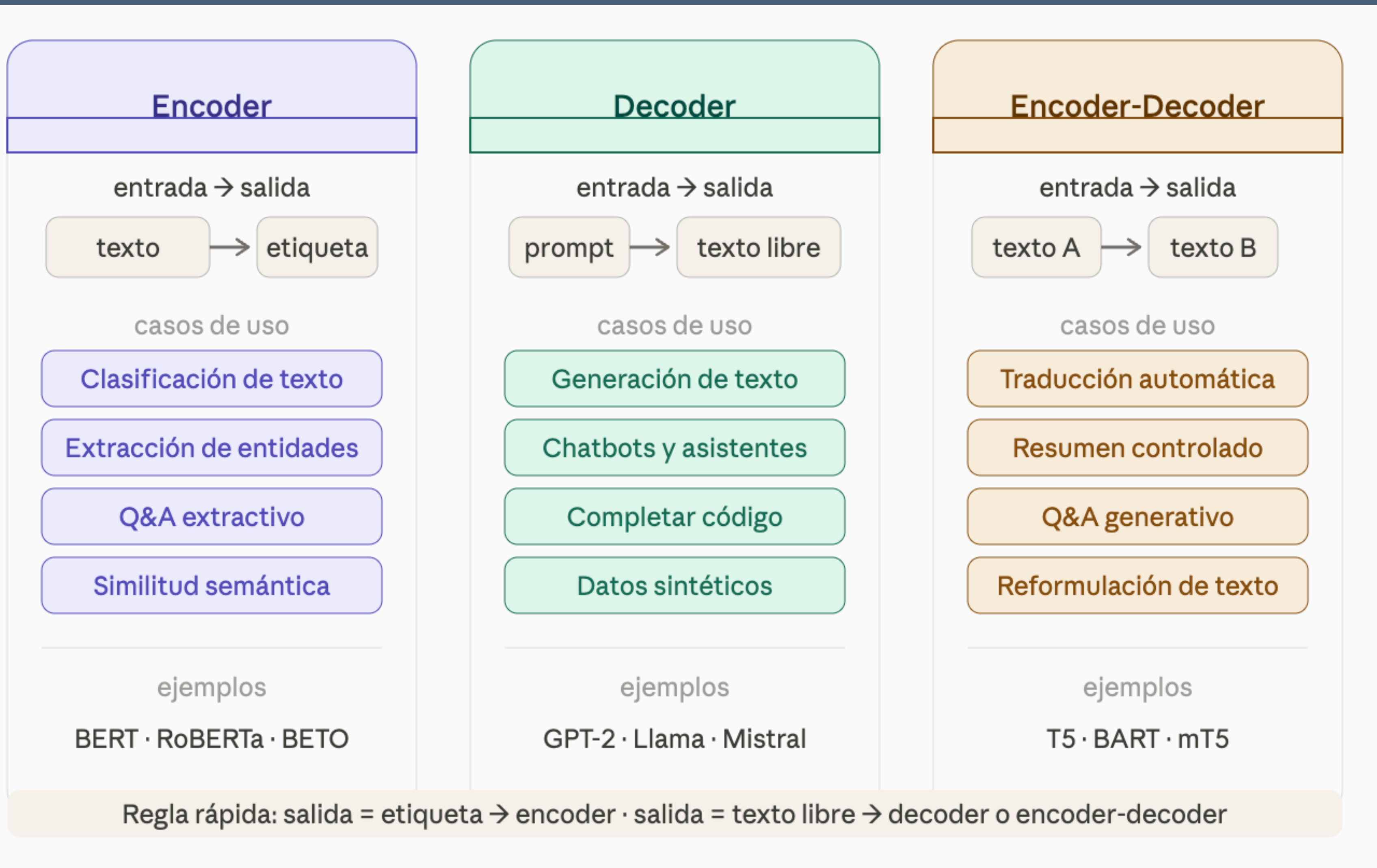
Modelos Encoder-Decoder: para transformar

Casos de uso

- Traducción automática
- Resumen con longitud o estilo controlado
- Reformulación y corrección de texto
- Question Answering generativo (el modelo construye la respuesta, no la extrae)
- Rellenar plantillas a partir de instrucciones

Modelos representativos

- t5-base/t5-large: todo en uno: traducción, resumen, Q&A con el mismo modelo
- facebook/bart-large-cnn: muy bueno para resumen de noticias
- Helsinki-NLP/opus-mt-es-en: traducción español → inglés, ligero
- google/flan-t5-base: T5 ajustado con instrucciones, más fácil de usar con prompts
- **Cuándo usarlos sobre un decoder puro**: cuando la salida tiene una estructura clara que depende del input (traducción, resumen de longitud fija) y quieres más control que con generación libre.



Quieres construir un sistema que lea CVs y devuelva una etiqueta: "apto" o "no apto"

¿Qué familia de modelos es la más adecuada?

- A) Decoder (GPT-style)
- B) Encoder (BERT-style)
- C) Encoder-Decoder (T5-style)
- D) Cualquiera, da igual la familia

Un modelo decoder como GPT-2 puede usarse para traducir texto de español a inglés con el mismo rendimiento que un modelo encoder-decoder entrenado específicamente para traducción.

Si la salida es texto libre, usar decoder; si la salida es una etiqueta o número, usar encoder

Cómo leer una Model Card

- **Intended uses & limitations** – para qué fue diseñado y para qué no. Si tu caso no está aquí, asume que puede fallar.
- **Training data** – en qué idiomas, dominios y fechas se entrenó. Un modelo entrenado con noticias de 2020 no conoce eventos recientes.
- **Evaluation results** – métricas en benchmarks estándar. Permiten comparar modelos de la misma familia.
- **License** – MIT y Apache 2.0 permiten uso comercial libre. Llama y algunos otros tienen restricciones. Siempre verificar antes de producción.
- **Carbon footprint** – algunos cards incluyen el coste de entrenamiento. Relevante para decisiones de sostenibilidad.
- Señales de alerta: sin Model Card, sin datos de entrenamiento documentados, sin métricas de evaluación → usar con mucha cautela o no usar

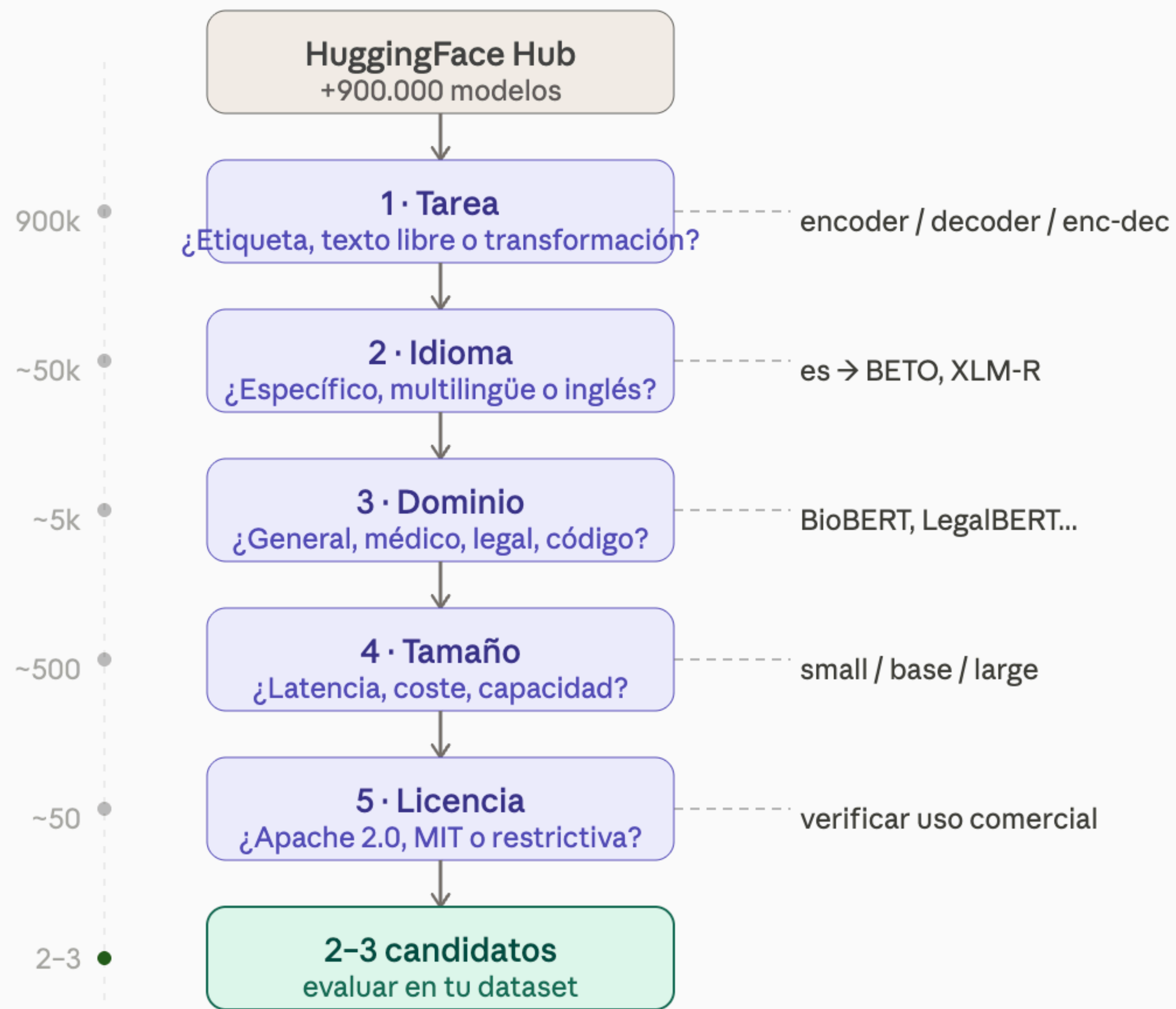
Cinco criterios para elegir modelo

- **Tarea** – ¿la salida es una etiqueta, texto libre o una transformación? Determina la familia (encoder / decoder / enc-dec).
- **Idioma** – ¿el modelo fue entrenado en tu idioma? Para español, buscar modelos específicos o multilingües antes que asumir que el inglés funciona.
- **Dominio** – ¿el texto es general, técnico, legal, médico? Un modelo de dominio general puede fallar en jerga especializada.
- **Tamaño** – modelos grandes son más capaces pero más lentos y costosos. Para producción con alto volumen, el tamaño importa mucho.
- **Licencia** – verificar siempre antes de cualquier uso comercial o con datos sensibles.

Modelos especializados que conviene conocer

Más allá de los genéricos, existen modelos entrenados para dominios o tareas específicas

- **Código:** salesforce/codegen-350M-mono, bigcode/starcoder2-3b – entrenados con repositorios de código
- **Biología / medicina:** allenai/biomed_roberta_base, PlanTL-GOB-ES/bsc-bio-ehr-es – corpus clínico español
- **Legal:** nlpaueb/legal-bert-base-uncased – entrenado con documentos jurídicos
- **Multilingüe:** xlm-roberta-base – 100 idiomas, un solo modelo, buen punto de partida antes de buscar específico
- **Embeddings / búsqueda semántica:** sentence-transformers/paraphrase-multilingual-mpnet-base-v2 – diseñado para comparar frases, no para clasificar



Vas a procesar historiales médicos de pacientes en español

¿Cuál es el primer criterio que debes verificar en la Model Card antes de cualquier otro?

- A) El número de descargas del modelo en el Hub
- B) La licencia y las restricciones de uso con datos sensibles
- C) El tamaño del modelo en parámetros
- D) La fecha de la última actualización

Vas a procesar historiales médicos de pacientes en español

¿Cuál es el primer criterio que debes verificar en la Model Card antes de cualquier otro?

- A) El número de descargas del modelo en el Hub
- B) La licencia y las restricciones de uso con datos sensibles
- C) El tamaño del modelo en parámetros
- D) La fecha de la última actualización

Prompting

- El prompt es la instrucción completa que le das al modelo: todo lo que procesa antes de generar su respuesta
- No es solo "la pregunta": incluye contexto, ejemplos, restricciones de formato y rol
- La calidad del output depende directamente de la calidad del prompt – con el mismo modelo, un prompt malo y uno bueno pueden dar resultados radicalmente distintos
- Prompting no es magia ni truco: es diseño de instrucciones

Componentes de prompting

- **Rol** – quién debe ser el modelo en esta interacción
- **Contexto** – información de fondo que el modelo necesita
- **Tarea** – qué debe hacer exactamente
- **Formato de salida** – cómo debe estructurar la respuesta
- **Ejemplos** – uno o varios pares input/output de referencia
- **Restricciones** – qué no debe hacer o qué debe evitar

PROMPT

Rol

Eres un analista de negocio senior.

Contexto

La empresa opera en el sector retail español.

Tarea

Clasifica la siguiente reseña de cliente.

Ejemplos

"Llegó tarde" → NEGATIVA · "Perfecto" → POSITIVA

Formato

Responde solo con: POSITIVA, NEGATIVA o NEUTRA.

Restricciones

Sin explicación. Si no es claro, responde NEUTRA.

Modelo
LLM

NEGATIVA

output

Solo la tarea es obligatoria — el resto se añade cuando el output lo requiere

Por qué importa el prompting

- Prompt A: "Resume este texto." → Resumen de longitud variable, en el idioma que el modelo prefiera, con el formato que decida.
- Prompt B: "Eres un analista de negocio. Resume el siguiente texto en exactamente 3 puntos, cada uno en una línea, en español, usando verbos de acción. Texto: [...]" → Resultado predecible, accionable, integrable en un pipeline.
- La diferencia no está en el modelo – está en la instrucción
- En producción, un prompt mal diseñado es tan costoso de mantener como código mal escrito

Zero-shot prompting

- Se le pide al modelo que realice una tarea sin darle ningún ejemplo previo
- Funciona cuando la tarea es suficientemente común en los datos de entrenamiento del modelo
- Es el **punto de partida**: siempre probar zero-shot primero antes de añadir complejidad
- Cuándo falla:
 - Tareas muy específicas del dominio que el modelo no ha visto en entrenamiento
 - Cuando el formato de salida es crítico y el modelo lo ignora
 - Tareas que requieren conocimiento propietario o interno



Clasifica la siguiente reseña como POSITIVA, NEGATIVA o NEUTRA.

Reseña: "El envío tardó más de lo esperado pero el producto es exactamente lo que buscaba."

Respuesta: ...

One-shot y Few-shot prompting

- Se incluyen uno o varios ejemplos del par input/output esperado antes de la tarea real
- El modelo aprende el patrón implícitamente de los ejemplos, sin reentrenamiento
- One-shot: un ejemplo. Few-shot: entre 2 y ~8 ejemplos. Más de 8 raramente mejora y encarece el prompt.
- Buenas prácticas:
 - Cubrir los casos edge, no solo los fáciles
 - Mantener el formato exactamente igual en todos los ejemplos
 - Equilibrar las clases si es una tarea de clasificación
 - Variar el estilo de los ejemplos para que el modelo no sobreajuste al formato



Texto: "¿Cuánto tarda el envío a Madrid?" → Intención: CONSULTA_ENVÍO
Texto: "Quiero cancelar mi pedido" → Intención: CANCELACIÓN
Texto: "El producto llegó roto" → Intención: RECLAMACIÓN
Texto: "¿Aceptáis devoluciones en tienda?" → Intención:

Chain-of-Thought (CoT)

- En lugar de pedir directamente la respuesta, se pide al modelo que razone paso a paso antes de responder
- Mejora drásticamente el rendimiento en tareas de razonamiento, matemáticas, lógica y decisiones multi-paso
- La forma más sencilla: añadir "Piensa paso a paso antes de responder" al final del prompt
- Por qué funciona: el modelo usa los tokens intermedios del razonamiento como "memoria de trabajo". Al forzar la cadena de pensamiento, reduce el error de saltos lógicos.



Un tren sale de Madrid a 200 km/h y otro de Barcelona a 160 km/h.
La distancia es 620 km. ¿Cuánto tardan en encontrarse?

Piensa paso a paso antes de responder.

→ Velocidad relativa: $200 + 160 = 360$ km/h.

Tiempo = $620 / 360 = 1,72$ horas $\approx$ 1h 43min.

Respuesta: 1 hora y 43 minutos.

Zero-shot

0 ejemplos

prompt
instrucción
texto de entrada

respuesta directa

úsalo cuando

tarea común y bien definida
formato simple
punto de partida siempre

Few-shot

2-5 ejemplos

prompt
ej. 1: input → output
ej. 2: input → output

texto de entrada

respuesta directa

úsalo cuando

tarea poco común o con matices
formato muy específico
zero-shot falla

Chain-of-Thought

razonamiento explícito

prompt
instrucción

texto de entrada

Piensa paso a paso.

paso 1... paso 2... paso 3...

respuesta final

úsalo cuando

razonamiento multi-paso
cálculos o decisiones
lógica encadenada

Empezar siempre en zero-shot · añadir complejidad solo si el output lo requiere

Salida estructurada

- Por defecto, un modelo generativo produce texto libre: útil para leer, difícil de procesar en código
- En producción casi siempre necesitamos que la salida sea parseable: JSON, listas, tablas, campos concretos
- La técnica es simple: especificar el formato exacto en el prompt y, si es posible, mostrar la estructura esperada
- Buenas prácticas:
 - Decir explícitamente "sin texto adicional" o "solo el JSON"
 - Proporcionar la estructura exacta con los nombres de campo
 - Validar siempre la salida con un parser antes de usarla (los modelos pueden añadir texto fuera del JSON)
 - Para casos críticos: usar librerías como outlines o la funcionalidad de structured outputs de la API



Extrae las entidades del siguiente texto y devuelve ÚNICAMENTE un JSON válido con esta estructura, sin texto adicional:

```
{"persona": "", "organización": "", "fecha": "", "lugar": ""}
```

Texto: "Ana García firmó el acuerdo con Telefónica el 15 de marzo en las oficinas de Madrid."



```
{  
  "persona": "Ana García",  
  "organización": "Telefónica",  
  "fecha": "15 de marzo",  
  "lugar": "Madrid"  
}
```


Role prompting

- Asignar un rol al modelo antes de la tarea condiciona el tono, el vocabulario, el nivel de detalle y la perspectiva de la respuesta
- No es solo estético: cambia qué conocimiento activa el modelo y cómo lo aplica
- Limitaciones
 - El modelo no tiene conocimiento que no tenga – el rol no añade datos, solo estilo y enfoque
 - Roles muy específicos ("eres el CTO de nuestra empresa") no aportan sin contexto adicional
 - El rol no garantiza precisión técnica: siempre verificar outputs críticos

Rol	Efecto en la respuesta
"Eres un abogado experto en GDPR"	Lenguaje técnico-legal, cita normativa
"Eres un profesor que explica a niños de 10 años"	Analogías simples, frases cortas
"Eres un analista financiero senior"	Métricas, ratios, perspectiva de riesgo
"Eres un redactor de atención al cliente"	Tono empático, orientado a solución

¿Cuál de estas definiciones describe mejor qué es un prompt?

- A) El parámetro de temperatura que controla la aleatoriedad del modelo
- B) La instrucción completa que recibe el modelo, incluyendo contexto, tarea y restricciones de formato
- C) El texto que genera el modelo como respuesta
- D) El nombre del archivo de configuración del pipeline

¿Cuál de estas definiciones describe mejor qué es un prompt?

- A) El parámetro de temperatura que controla la aleatoriedad del modelo
- B) La instrucción completa que recibe el modelo, incluyendo contexto, tarea y restricciones de formato
- C) El texto que genera el modelo como respuesta
- D) El nombre del archivo de configuración del pipeline

El zero-shot prompting es siempre la técnica más débil y se debe evitar en favor de few-shot siempre que sea posible.

Especificar explícitamente qué debe hacer el modelo cuando no tiene suficiente información para responder reduce el riesgo de alucinaciones.

¿En qué tipo de tarea aporta más mejora la técnica Chain-of-Thought respecto a un prompt directo?

- A) Clasificación de sentimiento de frases cortas
- B) Extracción de entidades en texto estructurado
- C) Razonamiento multi-paso: cálculos, deducciones lógicas, decisiones con varias condiciones
- D) Traducción de idiomas

¿En qué tipo de tarea aporta más mejora la técnica Chain-of-Thought respecto a un prompt directo?

- A) Clasificación de sentimiento de frases cortas
- B) Extracción de entidades en texto estructurado
- C) Razonamiento multi-paso: cálculos, deducciones lógicas, decisiones con varias condiciones
- D) Traducción de idiomas

Situación	Técnica recomendada
Tarea estándar y el modelo la conoce bien	Zero-shot
Formato de salida muy específico	Zero-shot + especificación de formato
Tarea poco común o con matices propios	Few-shot (2-5 ejemplos)
Razonamiento, cálculo, decisiones multi-paso	Chain-of-Thought
Output que va a procesar código	Salida estructurada (JSON)
Tono o perspectiva específica	Role prompting
Tarea compleja con varios pasos	Combinar CoT + role + formato

Añadir "Piensa paso a paso antes de responder"
al final de cualquier prompt siempre mejora el
resultado, independientemente de la tarea.

Le pides al modelo que devuelva un JSON con campos "nombre", "importe" y "fecha". El modelo responde: "Aquí tienes el resultado {"nombre": "Acme", "importe": 1200, "fecha": "2024-03-15"}"

¿Qué problema tiene esto en producción?

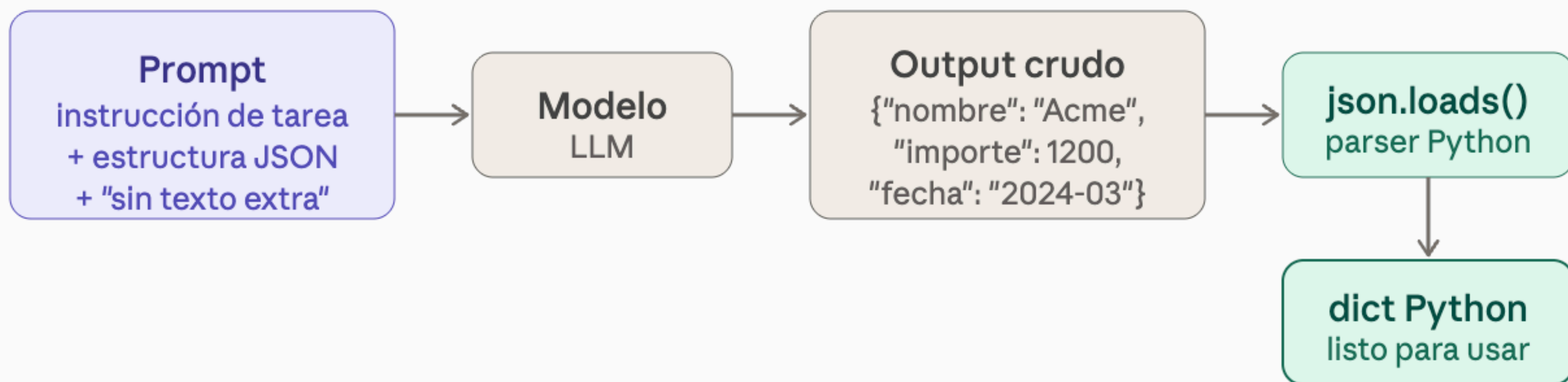
- A) Ninguno, el JSON es correcto
- B) El texto "Aquí tienes el resultado:" antes del JSON rompe el parser
- C) El modelo debería usar comillas simples en lugar de dobles
- D) El campo "importe" debería ser string, no número

Le pides al modelo que devuelva un JSON con campos "nombre", "importe" y "fecha". El modelo responde: "Aquí tienes el resultado {"nombre": "Acme", "importe": 1200, "fecha": "2024-03-15"}"

¿Qué problema tiene esto en producción?

- A) Ninguno, el JSON es correcto
- B) El texto "Aquí tienes el resultado:" antes del JSON rompe el parser
- C) El modelo debería usar comillas simples en lugar de dobles
- D) El campo "importe" debería ser string, no número

camino feliz



camino de error

Prevención

- Añadir "sin texto adicional" al prompt
- Mostrar estructura JSON vacía como ejemplo
- Usar structured outputs (API) si disponible
- Validar siempre antes de consumir el dict

Output con prefijo

Aquí el resultado:
{"nombre": "Acme"...}

ValueError

json.loads() falla

Sanear output

strip + extraer JSON

Asignar el rol "eres un médico especialista en cardiología" garantiza que las respuestas del modelo sean médicamente precisas y verificadas.

Un prompt pide al modelo: "Clasifica el tema, resume en 2 frases, sugiere tres etiquetas y evalúa el tono."

¿Cuál es el problema principal de este prompt?

- A) Es demasiado corto
- B) Mezcla cuatro tareas distintas sin separar el formato de cada salida
- C) No tiene role prompting
- D) Debería usar few-shot en lugar de zero-shot

Un prompt pide al modelo: "Clasifica el tema, resume en 2 frases, sugiere tres etiquetas y evalúa el tono."

¿Cuál es el problema principal de este prompt?

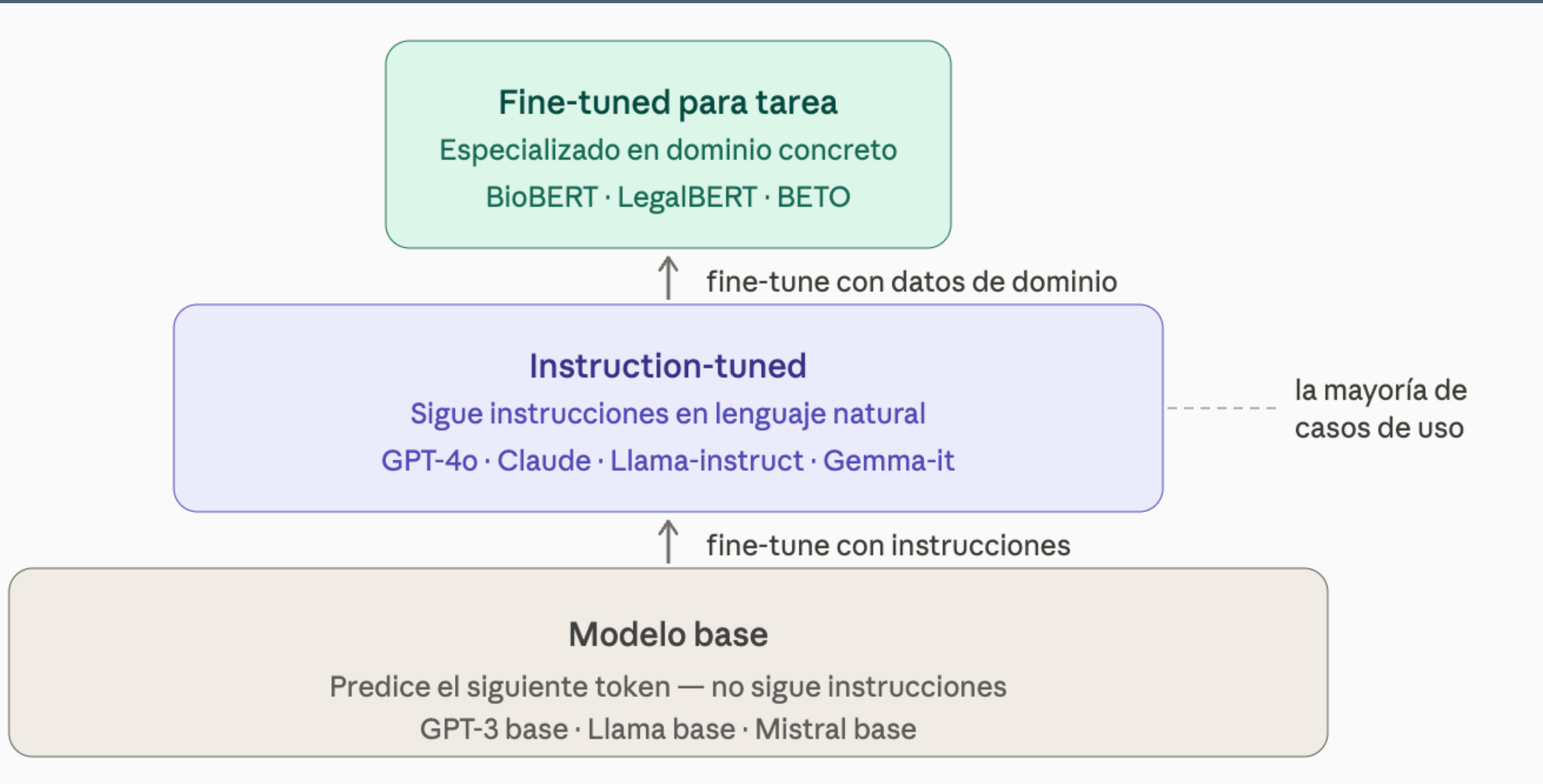
- A) Es demasiado corto
- B) Mezcla cuatro tareas distintas sin separar el formato de cada salida
- C) No tiene role prompting
- D) Debería usar few-shot en lugar de zero-shot

"Dame información útil sobre este documento" es un buen punto de partida para un sistema de producción.

Tenéis que implementar un sistema que clasifica emails de soporte en 5 categorías y debe devolverlo en un json

¿Qué técnicas de las vistas combinaríais?

El ecosistema de LLMs



el mapa

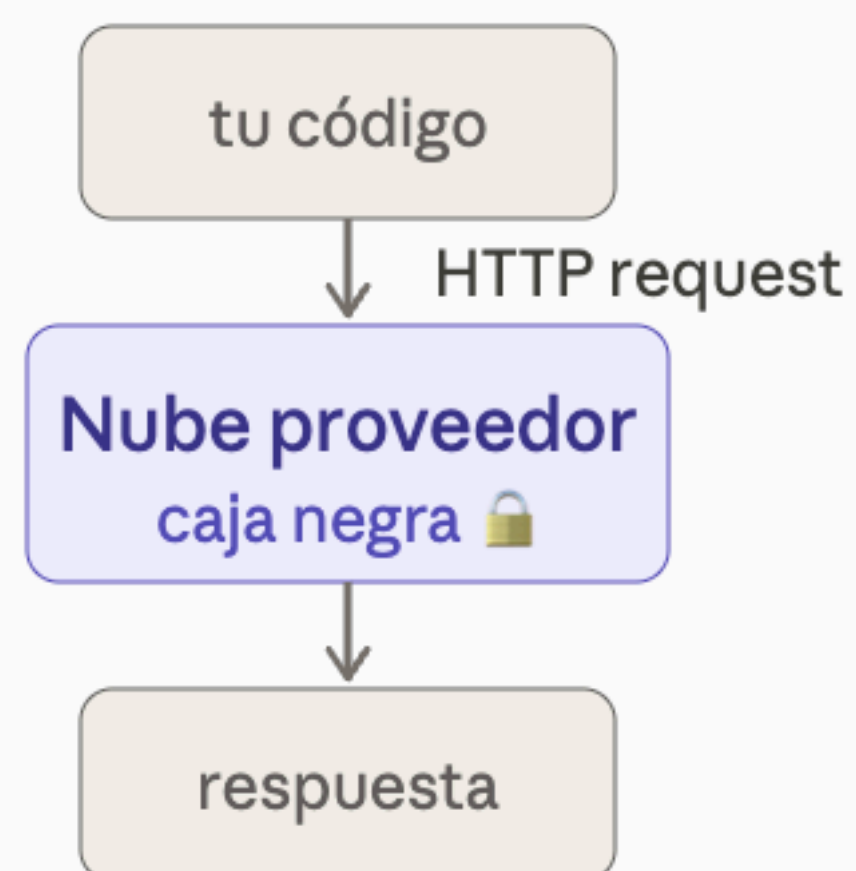
qué hay disponible, cómo se usa y cuándo tiene sentido

- **Modelos base** – se entrenan con enormes cantidades de texto. No siguen instrucciones, solo predicen el siguiente token. No son útiles directamente
- **Modelos instruction-tuned** – se ajustan sobre el modelo base para seguir instrucciones en lenguaje natural
- **Modelos fine-tuned para tarea específica** – se ajustan adicionalmente con datos de un dominio concreto. Los más especializados del HuggingFace Hub que vimos en el módulo anterior
- La mayoría de los casos de uso reales viven en la segunda capa. La tercera solo tiene sentido cuando la segunda falla sistemáticamente en tu dominio

2 formas de acceder a un LLM

- Hay dos caminos radicalmente distintos para usar un LLM en producción:
- Vía API
 - No tienes acceso a los pesos, no puedes modificar el modelo
 - Pagas por token procesado
- Vía modelo propio (local o cloud)
 - Descargas los pesos del modelo y lo ejecutas tú
 - Tienes control total: puedes hacer fine-tuning, cuantizar, modificar

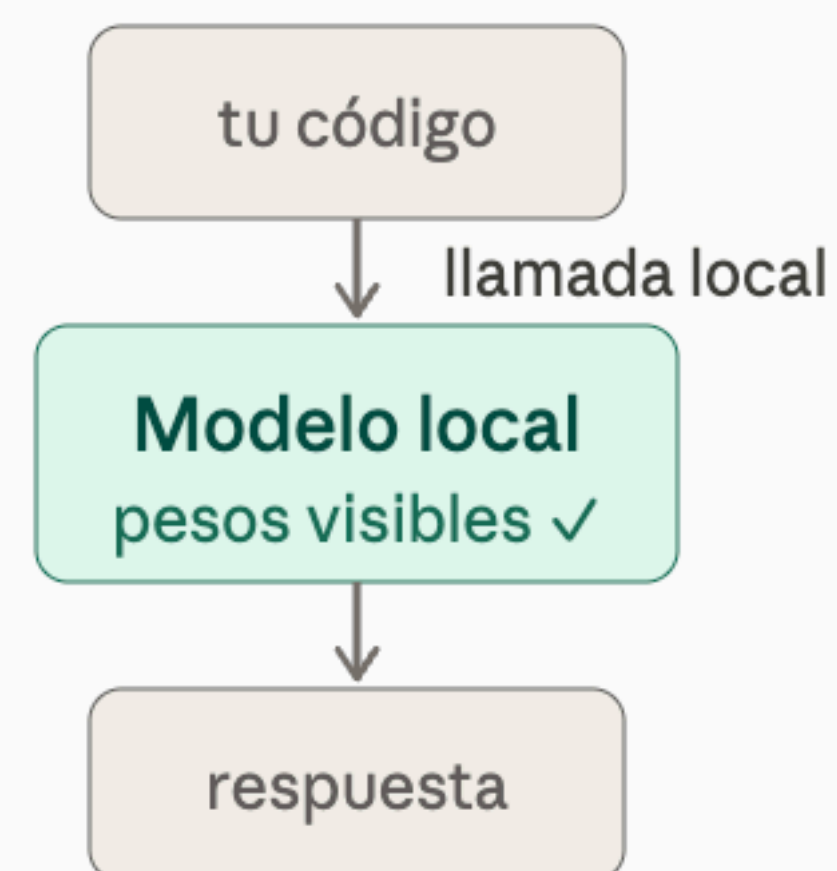
API comercial



- ✓ Sin coste inicial
- ✓ Sin mantenimiento
- ✓ Modelos muy grandes
- ✗ Datos salen de tu infra
- ✗ Coste crece con volumen
- ✗ Sin control del modelo

vs

Modelo local / open weights



- ✓ Datos no salen de tu infra
- ✓ Coste fijo a escala
- ✓ Control y personalización total
- ✗ Coste inicial de hardware
- ✗ Tú gestionas versiones
- ✗ Calidad inferior en tareas generales

Fine-tuning: cuándo tiene sentido y cuándo no

- Fine-tuning significa continuar el entrenamiento de un modelo pre-entrenado con tus propios datos. No es reentrenar desde cero.
- Cuándo tiene sentido:
 - El modelo necesita aprender un estilo muy específico que el prompting no consigue (tono de marca, formato propietario)
 - El dominio tiene vocabulario o convenciones que el modelo base desconoce (jerga médica muy específica, sistema interno de codificación)
 - Necesitas reducir el tamaño del prompt en producción para ahorrar costes a escala
 - La tarea es siempre la misma y muy repetitiva – fine-tuning la hace más rápida y barata que prompting

- Cuándo NO tiene sentido:
 - El prompting zero-shot o few-shot ya da buenos resultados – fine-tuning añade coste sin beneficio
 - No tienes suficientes datos etiquetados de calidad (mínimo realista: varios cientos de ejemplos, idealmente miles)
 - La tarea cambia frecuentemente – el modelo fine-tuned se vuelve obsoleto
 - Quieres que el modelo sepa cosas nuevas – fine-tuning no añade conocimiento, solo ajusta comportamiento

Fine-tuning no es la forma de darle información al modelo sobre tu empresa. Para eso está RAG. Fine-tuning es para cambiar cómo se comporta, no qué sabe.

Ventajas y limitaciones de los LLMs

- **1. Generalización sin datos etiquetados** Un modelo pre-entrenado puede clasificar, extraer o resumir en dominios que nunca ha visto explícitamente, solo con una instrucción. Esto es el zero-shot que haremos hoy.
- **2. Comprensión de lenguaje natural imperfecto** Los tickets de soporte reales tienen erratas, abreviaciones y frases incompletas. Un LLM los entiende; un clasificador clásico entrenado con texto limpio falla.
- **3. Flexibilidad de tarea con el mismo modelo** El mismo modelo que clasifica tickets hoy puede resumirlos mañana y generar respuestas pasado. Sin reentrenar, sin nuevo pipeline.
- **4. Generación de texto coherente y adaptable** Puede generar texto en distintos tonos, formatos y longitudes según instrucción. Útil para borradores, informes, datos sintéticos.
- **5. Extracción de estructura desde texto no estructurado** Convertir prosa en JSON, tablas o campos específicos. Conecta el lenguaje natural con el stack de datos.

... lo que hacen mal (desventajas)

- Alucinaciones
- No determinismo
- Sesgo en los datos de entrenamiento
- Fecha de corte
- Coste computacional
- Contexto limitado

Tres patrones que cubren el 80% de los casos de empresa

- Casi cualquier aplicación de LLMs en contexto empresarial es una combinación de tres patrones básicos. Los tres los vais a practicar hoy con el mismo dataset.
 - **Generación** – el modelo produce texto nuevo a partir de una instrucción y contexto
 - **Resumen** – el modelo condensa texto largo en algo más corto y accionable
 - **Extracción estructurada** – el modelo convierte texto libre en datos estructurados
- Son distintos en lo que producen pero comparten la misma mecánica: un prompt bien diseñado y una llamada al modelo. La diferencia está en qué le pides y cómo validas el output.

Generación

- Qué es: el modelo produce texto que no existía antes – respuestas, borradores, datos sintéticos, descripciones.
- Casos de uso en empresa:
 - Borradores de respuesta para tickets de soporte (lo haremos hoy)
 - Generación de datos sintéticos para entrenar clasificadores (lo haremos hoy)
 - Redacción de informes ejecutivos a partir de datos
 - Generación de variantes de copy para tests A/B
 - Resumen de reuniones a partir de transcripciones

- Lo que hay que controlar:
 - **Tono y longitud:** sin especificar, el modelo elige. En producción siempre se especifica.
 - **Alucinaciones de hechos:** el modelo puede inventar datos, cifras, nombres. Nunca usar generación para producir contenido factual sin verificación.
 - **Consistencia entre llamadas:** el mismo prompt puede dar respuestas distintas. Si necesitas consistencia, baja la temperatura.
- Cuándo NO usar generación: cuando la salida va a ser leída por un cliente sin revisión humana y contiene información factual crítica. Un borrador sin revisar enviado automáticamente es un riesgo reputacional.

Resumen

- Qué es: el modelo condensa texto largo en algo más corto preservando la información relevante. Dos tipos con usos distintos:
 - **Extractivo:** selecciona y combina frases del texto original. Más fiel, menos fluido.
 - **Abstractivo:** reescribe con sus propias palabras. Más fluido, puede perder matices o introducir errores.
- Los LLMs hacen resumen abstractivo por defecto.
- Casos de uso en empresa:
 - Resumen de tickets de soporte para informes semanales (lo haremos hoy)
 - Condensar hilos de email largos en puntos de acción
 - Resumen de documentos legales o contratos para revisión rápida
 - Generar descripciones cortas de productos desde fichas técnicas largas
 - Sintetizar feedback de clientes por categoría

- Lo que hay que controlar:
 - **Qué se pierde:** el resumen siempre omite información. Hay que definir qué es relevante para tu caso de uso y especificarlo en el prompt
 - **Alucinaciones por completar:** a veces el modelo añade información que no estaba en el original para hacer el resumen más coherente. Con textos técnicos esto es especialmente peligroso.
 - **Longitud del contexto:** si el texto original es muy largo hay que trocearlo. La calidad del resumen depende de cómo se trocee.

Extracción estructurada

- Qué es: el modelo lee texto libre y devuelve campos concretos en un formato estructurado (JSON, CSV, tabla).
- Es el caso de uso con más impacto inmediato en pipelines de datos porque conecta directamente lenguaje natural con bases de datos y DataFrames.
- Casos de uso en empresa:
 - Extraer producto, urgencia y cliente de tickets de soporte (lo haremos hoy)
 - Extraer fechas, partes y condiciones de contratos
 - arsear CVs en campos estructurados para sistemas de RRHH
 - Extraer métricas de informes financieros en texto
 - Normalizar direcciones o nombres de empresa con variantes ortográficas

- Lo que hay que controlar:
 - **Campos inventados:** si el campo no está en el texto, el modelo tiende a inventar un valor plausible. Hay que instruir explícitamente `null` cuando no hay información.
 - **Formato de salida no válido:** el modelo puede añadir texto fuera del JSON. Siempre sanear antes de parsear.
 - **Tipos de datos:** el modelo puede devolver "1200€" como string cuando necesitas float. Especificar el tipo en el prompt o transformar en post-proceso.
 - **Validación post-extracción:** comparar el campo extraído con el texto original para detectar invenciones. Automatizable con una búsqueda de substring.

Texto libre

Hola, llevo tres días sin poder acceder a mi cuenta. El error dice "token inválido". Es urgente, soy Ana García de Acme S.L.

LLM
+ prompt

JSON

```
{  
  "producto": "login",  
  "urgencia": "alta",  
  "cliente": "A. García",  
  "accion": "reset token"  
}
```

json.load

DataFrame

producto	urgencia
login	alta
factura	media
exportar	baja
...	...

¿Cuál es la diferencia principal entre un modelo base y un modelo instruction-tuned?

- A) El modelo base es más grande y tiene más parámetros
- B) El modelo base solo predice el siguiente token; el instruction-tuned ha sido ajustado para seguir instrucciones
- C) El modelo base es de código abierto; el instruction-tuned es siempre de pago
- D) El modelo base funciona en local; el instruction-tuned solo en la nube

Hacer fine-tuning sobre un modelo pre-entrenado es la forma recomendada de darle a un LLM información actualizada sobre tu empresa o documentos internos.

¿En qué situación tiene más sentido invertir en fine-tuning en lugar de mejorar el prompting?

- A) Cuando el modelo no conoce eventos recientes
- B) Cuando la tarea cambia frecuentemente y necesitas flexibilidad
- C) Cuando el modelo necesita aprender un formato de salida muy específico y repetitivo que el prompting no consigue de forma consistente
- D) Cuando no tienes suficientes datos etiquetados

¿En qué situación tiene más sentido invertir en fine-tuning en lugar de mejorar el prompting?

- A) Cuando el modelo no conoce eventos recientes
- B) Cuando la tarea cambia frecuentemente y necesitas flexibilidad
- C) Cuando el modelo necesita aprender un formato de salida muy específico y repetitivo que el prompting no consigue de forma consistente
- D) Cuando no tienes suficientes datos etiquetados

En la práctica, la mayoría de casos de uso empresariales se resuelven con modelos instruction-tuned accedidos vía API o en local, sin necesidad de fine-tuning.

Una empresa del sector sanitario quiere procesar historiales clínicos con un LLM.

¿Qué consideración es determinante para elegir entre API comercial y modelo local?

- A) El precio por token de la API
- B) Que los datos de pacientes no pueden salir de la infraestructura de la empresa
- C) Que los modelos locales son siempre más precisos en texto médico
- D) Que las APIs comerciales no permiten procesar documentos largos

Una empresa del sector sanitario quiere procesar historiales clínicos con un LLM.

¿Qué consideración es determinante para elegir entre API comercial y modelo local?

- A) El precio por token de la API
- B) Que los datos de pacientes no pueden salir de la infraestructura de la empresa
- C) Que los modelos locales son siempre más precisos en texto médico
- D) Que las APIs comerciales no permiten procesar documentos largos

¿Cuál de estos perfiles describe mejor el caso de uso ideal para una API comercial frente a un modelo local?

- A) Alto volumen de llamadas con datos confidenciales y presupuesto ajustado
- B) Sistema en producción con requisitos estrictos de latencia y coste fijo
- C) Dominio muy especializado que requiere fine-tuning frecuente
- D) Prototipado rápido, datos no sensibles y equipo sin infraestructura GPU

¿Cuál de estos perfiles describe mejor el caso de uso ideal para una API comercial frente a un modelo local?

- A) Alto volumen de llamadas con datos confidenciales y presupuesto ajustado
- B) Sistema en producción con requisitos estrictos de latencia y coste fijo
- C) Dominio muy especializado que requiere fine-tuning frecuente
- D) Prototipado rápido, datos no sensibles y equipo sin infraestructura GPU

Estás generando borradores de respuesta automáticos para tickets de soporte que se enviarán directamente a clientes sin revisión humana
¿Cuál es el riesgo principal?

- A) El modelo genera respuestas demasiado cortas
- B) El modelo puede inventar información factual – números de teléfono, plazos, políticas – que no existe
- C) Los clientes prefieren respuestas escritas por humanos independientemente del contenido
- D) El modelo no puede mantener un tono profesional

Estás generando borradores de respuesta automáticos para tickets de soporte que se enviarán directamente a clientes sin revisión humana
¿Cuál es el riesgo principal?

- A) El modelo genera respuestas demasiado cortas
- B) El modelo puede inventar información factual – números de teléfono, plazos, políticas – que no existe
- C) Los clientes prefieren respuestas escritas por humanos independientemente del contenido
- D) El modelo no puede mantener un tono profesional

Integración en Flujos de Ciencia de Datos

Aumentación de datos sintéticos

- El entrenamiento de modelos de Machine Learning en NLP siempre ha dependido de una condición que pocas veces se cumple con facilidad: **datos etiquetados en cantidad suficiente, con distribución representativa y calidad aceptable.**
- En la práctica, los datasets reales presentan tres tipos de desequilibrio que degradan la capacidad generalizadora de cualquier modelo:
 - **Desequilibrio de clases:** en análisis de sentimientos sobre reseñas de Amazon, por ejemplo, las opiniones de 4 y 5 estrellas superan masivamente a las de 1 y 2. Un modelo entrenado así aprende el sesgo de su corpus, no la realidad del problema.
 - **El problema del long tail:** en catálogos de productos, miles de ítems tienen una o dos reseñas. No es posible entrenar un clasificador de aspectos con un solo ejemplo por clase.
 - **Dominio estrecho:** un modelo entrenado en reseñas de electrónica falla al aplicarse a reseñas de restaurantes o de libros, aunque la tarea (clasificar sentimiento) sea la misma. Los datos no cubren el espacio semántico necesario.

- La respuesta clásica a estos problemas era la **anotación manual**: pagar a anotadores humanos para que produjeran más ejemplos etiquetados. Es cara, lenta y difícilmente escalable. Con los LLMs, aparece una alternativa viable: **generación programática de datos sintéticos**.

- La **aumentación de datos sintéticos** (*synthetic data augmentation*) consiste en utilizar un modelo generativo –en este contexto, un LLM– para producir nuevos ejemplos de entrenamiento que sean:
 - **Semánticamente plausibles:** el texto generado debe leerse como si pudiera haber sido escrito por un humano real.
 - **Controlados en su etiqueta:** el dato generado debe corresponder a la clase deseada (positivo, negativo, aspecto X, intención Y), no generarse de forma aleatoria.
 - **Suficientemente diversos:** la aumentación solo tiene valor si los ejemplos nuevos aportan variabilidad léxica y estructural, no si son paráfrasis triviales del mismo ejemplo original.

- La distinción conceptual clave es entre dos estrategias:
 - Paráfrasis controlada: Reescribe un ejemplo existente manteniendo su significado y etiqueta. Usar cuando tienes pocos ejemplos reales y necesitas multiplicarlos
 - Generación desde especificación: Crea ejemplos desde cero a partir de una descripción (producto, aspecto, polaridad). Usar cuando directamente no tienes datos en esa clase o dominio
- Ambas estrategias se implementan con prompts estructurados. La diferencia está en si el LLM recibe un texto de partida o solo una especificación abstracta.

- El mecanismo que hace funcionar la generación sintética de calidad es el **few-shot prompting**: incluir en el prompt 2-4 ejemplos reales de la clase que queremos generar antes de pedir el nuevo ejemplo. Esto sirve como ancla estilística y semántica.
- Un prompt típico para generar reseñas sintéticas de auriculares con polaridad negativa sobre el aspecto *duración de batería* tendría esta estructura:
- El LLM, condicionado por los ejemplos, genera texto que mantiene el dominio (auriculares), el aspecto (batería) y la polaridad (negativa), pero con vocabulario y estructura distintos. **Eso es exactamente lo que necesita un clasificador para generalizar mejor.**

Eres un cliente que ha comprado auriculares inalámbricos.
Escribe una reseña de 2-3 frases centrada en el aspecto BATERÍA
con sentimiento NEGATIVO.
Usa un tono natural, como si fuera un comprador real.

Ejemplos de referencia:

- "La batería no dura ni 4 horas. Para el precio que tiene es una vergüenza."
- "Decepciona mucho que tengas que cargarlo cada tarde si lo usas en el trabajo."

Nueva reseña:

Consideraciones de calidad y filtrado

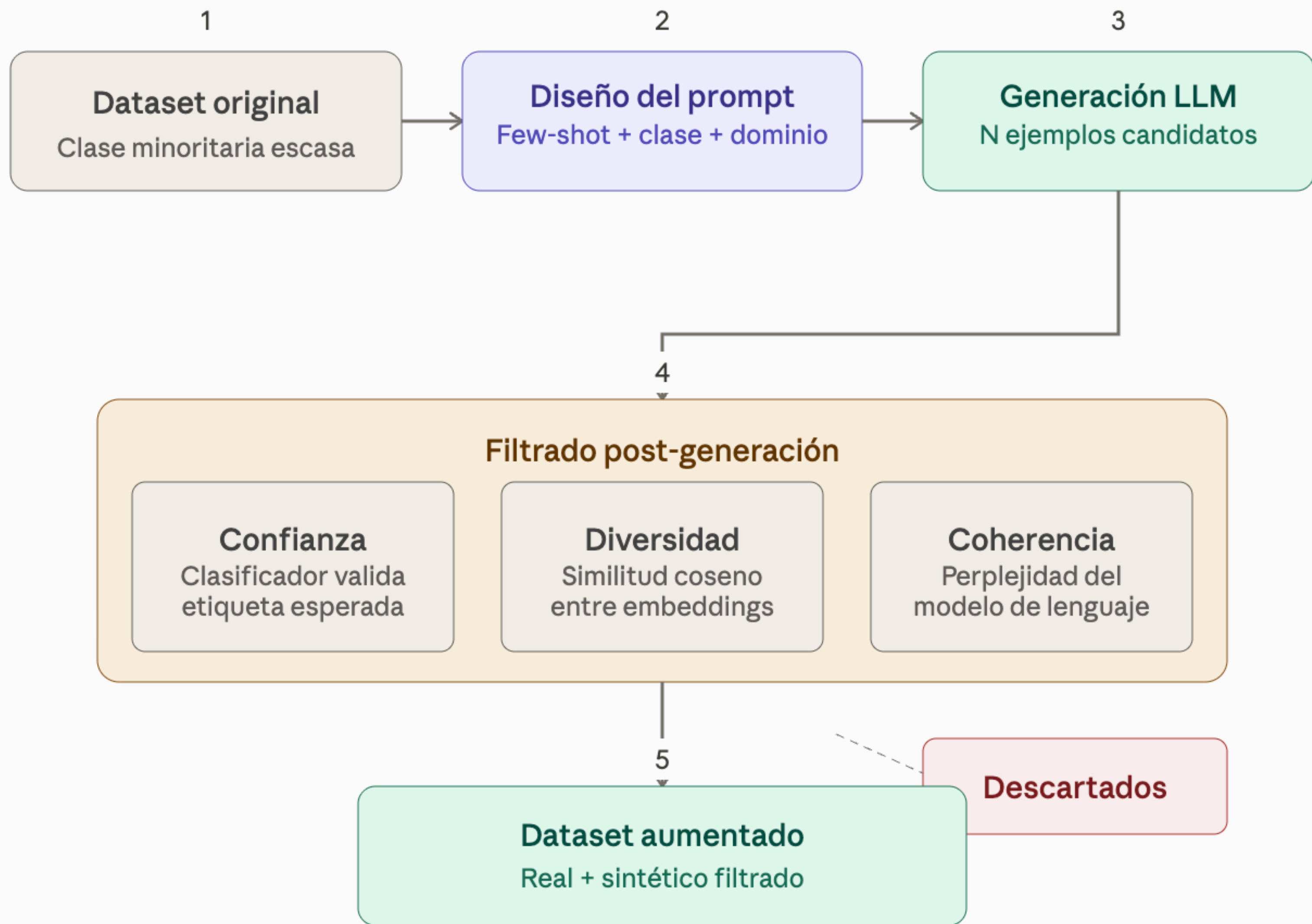
- No todos los ejemplos generados son igualmente útiles. La generación sintética sin supervisión produce, junto con ejemplos valiosos, ruido que puede perjudicar el entrenamiento. Los problemas más frecuentes son:
 - **Etiqueta-texto inconsistente:** el LLM genera un texto que no corresponde a la clase pedida. ("Batería negativa" puede producir *"la batería es sorprendentemente buena"* si el prompt está mal diseñado).
 - **Poca diversidad léxica:** el modelo reutiliza las mismas estructuras sintácticas de los ejemplos few-shot.
 - **Textos genéricos o vacíos:** frases que podrían pertenecer a cualquier clase (*"es un producto normal, cumple su función"*).

- Por ello, un pipeline de aumentación de datos sintéticos robusto siempre incluye una **etapa de filtrado post-generación**, que puede ser:
 - **Filtrado por classifier de confianza:** pasar los ejemplos generados por un modelo ya entrenado y descartar los que tengan baja confianza en la clase esperada.
 - **Filtrado por similitud semántica:** eliminar ejemplos demasiado parecidos (cosine similarity > 0.95 entre embeddings) para garantizar diversidad.
 - **Filtrado por perplejidad:** descartar textos con perplejidad anómalamente alta o baja (que indican incoherencia o texto demasiado formulaico).

Cuándo tiene sentido usar datos sintéticos

- La aumentación sintética **no es una solución universal**. Tiene un coste (llamadas a API o inferencia local) y un riesgo (introducir ruido). Los escenarios donde su valor es más claro:
 - **Clases minoritarias en un dataset desequilibrado:** en lugar de técnicas clásicas como SMOTE (que solo funciona en espacio de features), generar ejemplos textuales reales de la clase minority.
 - **Bootstrapping de un nuevo dominio:** cuando no existe ningún dataset etiquetado en el dominio objetivo y se necesita un punto de partida.
 - **Evaluación robusta:** generar casos edge o adversariales para testear la robustez de un modelo antes de ponerlo en producción.
 - **Privacidad:** cuando los datos reales contienen información sensible (datos médicos, financieros) y no pueden compartirse, los datos sintéticos pueden servir como proxy.

- Lo que **no** resuelve: no sustituye a datos reales cuando hay suficientes, y no compensa sesgos sistemáticos del dataset original (si todos los ejemplos reales son de un dialecto, los sintéticos heredarán ese sesgo).



Etiquetado de Datos a Gran Escala: Zero-Shot Labeling

- El etiquetado manual de datos es el cuello de botella histórico del Machine Learning supervisado. Proyectos como ImageNet requirieron años de trabajo de anotación distribuida.
- En NLP, tareas aparentemente sencillas –clasificar el aspecto de una reseña, detectar la intención de un mensaje de soporte– requieren anotadores especializados, guías de etiquetado detalladas, y rondas de revisión de consistencia.
- La consecuencia práctica: **la mayoría de los datasets en producción están sin etiquetar o parcialmente etiquetados**. Las empresas acumulan millones de textos –tickets de soporte, reseñas, correos, formularios – que no pueden aprovechar para entrenar modelos porque etiquetarlos manualmente es inviable.
- Los LLMs ofrecen una salida a este dilema a través del **zero-shot labeling**: usar un modelo de lenguaje preentrenado para asignar etiquetas a datos nuevos sin necesidad de ejemplos específicos del dominio.

Qué es el Zero-Shot Labeling

- El **zero-shot labeling** es la aplicación de modelos de clasificación zero-shot para etiquetar automáticamente grandes volúmenes de texto. A diferencia del entrenamiento supervisado clásico, no requiere ningún ejemplo etiquetado del dominio objetivo: el modelo infiere la etiqueta a partir de su comprensión del lenguaje y del significado de la propia etiqueta.
- El paradigma más extendido para esto en la práctica real es la **Natural Language Inference (NLI)** aplicada como clasificador:
 - Se formula la clasificación como un **problema de inferencia**: ¿este texto es "compatible" con esta hipótesis (etiqueta)?
 - Modelos como bart-large-mnli o DeBERTa-v3 han sido entrenados en datasets de NLI y pueden, sin ajuste adicional, clasificar texto en categorías arbitrarias definidas como texto libre.

Texto de entrada: "Los cascos se descargan en 3 horas,
inaceptable para el precio"

Hipótesis 1: "Esta reseña trata sobre la batería" →
score: 0.91

Hipótesis 2: "Esta reseña trata sobre el sonido" →
score: 0.12

Hipótesis 3: "Esta reseña trata sobre el precio" →
score: 0.74

Hipótesis 4: "Esta reseña trata sobre el diseño" →
score: 0.08

Etiqueta asignada: BATERÍA (max score)

- El modelo no ha visto ningún ejemplo de "reseña sobre batería" durante su uso: la etiqueta emerge de la semántica de la propia hipótesis.
- El zero-shot labeling con NLI ocupa un nicho específico: **gran volumen, sin datos etiquetados disponibles, restricción de coste**. No produce la calidad de la anotación manual, pero permite pasar de cero a un dataset funcional de decenas de miles de ejemplos en horas.

Uso estratégico: Zero-Shot como bootstrapping

- El patrón más potente en la práctica no es usar el zero-shot labeling como etiquetado definitivo, sino como **primera iteración de un ciclo activo**:
 - 1. **Zero-shot labeling** sobre el corpus completo → dataset ruidoso pero masivo.
 - 2. **Entrenamiento de un clasificador ligero** (DistilBERT fine-tuned) sobre ese dataset ruidoso.
 - 3. **Identificación de ejemplos de baja confianza** con el clasificador entrenado.
 - 4. **Revisión manual selectiva** solo de esos ejemplos dudosos (active learning).
 - 5. **Re-entrenamiento** con el dataset limpio.

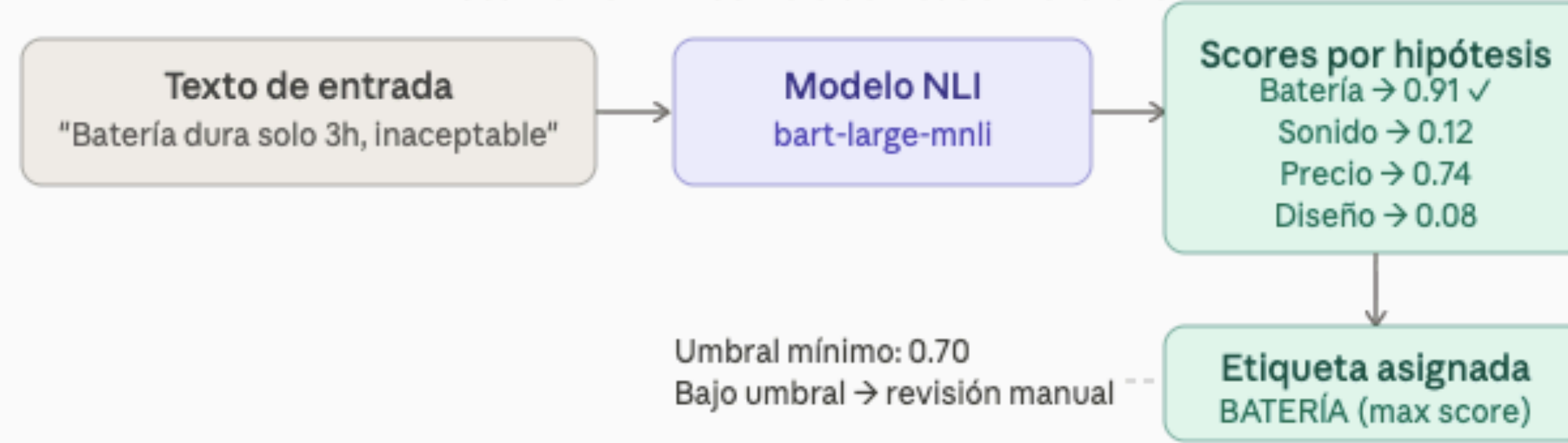
Uso estratégico: Zero-Shot como bootstrapping

- El zero-shot labeling no es infalible. Sus limitaciones principales son:
 - **Dependencia de la formulación de hipótesis:** cambiar ligeramente el wording de la etiqueta puede cambiar significativamente los scores. Requiere experimentación para encontrar las formulaciones óptimas.
 - **Sensibilidad al dominio:** los modelos NLI están entrenados en textos formales. En textos muy coloquiales, con argot o abreviaciones, la calidad del etiquetado cae.
 - **Escalabilidad de cómputo**
 - **No detecta lo que no sabe:** si una categoría relevante no está en el espacio de hipótesis definido, el modelo no la descubrirá. Es un clasificador cerrado, no un detector de tópicos exploratorio.

Conexión con el flujo completo

- Estos dos bloques –aumentación sintética y etiquetado zero-shot– no son técnicas aisladas. Son las piezas que cierran el ciclo del científico de datos cuando trabaja con texto en condiciones reales:
 - El **módulo de transformers** (sesiones anteriores) proporcionó los modelos que hacen posible ambas técnicas.
 - El **prompt engineering** es el mecanismo de control de la generación sintética.
 - El **zero-shot labeling** produce los datasets etiquetados que antes requerían meses de anotación manual.
 - Los datasets así contruidos alimentan directamente el **fine-tuning** de modelos especializados, o sirven como base de conocimiento para el **chatbot RAG** que se desarrollará en la siguiente sesión.
- Los LLMs no solo son modelos que se consultan en producción: son también **herramientas de construcción de datasets**, cambiando radicalmente el coste y el tiempo necesarios para poner en marcha un pipeline de NLP supervisado desde cero.

Mecanismo: NLI como clasificador zero-shot



Ciclo de bootstrapping activo



Retrieval-Augmented Generation (RAG)

- Los LLMs tienen conocimiento congelado en el tiempo: no saben nada de lo que ocurrió después de su fecha de corte de entrenamiento
- Afinar un modelo para incorporar conocimiento nuevo es costoso y no escala: cada actualización del corpus requiere **reentrenamiento**
- Los LLMs alucinan cuando se les pregunta sobre hechos específicos que no están en sus pesos – generan respuestas plausibles pero incorrectas
- **RAG separa el conocimiento del razonamiento**: el LLM aporta la capacidad de generar lenguaje natural; una base de datos externa aporta los hechos concretos
- Resultado: respuestas fundamentadas en **documentos reales, verificables y actualizables** sin tocar el modelo

- **RAG** (*Retrieval-Augmented Generation*) es una arquitectura que combina **búsqueda de información** (*retrieval*) con **generación de texto** (*generation*)
- Formulación original: Lewis et al., 2020 (Meta AI) – **propuesta como alternativa al fine-tuning** para tareas de conocimiento intensivo
- La idea central: **antes de generar una respuesta, el sistema recupera los fragmentos de texto más relevantes para la pregunta y los incluye en el contexto del LLM**
- El LLM no necesita "saber" la respuesta de memoria: la lee en el contexto que le pasamos
- Esto convierte cualquier corpus de documentos en una base de conocimiento consultable en lenguaje natural

Los tres componentes fundamentales:

- **Base de conocimiento:** conjunto de documentos del dominio (PDFs, páginas web, registros, reseñas, manuales...)
- **Motor de recuperación (*retriever*):** dado un query, localiza los fragmentos más relevantes del corpus
- **Modelo generativo (*generator*):** recibe el query + los fragmentos recuperados y produce la respuesta final en lenguaje natural

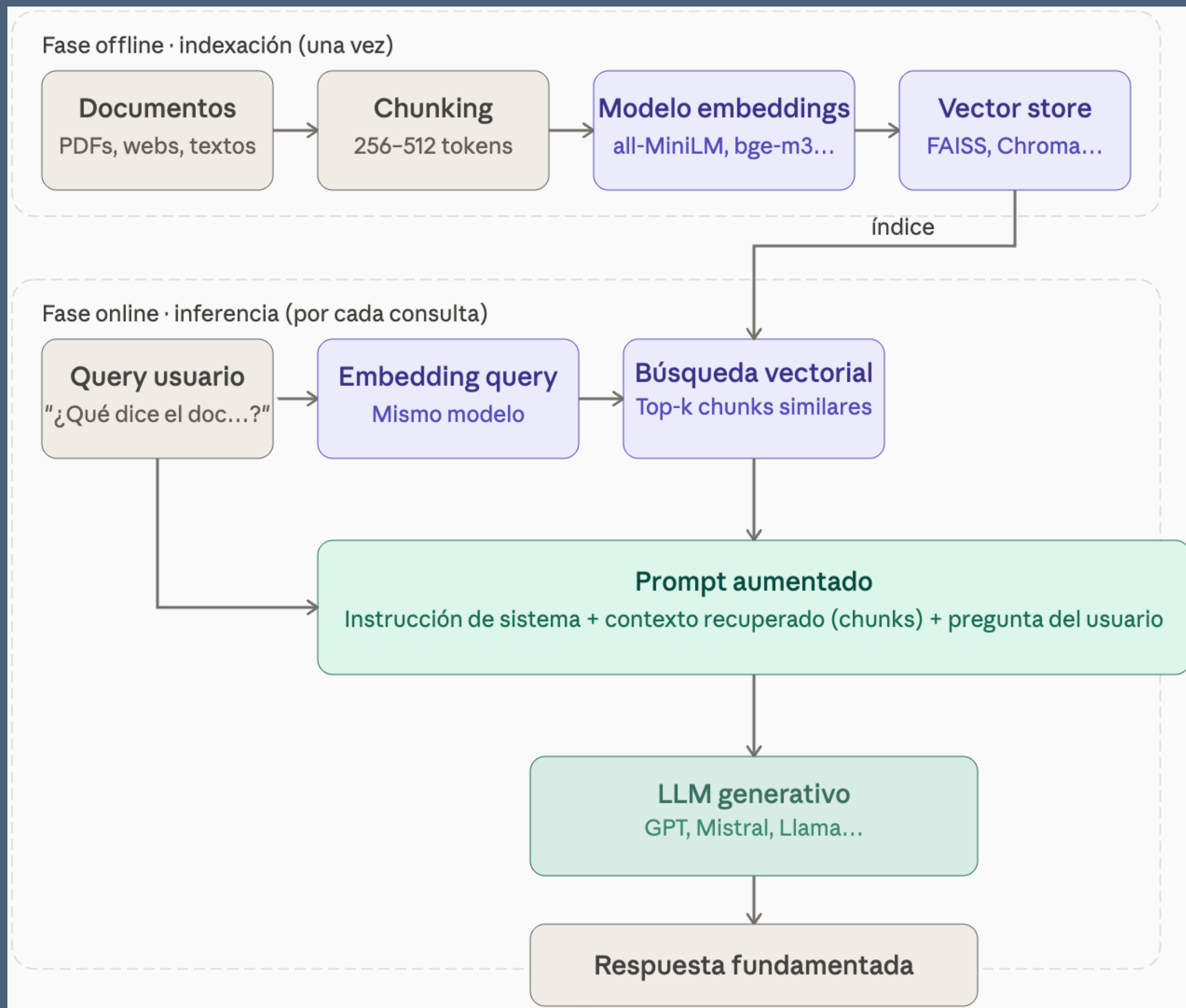
El pipeline RAG paso a paso

Fase offline (indexación) — se ejecuta una vez:

- Los documentos se dividen en fragmentos (*chunks*) de tamaño fijo o semántico
- Cada chunk se transforma en un *vector* denso mediante un *modelo de embeddings*
- Los vectores se almacenan en una *base de datos vectorial (vector store)*

Fase online (inferencia) — se ejecuta en cada consulta:

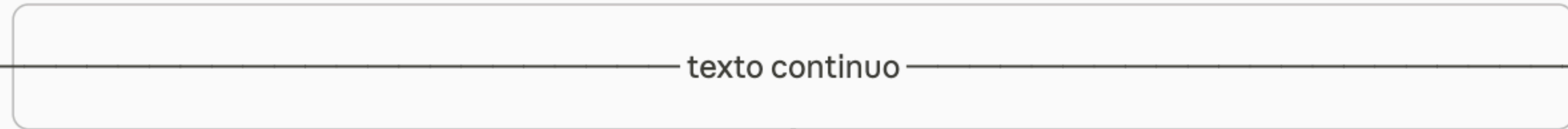
- La pregunta del usuario se transforma en un vector con el mismo modelo de embeddings
- Se buscan los k chunks más similares en la base vectorial (búsqueda por similitud coseno o producto escalar)
- Los chunks recuperados se concatenan al prompt como contexto
- El LLM genera la respuesta a partir del prompt aumentado



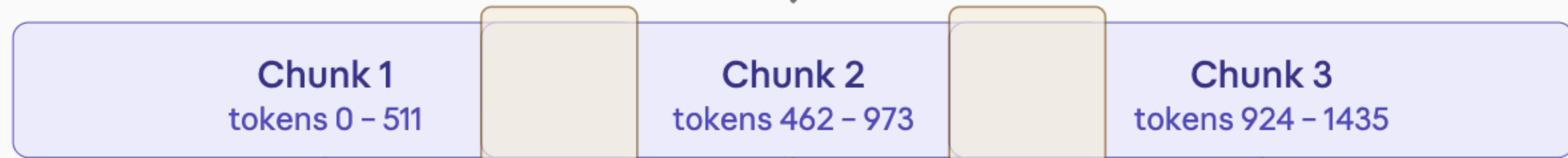
Chunking: dividir para recuperar

- Los documentos no se indexan enteros: se dividen en fragmentos porque los modelos de embeddings tienen ventana de contexto limitada y la búsqueda funciona mejor con unidades semánticas pequeñas
 - **Chunking por tamaño fijo:** dividir cada N tokens con **solapamiento (overlap)** entre chunks consecutivos para no perder contexto en los cortes
 - **Chunking semántico:** dividir por párrafos, secciones o frases completas, respetando la estructura natural del texto
- El **tamaño** del chunk afecta directamente la **calidad del retrieval**: chunks muy cortos pierden contexto; chunks muy largos diluyen la señal semántica
- Regla práctica: chunks de 256-512 tokens con solapamiento de 50-100 tokens funcionan bien como punto de partida

Documento original



chunking



overlap
50 tok.

overlap
50 tok.

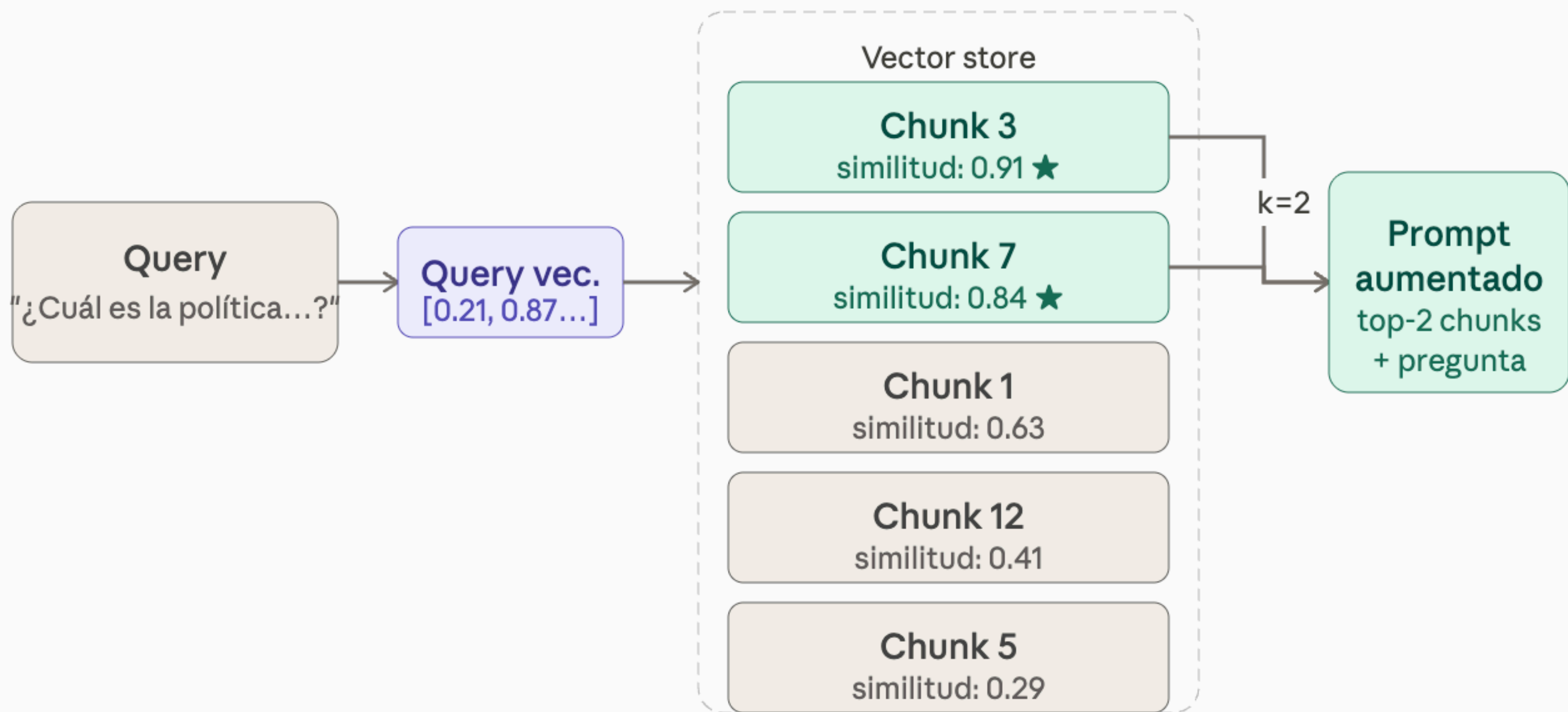
Embedding 1
vector 384d

Embedding 2
vector 384d

Embedding 3
vector 384d

Embeddings para recuperación

- El **retriever** convierte texto en vectores densos que capturan similitud semántica
- Un buen modelo de **embeddings para RAG** debe maximizar la similitud entre una pregunta y su respuesta correcta, no entre dos textos que hablan del mismo tema en general – esto se llama *asymmetric semantic search*
- La búsqueda vectorial encuentra chunks relevantes aunque no compartan ninguna palabra con la pregunta (sinónimos, reformulaciones, otro idioma)



Bases de datos vectoriales

- Una **base de datos vectorial** (*vector store*) almacena embeddings y permite búsqueda por similitud a alta velocidad, incluso sobre millones de vectores
- Opciones ligeras para desarrollo y uso local: **FAISS** (Meta, in-memory), **ChromaDB**, **SQLite-vec**
- Opciones para producción: **Pinecone**, **Weaviate**, **Qdrant**, **pgvector** (extensión de PostgreSQL)
- La búsqueda más común es *k-nearest neighbors aproximada (ANN)*: no garantiza los k vecinos exactos pero es órdenes de magnitud más rápida
- El índice vectorial se puede persistir en disco y actualizar sin reconstruirlo por completo al añadir documentos nuevos

- El corazón de RAG es la **construcción del prompt** que recibe el LLM, que tiene siempre tres partes:
 - **Instrucción de sistema:** define el rol del modelo y las reglas de comportamiento ("responde solo usando la información del contexto proporcionado")
 - **Contexto recuperado:** los k chunks más relevantes, concatenados y etiquetados con su origen
 - **Pregunta del usuario:** la consulta original
- La instrucción de **grounding** es crítica: sin ella el LLM puede ignorar el contexto y responder desde sus pesos, volviendo a alucinar
- Patrón habitual: "Si la respuesta no está en el contexto, indica que no tienes información suficiente"

Fallos comunes y cómo mitigarlos

- El retriever no encuentra el chunk correcto: Embeddings inadecuados o chunk demasiado grande. Cambiar modelo de embeddings; reducir tamaño de chunk
- El LLM ignora el contexto: Prompt mal diseñado. Reforzar instrucción de grounding
- El LLM "mezcla" contexto con memoria: Contexto insuficiente o ambiguo. Añadir más chunks; etiquetar origen de cada fragmento
- Respuestas lentas: Índice no optimizado o LLM pesado.
- Chunks duplicados en el contexto: Corpus con documentos redundantes. Hay que deduplicar en la fase de indexación

- RAG es un **cambio de paradigma en cómo se despliegan los LLMs en producción**
- **Desacopla el conocimiento del modelo:** actualizar la base de conocimiento no requiere tocar los pesos
- Hace los sistemas verificables: **cada respuesta puede trazarse hasta los documentos fuente**
- Es el puente entre los LLMs de propósito general y los asistentes especializados en un dominio concreto
- La mayor parte de los sistemas de IA conversacional en producción hoy (chatbots de soporte, asistentes documentales, sistemas de Q&A corporativos) usan alguna variante de RAG

¿Cuál es la idea central que distingue RAG de un LLM estándar?

- a) El LLM se reentrena continuamente con nuevos documentos
- b) Antes de generar, el sistema recupera fragmentos relevantes y los incluye en el contexto
- c) El modelo aprende a citar fuentes durante el fine-tuning
- d) RAG usa varios LLMs en paralelo y fusiona sus respuestas

¿Cuál es la idea central que distingue RAG de un LLM estándar?

- a) El LLM se reentrena continuamente con nuevos documentos
- b) Antes de generar, el sistema recupera fragmentos relevantes y los incluye en el contexto
- c) El modelo aprende a citar fuentes durante el fine-tuning
- d) RAG usa varios LLMs en paralelo y fusiona sus respuestas

¿Qué efecto tiene usar chunks demasiado grandes?

- a) El modelo de embeddings no puede procesarlos
- b) La búsqueda vectorial tarda más en ejecutarse
- c) La señal semántica se diluye: el embedding representa mal el tema concreto del fragmento
- d) Los chunks grandes mejoran siempre la calidad de la respuesta final

¿Qué efecto tiene usar chunks demasiado grandes?

- a) El modelo de embeddings no puede procesarlos
- b) La búsqueda vectorial tarda más en ejecutarse
- c) La señal semántica se diluye: el embedding representa mal el tema concreto del fragmento
- d) Los chunks grandes mejoran siempre la calidad de la respuesta final

¿Por qué es imprescindible usar el mismo modelo de embeddings para indexar y para transformar el query?

- a) Por una restricción técnica de FAISS que no permite mezclar modelos
- b) Porque los espacios vectoriales de distintos modelos no son comparables: las similitudes calculadas entre ellos no tienen significado
- c) Para reducir la latencia de inferencia
- d) Porque los modelos de embeddings están entrenados solo en preguntas, no en documentos

¿Por qué es imprescindible usar el mismo modelo de embeddings para indexar y para transformar el query?

- a) Por una restricción técnica de FAISS que no permite mezclar modelos
- b) Porque los espacios vectoriales de distintos modelos no son comparables: las similitudes calculadas entre ellos no tienen significado
- c) Para reducir la latencia de inferencia
- d) Porque los modelos de embeddings están entrenados solo en preguntas, no en documentos

outro